A Multi-Platform Specification Language, Transformation, and Dataset for DevOps Pipelines

Manas Manoj Bedekar

Department of Electrical and Computer Engineering

McGill University, Montreal

May, 2025

A thesis submitted to McGill University in partial fulfillment of the requirements of the degree of

Master of Science

©Manas Manoj Bedekar, May, 2025

Abstract

Fast software delivery and high quality are crucial in today's competitive market, leading many organizations to adopt DevOps practices to improve their Continuous Integration and Continuous Delivery (CI/CD) pipelines. Nevertheless, the introduction of various platforms like GitHub Actions, GitLab CI, Bitbucket Pipelines, Bamboo, Circle CI, and Azure DevOps each with their own pipeline dialect have made inter-dialect migration more complex.

A multi-platform domain-specific language that can represent abstracted concepts from individual platforms and can understand several pipeline dialects could address these issues. This thesis presents PolyPipeline, a novel domain-specific language (DSL), and an Xtext-based multi-platform editor based on a robust, generalized metamodel for modeling fundamental CI/CD concepts. This metamodel covers dialects from the GitHub Actions, GitLab, BitBucket, Bamboo, Circle CI, and Azure DevOps platforms. Furthermore, this thesis introduces an ATL-based model-to-model transformation designed to automate the migration of pipeline specifications between GitHub Actions and Azure DevOps platforms while preserving both functional integrity and dialect-specific features. Along with this, this thesis presents two heterogeneous datasets of automation pipelines from different platforms. To validate the metamodel and model-to-model transformation on GitHub Actions and Azure DevOps, this thesis conducted a two-phase data collection process. Phase 1 leveraged GitHub's REST API and Software Heritage's archival APIs to collect a dataset of 42,106 pipeline specification files. Phase 2 refined the GitHub query to address indexation issues and collected 103,158 pipeline files. According to our results, the proposed editor successfully parsed 40,160 pipelines after applying minor pre-processing in Phase 1. Based on

a random sample of the remaining 1,946 pipelines, these pipelines were not parsed successfully due to malformed pipeline specifications, or the files being intended for other purposes. Results obtained from Phase 2 confirm the robustness of the metamodel by successfully parsing 96,381 pipelines. Most of the files that were not parsed successfully in Phase 2 had similar issues observed in Phase 1. In addition, some failures were due to a limitation of the proposed editor, which currently does not support the inter-pipeline reference feature provided by Azure DevOps. Regarding the transformation, in Phase 1, all GitHub Actions pipelines were successfully transformed into Azure DevOps pipelines and vice versa through model-to-model transformation. Based on a random sample of the converted pipelines, the transformation accuracy was validated. However, in Phase 2, 4,522 Azure DevOps pipelines failed to transform to GitHub Actions due to the use of restricted keywords for job name declaration. Lastly, a round-trip transformation experiment (dialect A to dialect B and then back to A) confirmed the reliability of the transformation process, with no functionality loss for 87% of GitHub Actions pipelines (24,163) and 68% of Azure DevOps pipelines (2,372) in Phase 1, and for 80% of GitHub Actions pipelines (57,242) and 76% of Azure DevOps pipelines (7,878) in Phase 2. In the future, the multi-platform editor can serve as a valuable tool for cross-platform analysis.

Résumé

Une livraison rapide de logiciels et une qualité élevée sont cruciales sur le marché concurrentiel d'aujourd'hui, ce qui amène de nombreuses organisations à adopter des pratiques DevOps pour améliorer leurs pipelines d'intégration continue et de livraison continue (CI/CD). Néanmoins, l'introduction de diverses plateformes telles que GitHub Actions, GitLab CI, Bitbucket Pipelines, Bamboo, Circle CI et Azure DevOps, chacune avec son propre dialecte de pipeline, a rendu la migration inter-dialecte plus complexe.

Un langage multiplateforme spécifique à un domaine capable de représenter des concepts abstraits à partir de plateformes individuelles et capable de comprendre plusieurs dialectes de pipeline pourrait résoudre ces problèmes. Cette thèse présente PolyPipeline, un nouveau langage spécifique à un domaine (DSL) et un éditeur multiplateforme basé sur Xtext basé sur un métamodèle robuste et généralisé pour la modélisation des concepts fondamentaux de CI/CD. Ce métamodèle couvre les dialectes des plateformes GitHub Actions, GitLab, BitBucket, Bamboo, Circle CI et Azure DevOps. De plus, cette thèse introduit une transformation de modèle à modèle basée sur ATL conçue pour automatiser la migration des spécifications de pipeline entre les plateformes GitHub Actions et Azure DevOps tout en préservant à la fois l'intégrité fonctionnelle et les fonctionnalités spécifiques au dialecte. Parallèlement à cela, cette thèse présente deux ensembles de données hétérogènes de pipelines d'automatisation provenant de différentes plates-formes. Pour valider le métamodèle et la transformation de modèle à modèle sur GitHub Actions et Azure DevOps, cette thèse a mené un processus de collecte de données en deux phases. La phase 1 a exploité l'API REST de GitHub et les API d'archivage de Software Heritage pour recueillir un ensemble de données

de 42 106 fichiers de spécifications de pipeline. La phase 2 a peaufiné la requête GitHub pour résoudre les problèmes d'indexation et recueilli 103 158 fichiers de pipeline. Selon nos résultats, l'éditeur proposé a analysé avec succès 40 160 pipelines après avoir appliqué un prétraitement mineur dans la phase 1. Sur la base d'un échantillon aléatoire des 1 946 pipelines restants, ces pipelines n'ont pas été analysés avec succès en raison de spécifications de pipeline mal formées ou de fichiers destinés à d'autres fins. Les résultats obtenus à partir de la phase 2 confirment la robustesse du métamodèle en analysant avec succès 96 381 pipelines. La plupart des fichiers qui n'ont pas été analysés avec succès lors de la phase 2 présentaient des problèmes similaires à ceux observés lors de la phase 1. De plus, certains échecs étaient dus à une limitation de l'éditeur proposé, qui ne prend actuellement pas en charge la fonctionnalité de référence interpipeline fournie par Azure DevOps. Concernant la transformation, dans la phase 1, tous les pipelines GitHub Actions ont été transformés avec succès en pipelines Azure DevOps et vice versa via une transformation de modèle à modèle. Sur la base d'un échantillon aléatoire des pipelines convertis, la précision de la transformation a été validée. Cependant, au cours de la phase 2, 4 522 pipelines Azure DevOps n'ont pas réussi à se transformer en actions GitHub en raison de l'utilisation de mots-clés restreints pour la déclaration du nom du travail. Enfin, une expérience de transformation aller-retour (dialecte A vers dialecte B puis retour vers A) a confirmé la fiabilité du processus de transformation, sans perte de fonctionnalité pour 87% des pipelines GitHub Actions (24 163) et 68% des pipelines Azure DevOps (2 372) en phase 1, et pour 80% des pipelines GitHub Actions (57 242) et 76% des pipelines Azure DevOps (7 878) en phase 2. À l'avenir, l'éditeur multiplateforme pourra servir d'outil précieux pour l'analyse multiplateforme.

Acknowledgements

I would like to extend my deepest gratitude to my respected supervisor, Professor Gunter Mussbacher, whose generous support, valuable guidance, and insightful feedback have been pivotal throughout my master's studies. It is truly an honor to have him as my supervisor during my graduate journey. His expertise and encouragement have profoundly shaped my research and helped refine my ideas.

I would also like to sincerely thank Amdocs Development Centre India LLP, and in particular, my mentors Chetan Kadam and Sonam Mishra, for nurturing my early interest in the field of DevOps. Their guidance and encouragement during the initial stages of my thesis provided a strong foundation for this work.

Finally, I express my heartfelt gratitude to my beloved family — Sumukh Bedekar, Manali Bedekar, and Manoj Bedekar — for their unwavering love and support throughout this journey. I am equally grateful to my girlfriend, Aditi Deodhar, whose constant encouragement, understanding, and belief in me have been an invaluable source of strength and motivation.

Table of Contents

Abstract ii

Résumé iv

Acknowledgements v

List of Figures ix

List of Tables x

List of Listings xi

List of Abbreviations xii

1 Introduction **1**

1.1 Context and Motivation 1

1.2 Research Questions, Objectives, and Contributions 4

1.3 Thesis Organization 7

1.4 Summary 9

2 Background **10**

2.1 DevOps Practices 10

2.1.1 Advantages 11

2.1.2 Challenges 12

2.2 Data Mining Sources 14

2.2.1 GitHub 15

2.2.2 Software Heritage 17

2.3 Domain-Specific Languages and Xtext 19

2.4 Model-to-Model Transformation 20
2.4.1 Process of Model-to-Model Transformation 20
2.4.2 Benefits 21
2.5 Summary 22

3 Dataset Preparation **24**
3.1 Overview 25
3.2 Phase 1 25
3.2.1 Step A: Query Design and GitHub Repository Identification 26
3.2.2 Step B: Data Extraction from Software Heritage 29
3.2.3 Step C: Data Extraction from GitHub 30
3.3 Phase 2 31
3.3.1 Step A: Refined Query Strategy 31
3.3.2 Step B: Parallel Data Retrieval and Storage 33
3.4 Results 33
3.5 Summary 34

4 PolyPipeline DSL Specification **36**
4.1 CI/CD Concept Metamodel Design 37
4.2 Enhanced Metamodel Design 44
4.3 Dataset Pre-Processing and Analysis 47
4.4 RQ 1: Evaluation of DSL Generalizability 49
4.4.1 Validity of Phase 1 Dataset 50
4.4.2 Validity of Phase 2 Dataset 55
4.5 Summary 57

5 Inter-Dialect Transformation of Pipeline Specifications **58**
5.1 GitHub Actions to Azure DevOps Transformation 59
5.2 Azure DevOps to GitHub Actions Transformation 68

5.3 XMI-to-Text Custom Formatter 71
5.4 RQ 2: Evaluating Dialect Migration Feasibility 72
5.4.1 Evaluation of Phase 1 Dataset 73
5.4.2 Evaluation of Phase 2 Dataset 74
5.4.3 Round-Trip Transformation Results of Phase 1 75
5.4.4 Round-Trip Transformation Results of Phase 2 77
5.5 Summary 80

6 Related Work **81**
6.1 DevOps Practice Implementation 81
6.1.1 MDE Approach for DevOps 82
6.1.2 DevOps Impact on Software Quality 83
6.1.3 DevOps Adoption Challenges and Benefits 84
6.2 DSL Development Platforms and Comparison 86
6.3 Model-to-Model Transformation Techniques 88
6.3.1 ATL: A Hybrid and Compliant Transformation Language 89
6.3.2 Transformation in the Context of DevOps Pipelines 90
6.4 Summary 91

7 Conclusion **92**
7.1 Contributions and Findings 92
7.2 Opportunities for Future Research 95

A Xtext Grammar **97**

List of Figures

3.1 Phase 1 Data Collection Process 26
3.2 Count of GitHub Repositories containing YAML/YML Files 27
3.3 Phase 2 Data Collection Process 32

4.1 Metamodel for Multi-Platform DevOps Language 38
4.2 Enumerations in Metamodel 42
4.3 Enhanced Metamodel for Model-to-Model (M2M) Transformation 45
4.4 Preprocessing and Analysis for Phase 1 (Top) and Phase 2 (Bottom) 47

List of Tables

3.1 Summary of Dataset Collection Statistics for Software Heritage and GitHub Data Retrieval 34

5.1 Mapping of Pipeline and Configuration Keywords and Metamodel Elements 62

5.2 Mapping of Event Keywords and Metamodel Elements 63

5.3 Mapping of Job Keywords and Metamodel Elements 65

5.4 Mapping of Step Keywords and Metamodel Elements 68

5.5 Mapping of Azure DevOps Keywords and Metamodel Elements 71

6.1 Comparison of DSL Development Platforms 88

List of Listings

4.1 GitHub Actions Pipeline Syntax Example 39

4.2 Azure DevOps Pipeline Syntax Example 40

4.3 Error While Declaring Job and Stage at Same Level 52

4.4 Multiple Times Same Keyword Declaration 52

4.5 Unexpected Keyword Error (Line 5) and EOF Error (Line 43) 53

4.6 Source File for Other Program 54

5.1 GitHub Actions Configurations and Events 61

5.2 Transformed Azure DevOps Configurations and Events 62

5.3 GitHub Actions Jobs 66

5.4 Transformed Azure DevOps Stages and Jobs 67

A.1 Reference Xtext Grammar Code 97

List of Abbreviations

ATL Atlas Transformation Language

CI/CD Continuous Integration / Continuous Deployment

DSL Domain Specific Language

EMF Eclipse Modeling Framework

ETL Epsilon Transformation Language

M2M Model-to-Model

MDE Model-driven Engineering

MOF Meta-Object Facility

QVT Query/View/Transformation

SWH Software Heritage

XMI Extensible Markup Language (XML) Metadata Interchange

XML Extensible Markup Language

Chapter 1

Introduction

This chapter introduces the thesis starting by stating the context and motivation of a multi-platform Domain Specific Language (DSL) to address challenges in the field of DevOps in Section 1.1 and elaborates on the proposed research questions, objectives, and contributions for this thesis in Section 1.2. This chapter concludes with an overview of the organization of the thesis in Section 1.3.

1.1 Context and Motivation

Enterprises delivering software solutions often have distinct teams and units for development, operations, and quality assurance. These departments focus on specialized tasks. The development team designs and develops software solutions according to customer requirements and by commonly following agile practices. The development team is responsible for developing features according to sprint deadlines and the desired development plan. The latter makes sure that features implemented by the development team are of high quality before delivering them to the customer. The operations team, on the other hand, maintains the existing delivered solutions. While the development team is more interested in providing new features or modifications to customers rapidly, the operations team wants greater reliability and security and would rather not alter their products as regularly. It is difficult

for the operations team to tolerate the frequent release of new versions, due to the lack of automation and unavailability of defined pipeline processes [1] [2].

In this competitive world with rapidly changing technology, having a great software solution is not sufficient to hold a stake in the market. The speed of software development is equally crucial. With a traditional software development and delivery approach, the disagreements between the development team and operations team may impede the advancement of software. To bridge the gap between development and operations, DevOps combines the practices of development and operations teams to change incentives and share techniques throughout the whole development process [1]. Development and operations teams must work together more effectively to address crucial problems throughout the software development process, and DevOps is a collection of methods to do so [3].

One of the key pillars of DevOps is automation [4]. Automation involves the systematic use of tools and practices to reduce manual interventions, minimize errors, and expedite the delivery pipeline. Automation binds development, quality assurance, and deployment together. With automation, an enterprise can get closer to the goal of defectless, reliable, and fast product delivery. Deployment pipelines are automated to ensure that code moves seamlessly from development to production environments. Automation not only accelerates the delivery of software but also enhances consistency and repeatability, making it a cornerstone of DevOps success. This will result in gaining the trust of a customer.

GitHub Actions, GitLab, Bitbucket, Azure DevOps, Bamboo, Circle CI, and Jenkins are some of the most commonly used platforms for automating deployment pipelines. GitHub Actions[1], GitLab[2], Bitbucket[3], Azure DevOps[4], Bamboo[5], and Circle CI[6] use YAML format to specify pipelines, whereas Jenkins[7] uses Groovy format. GitHub Actions and GitLab

[1] https://docs.github.com/en/actions/using-workflows/workflow-syntax-for-github-actions
[2] https://docs.gitlab.com/ee/ci/yaml/
[3] https://support.atlassian.com/bitbucket-cloud/docs/bitbucket-pipelines-configuration-reference/
[4] https://learn.microsoft.com/en-us/azure/devops/pipelines/yaml-schema
[5] https://docs.atlassian.com/bamboo-specs-docs/7.2.4
[6] https://circleci.com/docs/introduction-to-yaml-configurations/
[7] https://www.jenkins.io/doc/pipeline/steps/workflow-cps/

remain indispensable in software development and DevOps due to their pivotal role in version control, collaborative coding, and efficient project management.

As a variety of DevOps platforms are available in the market, it has become difficult to choose the appropriate platform according to infrastructure requirements. The migration of the infrastructure setup of Continuous Integration / Continuous Deployment (CI/CD) and Continuous Testing (CT) pipelines from one platform to another is a very challenging task [1] [2]. It requires a detailed understanding of all technology stacks. Each platform comes with its own configuration details, terminology, and environment assumptions, making it difficult to preserve the functionality of existing pipelines. For example, what may be defined as a "job" in one platform could appear as a "stage" in another, or vice versa, with platform-specific settings that do not easily translate. Without a flexible and generalized metamodel that captures the essential CI/CD building blocks common to all these platforms, every migration turns into a time-consuming, error-prone process. This serves as a primary motivation for researching a multi-platform domain-specific language which can understand all of these dialects.

Another significant motivation for developing a multi-platform DSL is the challenge in standardizing the pipeline structure across the platforms. CI/CD pipelines declared in different dialects, while conceptually similar, differ significantly in their implementation, syntax, and semantics across platforms. This heterogeneity makes it difficult for teams to standardize their workflows, especially in environments where multiple DevOps platforms are concurrently used for single or multiple product delivery. The lack of standardization leads to inefficiencies, as each platform demands platform-specific expertise and dedicated maintenance efforts, which can effectively slow down the software delivery process and increase the operation workload. From the research perspective, the lack of standardization makes it challenging to systematically analyze and compare pipeline design processes. Uniform standardization enables organizations to create reusable, interchangeable pipeline configurations, reducing time spent on reworking pipelines for different platforms.

Moreover, the introduction of a multi-platform DSL could also promote greater inclusiveness and accessibility within the DevOps pipelines. This could lower the entry barrier for new developers and operation personnel enabling them to contribute effectively without extensive training in each platform. Ultimately, this would lead to enhanced collaboration, as team members with diverse skill sets could work together seamlessly.

Problem Statement. This thesis aims to address the challenges of using multiple DevOps platforms, each following its own dialect for defining CI/CD pipelines, including pipeline migration challenges. It aims to deliver a robust, generalized metamodel representation of DevOps CI/CD concepts, a corresponding pipeline editor, and inter-pipeline transformations to address these challenges.

1.2 Research Questions, Objectives, and Contributions

The problem statement in this thesis underlines a key obstacle in modern DevOps: multiple platforms—such as GitHub, GitLab, BitBucket, Bamboo, Circle CI, and Azure DevOps—each have distinct dialects for defining CI/CD pipelines. This scenario complicates cross-platform workflow standardization, raises maintenance overhead, and creates significant challenges in comparing or migrating pipelines. The core goal of this thesis is to introduce a robust, generalized metamodel of DevOps CI/CD concepts that can address these limitations and streamline the migration between different platform dialects.

Developing a generalized metamodel requires investigating which pipeline elements are shared across platforms and how to represent platform-specific features without losing essential functionality. Moreover, maintaining the logical sequence of tasks and reusing existing configurations across platforms remain crucial to preserve the reliability of pipelines. These overarching issues inspire the following research questions:

Research Questions (RQs)

The proposed DSL is to incorporate all DevOps CI/CD concepts and should understand six YAML based dialects including GitHub, GitLab, BitBucket, Bamboo, Circle CI, and Azure DevOps platforms. Therefore, it is necessary to investigate the generalizability of the DSL on a large diverse dataset.

RQ 1

To what extent does the proposed domain-specific language represent a generalized abstraction of CI/CD concepts across multiple platforms?

A Model-to-Model (M2M) transformation can address the pipeline migration difficulties by eliminating manual efforts for rewriting or redesigning the pipeline. However, this transformation should not change the functionality flow of the pipeline and should utilize dialect-specific features. Therefore, it is necessary to check the feasibility of the M2M transformation by incorporating dialect-specific features and maintaining pipeline flow.

RQ 2

To what extent does this DSL environment help with migration between pipeline dialect specifications while preserving functionality flow and incorporating dialect-specific features?

Objective

The primary objective of this thesis is to create a DSL that encapsulates core CI/CD concepts and supports six widely-used YAML-based pipeline dialects including GitHub, GitLab, BitBucket, Bamboo, Circle CI, and Azure DevOps. Specifically, this thesis ensures the DSL provides a robust and generalized metamodel representation to standardize and streamline pipeline specifications across different platforms. Also, this thesis aims to develop an Xtext-based editor with features such as auto-dialect detection, keyword highlighting, and syntax validation. In addition, this thesis aims to present a heterogeneous dataset of automation

pipelines from different open-source platforms, along with a robust process capable of reproducing a heterogeneous dataset. Lastly, the thesis seeks to introduce a M2M transformation feature that simplifies pipeline migration between platforms, specifically targeting GitHub Actions and Azure DevOps.

Contribution

To address the drawbacks of using multiple DevOps platforms and to standardize the pipeline structure across the platforms for analysis purposes, this thesis proposes PolyPipeline, a DSL and Xtext-based multi-platform editor for DevOps pipelines, that covers six different dialects and uses YAML format for pipeline specification declaration. The proposed solution is enabled with features like auto dialect detection, keyword highlighting, and syntax validation.

Furthermore, this thesis presents a heterogeneous dataset of automation pipelines from different platforms. To address any problem, the availability of diverse datasets is very crucial, as they cover a greater number of possible test cases. To test the generalizability of this editor, a diverse dataset was needed, which led to the design of a process capable of scraping pipeline specifications from open-source platforms. This thesis introduces a detailed process of collecting different automation pipelines from open-source projects available on GitHub and Software Heritage. It also discusses the limitations and advantages of using individual platforms. The first phase of the dataset scraping process collected more than 42,000 potential pipelines while the second phase of the dataset scraping process collected around 100,000 potential pipelines and the vast majority of potential pipelines were parsed successfully by the proposed editor. An analysis of the potential pipelines that were not parsed successfully revealed that the pipeline files themselves were invalid in majority of the cases, in addition, some failures were due to a limitation of the proposed editor, which currently does not support the inter-pipeline reference feature provided by Azure DevOps. The DSL, the Xtext-based editor, and the resulting dataset pave the way for further cross-platform analyses of automation pipelines.

Lastly, this thesis introduces a novel M2M transformation feature designed specifically for GitHub Action and Azure DevOps pipeline specifications. This feature offers an efficient mechanism to automatically incorporate dialect-specific concepts while ensuring that the original pipeline functionality and flow remain unchanged. The key advantage of this transformation feature is its ability to simplify the migration process for existing pipeline setups by eliminating the need for manually rewriting or redesigning the pipeline specifications.

To the best of my knowledge, work in the field of a DevOps and domain-modeling does not address the difficulty of research across different DevOps pipeline platforms and a practical approach to collecting a heterogeneous set of DevOps pipelines from open-source projects, which is the focus of my proposed DSL and dataset. They are valuable for future research aimed at abstracting various DevOps dialects. The specific contributions of this thesis are the following:

- This thesis provides an environment for DevOps pipelines including the PolyPipeline DSL and Xtext-based multi-platform editor.
- This thesis proposes a robust process capable of reproducing a heterogeneous dataset of DevOps pipeline specifications available on open-source platforms.
- This thesis introduces a novel M2M transformation for pipeline migration between GitHub Actions and Azure DevOps.

1.3 Thesis Organization

This section provides an overview of the organization of the thesis.

- **Chapter 2:** This chapter provides background information and defines fundamental concepts used throughout the thesis. This chapter gives a brief overview of DevOps methodology and different challenges that are being faced while adopting the DevOps methodology. Also, this chapter discusses domain-specific languages and Xtext, along

with M2M transformations. The author of this thesis makes a full contribution to this chapter.

- **Chapter 3** This chapter discusses the method to collect a relevant open-source heterogeneous dataset from platforms such as GitHub and Software Heritage. Along with the data collection process, this chapter also highlights advantages and disadvantages of using different open-source platforms. This process is designed solely by the author of this thesis and, as such, the author makes a full contribution to this chapter.

- **Chapter 4:** This chapter presents a detailed discussion of the novel, enhanced metamodel of the proposed DSL that generalizes the representation of CI/CD pipelines across six major DevOps platforms: GitHub Actions, GitLab, Bamboo, BitBucket, Circle CI, and Azure DevOps. This chapter concludes with a discussion on the key findings related to RQ 1, including validation results for the metamodel based on experiments with the two different datasets introduced in Chapter 3. The author of this thesis makes a full contribution to this chapter.

- **Chapter 5:** This chapter focuses on addressing pipeline migration using M2M transformations. This chapter concludes with a discussion on the key findings related to RQ 2, including validation results for the transformation based on experiments with the two different datasets introduced in Chapter 3. The author of this thesis makes a full contribution to this chapter.

- **Chapter 6:** This chapter presents research related to this thesis, including an analysis of the 2022 metamodel for DevOps pipelines [5], challenges and benefits in adopting DevOps practices, DSL, and M2M transformation. The author of this thesis makes a full contribution to this chapter by focusing on the lack of existing research addressing unified representation of CI/CD pipelines and practical approaches for model-to-model transformation to address the inter-dialect migration challenge.

- **Chapter 7:** This chapter presents a summary of the key accomplishments of the thesis and outlines future work. The author of this thesis makes a full contribution to this chapter.

Earlier results of the research effort described in this thesis in Chapter 2, Chapter 3, Chapter 4, and Chapter 6 were reported in Bedekar, M.M. and Mussbacher, G. (September 2024) A Multi-Platform Specification Language and Dataset for the Analysis of DevOps Pipelines. In Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems (MODELS Companion 2024), ACM (pp. 264-274) [6]. The author of this thesis contributed fully to this publication, while the publication's second author provided feedback and guidance.

The Xtext grammar, the dataset collection pipeline, and the transformation scripts are stored in a public GitHub repository[8].

1.4 Summary

In conclusion, this chapter provides a comprehensive overview of the thesis. It begins by explaining the context and motivation of the proposed PolyPipeline DSL and Xtext-based multi-platform editor for DevOps pipelines in Section 1.1, followed by the proposed research questions, objectives, and contributions in Section 1.2. Finally, the organizational structure of this thesis is demonstrated in Section 1.3. The next chapter discusses research background related to DevOps, data mining sources, DSL, and M2M transformations.

[8] `https://github.com/manasbede/DevOps-Dialect-Editor`

Chapter 2

Background

This chapter describes the foundations for the relevant research areas of this thesis. As a starting point, Section 2.1 discusses the advantages, challenges, and background of DevOps practices. Next, Section 2.2 briefly discusses different data mining sources and their advantages and disadvantages. Lastly, this chapter discusses domain-specific languages and different open-source DSL frameworks in Section 2.3 and model-to-model transformation in Section 2.4.

2.1 DevOps Practices

With the primary motivation of delivering reliable products with faster development and delivery, the DevOps methodology has become increasingly popular in the last few years. DevOps defines a collaborative practice to be followed in development and operations teams. The DevOps methodology focuses on streamlining the product delivery process with the help of automation, collaboration, CI/CD principles, monitoring, and feedback. Before DevOps in mid-2000, the agile methodology emerged as a response to the limitations of a traditional, more rigid software development process. The DevOps methodology has become more popular in the last decade. This rise has its origin in well-known companies such as Google, Apple, and Amazon being able to successfully implement and reap profits from it. Both

DevOps and agile methodologies focus on improving the software development and delivery process. They also promote collaboration, efficiency, and continuous improvement.

Automation is one of the major reasons DevOps practices are being adopted by enterprises. Automation has led to the development and delivery of a product by avoiding human errors while promising faster processing time [7] [4]. By automating quality assurance, deployment, and operations, enterprises have successfully built trust with their customers [8] [9]. According to a recent report by "Research and Market" the global DevOps market will hit $37.2 billion by 2028 [10].

2.1.1 Advantages

By combining automation, continuous integration, continuous deployment, and collaboration cultural shifts, DevOps presents a holistic framework designed to speed up software delivery while maintaining high-quality and reliability standards. Numerous recorded empirical studies and case analyzes confirm the effectiveness of DevOps in improving organizational agility to respond to dynamic changes in the market, optimizing resource utilization, and fostering innovation [8] [1] [11] [2] [12].

Improved Collaboration and Integration

DevOps frameworks reconcile, in a very efficient manner, the traditional divide that exists between software development and IT operations, hence fostering a corporate culture distinguished by collective accountability, clear communication, and unified collaboration. This holistic strategy not only improves team interactions but also optimizes workflow mechanisms, hence fast-tracking development timelines. Meanwhile, alignment of goals and reduction in interdepartmental tensions make it easier to address challenges quickly, which is important for the maintenance of high-quality standards and operability [1].

Robust Response to Market Changes

DevOps practices provide agility to an organization in responding to changes in the market or new demands from customers. This is achieved through iterative and incremental DevOps processes where a product is built and released in small, manageable increments. This kind of structure allows teams to change direction easily without much waste, making it possible to innovate and meet market demands with agility [13].

Continuous Improvement

The feedback loops integral to DevOps encourage continuous assessment and improvement of products, services, and processes. This culture of ongoing development and refinement leads to superior product quality and operational excellence. By consistently monitoring and optimizing their approaches, teams can continually adapt and evolve their practices to drive better business outcomes, ensuring long-term sustainability and success in a competitive landscape [14].

2.1.2 Challenges

While DevOps has been feted for its potential to increase efficiency, speed, and collaboration within software development and IT operations, it does not come without its disadvantages. This section identifies some of the less discussed but important disadvantages that organizations face when adopting DevOps methodologies. From challenges in integrating and managing diverse toolchains to cultural and operational challenges that can impede the seamless implementation of DevOps practices, let us now look at how these drawbacks can impact the overall effectiveness of the DevOps model. While the promised benefits of rapid deployment and improved cross-functional teamwork are very attractive, these disadvantages bring to light the need for careful planning, strong management, and continuous adaptation in order to harvest the full potential of DevOps without falling into its pitfalls.

Diverse Platform Ecosystem and Migration Challenge

With the increasing demand for the DevOps methodology, different open-source platforms started competing against each other by offering functionalities for automation, CI/CD, and quality assurance. Jenkins, GitLab, GitHub Actions, Azure DevOps, Circle CI, Travis CI, Bamboo, BitBucket, Ansible Tower, and AWS CodePipeline are the most popular DevOps automation platforms [15]. With the availability of multiple platforms, it has become challenging to choose an appropriate platform. These platforms use different dialects to define the pipeline and infrastructure. An organization may have to use a mix of technology stacks, and ensuring compatibility across platforms can be very challenging.

Compatibility and Standardization Issues

For different platforms, separate scripts and infrastructure setups are required. This may lead to inconsistency, leading to difficulty in maintaining standardization [16]. Such disparities may slow down the deployment processes that DevOps aims to expedite, increasing operational complexity.

Community Support and Security Vulnerabilities

Another significant challenge faced by some DevOps platforms is having a smaller maintenance community or third-party integrations. Platforms with smaller maintenance communities may lack the resources to promptly address security flaws, posing significant security risks in rapidly evolving environments [17]. The strength and responsiveness of a platform's community are crucial for managing security effectively. It may also be difficult to migrate infrastructure and pipelines from one platform to another.

Toolchain Complexity and Skill Gaps

These platform-centric challenges underscore a broader issue within DevOps practices: the complexity of managing and integrating diverse tools and systems. This complexity can exacerbate existing challenges such as toolchain complexity and skill gaps within teams,

complicating the balance between rapid deployment benefits and the risks of a fragmented tool ecosystem [18].

2.2 Data Mining Sources

To conduct empirical studies in the software engineering field, obtaining relevant datasets is a challenging task. Lack of appropriate tools, change in the code base, and lack of documentation of extraction processes make the dataset collection process difficult and fail to reproduce a similar dataset [19]. In the digital age, software development practices and their evolution are increasingly studied through the lens of vast datasets provided by platforms like GitHub[1] and Software Heritage (SWH)[2]. These platforms serve as repositories that not only host an extensive range of software projects but also meticulously record each alteration, contribution, and evolution in software development. Data mining from these repositories involves a series of systematic steps aimed at extracting, processing, and analyzing the digital traces left by millions of developers around the world.

Data mining from these platforms generally includes several key phases: data collection, where API calls or direct downloads are used to gather the required datasets; data preprocessing, which includes cleaning, normalizing, and structuring the data for analysis; data analysis, where statistical and computational techniques are employed to derive insights; and data interpretation, which involves converting analytical results into comprehensible outcomes, often visualized to facilitate better understanding. Finally, storage and maintenance ensure that the data remains accurate, updated, and accessible for ongoing or future research. An integration of data and information gathered from sources like GitHub and SWH enables researchers to carry out extensive studies tracing the evolution of software development practices over time, not just the current state [20] [21]. This integrated approach enriches understanding of how people develop software in different contexts and helps to determine both general trends and unique features of a particular platform.

[1] `https://docs.github.com/en/rest?apiVersion=2022-11-28`
[2] `https://www.softwareheritage.org/`

2.2.1 GitHub

Using data from GitHub opens up a range of opportunities for analysis on many dimensions of the software development methodologies, community interactions, and technological advancement. Using GitHub's REST API makes it easier to have organized access to a wide variety of categories of data including such repositories, commits, code modifications, pull requests, and issues. All these are rich sources for the fulfillment of different research objectives.

Repositories and Commits

Researchers usually analyze repositories in conjunction with their commit history to understand the evolution of coding practices and the reliability of software. Analyzing commits traces the temporal development of software projects, identifies major changes or improvements, and finds trends in development practices. For instance, one of the previous research efforts discusses the challenges and opportunities in mining GitHub, noting that commit data can provide useful insights into project development and developer behavior [21].

Code Changes and Pull Requests

The data of code changes and pull requests are the most useful to study collaboration and code review practices. Analyzing pull requests may tell how community members interact, how decisions about code changes are made, and what social dynamics a project holds [22].

Issue Tracking

The monitoring of issues forms a critical dimension that can be derived from GitHub. Issues provide excellent insights into the different challenges that projects face, the ways through which they are solved, and the communication patterns of developers. Researchers use this information to study bug tracking, requests for new features, and the overall sustainability of software projects [23].

This variety of data types enables further research in software development processes, teamwork, and management practices. For instance, analyzing pull requests and issues together helps reveal how coding decisions are debated and concluded in an organization's context concerning continuously evolving project requirements and never-ending bugs.

Limitation

Data mining from GitHub, while a rich source for research, presents several limitations that can impact the quality and scope of the insights gained. These limitations are recognized in various studies and are as follow.

The first limitation in GitHub data mining is selection bias [21]. The data on GitHub is not representative of all software development projects, as they predominantly include open source projects, which may not reflect practices in proprietary or enterprise environments. This selection bias can skew results and limit the generalizability of the findings. Also, many projects on GitHub are inactive or are mere forks of other repositories with little to no new development. This can clutter datasets with redundant or irrelevant data, complicating analysis and interpretation [21].

Another crucial limitation is the API rate limitation. GitHub's API imposes rate limits that restrict the amount of data that can be accessed within a certain time frame, potentially limiting extensive data collection efforts. GitHub imposes rate limits on the number of API requests that can be made in a given time period to ensure fair usage and protect the platform from abuse [24]. For unauthenticated requests, the rate limit allows for up to 60 requests per hour. For authenticated requests, this limit increases to 5,000 requests per hour. These limits can pose challenges for large-scale data mining projects as they restrict the volume of data that can be gathered in a short period [25].

Lastly, the consistency, completeness of data, and dependency on API changes makes it really hard to collect heterogeneous datasets in a reproducible way. The GitHub REST API is not consistent with indexation. Also, GitHub may change its API structure, endpoint functionalities, or policies around access and rate limits, potentially breaking existing data

collection scripts or tools [26]. In addition, there is no guarantee that the data on GitHub will not change. Over time repositories might get deleted which will lead to inconsistency.

2.2.2 Software Heritage

Software Heritage undertakes an ambitious project of gathering, preserving, and making accessible a huge collection of source codes of software in the world. Besides supporting technological research, this helps achieve the goal of historical and cultural preservation.

Repository Identification and Harvesting

The process begins with the identification and harvesting of source code from various publicly accessible repositories. Software Heritage uses automated crawlers that navigate through platforms like GitHub, GitLab, and Bitbucket, capturing every piece of source code they can find. This approach ensures a diverse collection that spans multiple programming languages, frameworks, and development histories [20].

Archival Structure and Data Deduplication

Once the source code is collected, it is stored in a graph-based archival system. This system links different versions and components of the software projects, allowing for efficient retrieval and analysis. The archival structure is designed to handle the complexity and scale of the data collected, employing deduplication techniques to minimize storage space and avoid redundancy [27]. Deduplication is a crucial feature of the archiving process, ensuring that identical data elements are stored only once. This not only conserves storage but also simplifies the data structure, making it easier to manage and query the archive [27].

Multiple Options to Explore Dataset

Software Heritage offers a variety of methods for accessing its vast dataset of archived source code, catering to different needs. Users can explore repositories using a user-friendly web interface or retrieve data programmatically via an API, which is ideal for automated systems

and applications needing integration with Software Heritage data. For comprehensive offline analysis, large datasets are available for download, while advanced queries can be executed using a graph query interface designed to navigate the complex relationships within the archive.

Challenges and Limitation

While the objective of Software Heritage to archive all publicly accessible software is admirable, it encounters several challenges that affect its capacity and efficiency. Despite efforts to achieve comprehensive coverage, the archive often misses projects that are not on prominent platforms or that are kept private. Moreover, not all developers are aware of or opt to include their repositories, leading to incomplete data. The archival process also faces delays; updates to source code can take time before they are reflected in the archive, which may be problematic for research needing the most current data.

Additionally, the archive's reliance on a complex graph-based data structure, although useful for linking related data, can be difficult for users who are not familiar with such systems to navigate. This issue is made more challenging by the archive's dependence on external platforms for source code. Any changes to the policies or technical limitations of these platforms can directly affect the archive's capability to gather and maintain data. Furthermore, the archive provides a web search API that helps locate the origin of source codes, but it has limitations, such as only allowing 10,000 results per query without the option for customization, which restricts comprehensive data retrieval. These challenges highlight the ongoing need for improvements in how data is collected, the design of user interfaces, and collaboration with external platforms to enhance the archive's accessibility and relevance.

2.3 Domain-Specific Languages and Xtext

A DSL is a way to represent the unique features and requirements of a model created within a domain-specific modeling framework [28]. Domain-specific languages enable the software to target a particular type of problem to address. DSLs particularly help to identify common patterns within a domain and form an abstract and concrete syntax to represent the same. One of the most important advantages of domain-specific languages is the close mapping of the abstract and concrete syntax of the language to the problems within a desired scope and the ability to automatically generate downstream models or even executable code from the model specified with the DSL. DSLs can help boost and optimize code generated for the specific platform, resulting in improvement in productivity. To enable code generation, a DSL is required to be narrow in scope, and it is constrained by a language specification (i.e., the metamodel). The formal specification afforded by a metamodel enables model validation and the generation of tailored implementations. DSLs provide flexibility to be expressed in textual, graphical, or mixed notation according to the context of the domain. Dedicated analysis tools may be built on top of a DSL specification.

Xtext[3] is a widely used framework for developing textual domain-specific languages. Originally published in 2006 and developed under the Eclipse modeling project since 2008, Xtext provides comprehensive support for creating DSLs with rich features such as syntax highlighting, validation, and code generation. Xtext supports multi-platform integration and customization, making it a versatile tool for creating DSLs that can work across different ecosystems [29].

One of Xtext's key advantages is its seamless integration with other Eclipse frameworks, based on Eclipse Modeling Framework (EMF)[4], Ecore[5], and Meta-Object Facility (MOF)[6]. EMF, Ecore, and the MOF form a foundational set of technologies for model-driven engineering. EMF is a core framework for developing model-driven tools and applica-

[3] `https://eclipse.dev/Xtext/documentation/index.html`
[4] `https://projects.eclipse.org/projects/modeling.emf.emf`
[5] `https://projects.eclipse.org/projects/modeling.ecoretools`
[6] `https://www.omg.org/mof/`

tions. It enables the definition of models and automatically generates Java code, providing runtime support for editing and persisting model instances. Ecore is the principal metamodeling language in EMF. It is a Java-based implementation of MOF and defines elements such as classes, attributes, references, and data types. Ecore serves as the basis for creating and manipulating models within EMF. MOF is an OMG (Object Management Group) standard for defining metamodels.

As Xtext generates Ecore-based metamodels, Xtext can be integrated with multiple transformation languages based on EMF, Ecore, and MOF. This includes Atlas Transformation Language (ATL), Query/View/Transformation (QVT), Epsilon Transformation Language (ETL), and Acceleo. This integration enables developers to leverage existing modeling tools and libraries, reducing development time and effort. Xtext generates not only a parser for the DSL but also a complete integrated development environment (IDE) with features such as auto-completion, refactoring support, and debugging capabilities. These features make Xtext an ideal choice for creating sophisticated DSLs that cater to modern software engineering needs [30].

2.4 Model-to-Model Transformation

M2M transformation is a critical process within Model-driven Engineering (MDE) that facilitates the transition from abstract models to specific, implementable models. This method enhances the clarity and precision of software designs and ensures that the final implementations align closely with the initial conceptualizations. The following sections detail the steps involved in M2M transformation and highlight its significant benefits in software development.

2.4.1 Process of Model-to-Model Transformation

The process of model-to-model transformation begins by selecting the source and target models, each conforming to specific modeling languages appropriate for the original and de-

sired outcomes [31]. The transformation is specified at the metamodel-level which makes it possible to apply the transformation rules to any model that conforms to the source metamodel to produce a model that conforms to the target metamodel. Following this, transformation rules are established, which precisely dictate how elements from the source model are to be converted or adapted to fit the target model [32]. To implement these rules effectively, transformation engines such as ATL [33](Atlas Transformation Language)[7], QVT (Query/View/Transformation)[8], ETL (Epsilon Transformation Language)[9], and Acceleo[10] are employed. These tools ensure that the transformation process is accurately executed, producing a target model that meets the predefined criteria. Finally, the integrity and accuracy of the transformed model are verified through rigorous validation checks and iterative refinements, ensuring that the final product accurately reflects the intended design and functionality [34].

2.4.2 Benefits

Drawing from a detailed previous study, the benefits of model-to-model transformation in software development are extensive and encompass various critical aspects of software engineering [34]. The process significantly enhances the clarity and precision of software designs by ensuring that models align closely with strategic intents and stakeholder requirements. This alignment not only facilitates better communication among teams but also ensures that the final implementations accurately reflect the initial conceptual goals [34]. By adhering to well-defined transformation intents, model-to-model transformation aids in maintaining consistency across different development stages. This systematic approach ensures that every transformation accurately captures and reflects intended design changes and goals, reducing errors and misalignment as the project progresses [35].

[7] `https://wiki.eclipse.org/ATL/User_Guide_-_The_ATL_Tools`
[8] `https://projects.eclipse.org/projects/modeling.mmt.qvt-oml`
[9] `https://eclipse.dev/epsilon/doc/etl/`
[10] `https://eclipse.dev/acceleo/`

Moreover, model-to-model transformation supports scalability by enabling the efficient management of large and complex models, a critical requirement in modern software systems. It also provides flexibility, allowing multiple design options to be explored by transforming a source model into various target models under different constraints, thus fostering innovation and adaptability in software development [36]. Another significant advantage is enhanced traceability, where changes in the final software product can be traced back to specific transformations and the original model intents. This capability is crucial for debugging, maintenance, and ensuring compliance with design specifications, particularly in projects with strict regulatory or quality requirements [37].

2.5 Summary

This chapter establishes the foundational concepts and techniques that support the key areas of this thesis. The discussion begins with DevOps practices (Section 2.1), highlighting their advantages, challenges, and increasing adoption in modern software development, with a particular emphasis on automation and continuous integration principles. Next, the chapter explores the challenges and opportunities in data mining from repositories like GitHub and Software Heritage (Section 2.2), outlining their roles in capturing software development history, their respective data collection processes, and the associated limitations. This is followed by a detailed discussion on domain-specific languages (Section 2.3), illustrating how DSLs enhance software productivity through tailored representations of domain-specific problems. Finally, the chapter introduces model-to-model transformations (Section 2.4), detailing the transformation process, from selecting metamodels and establishing rules to applying transformation engines, and the benefits they bring in terms of clarity, precision, scalability, and traceability in software development.

The next chapter discusses in detail the two-phased approach for gathering YAML files for DevOps pipeline specifications from open-source repositories available on GitHub and

Software Heritage, yielding a large dataset of over 40,000 files in Phase 1 and 100,000 files in Phase 2.

Chapter 3

Dataset Preparation

This chapter discusses the approach for gathering YAML files from open-source repositories hosted on platforms such as GitHub and SWH. It details how GitHub's REST APIs can be effectively utilized to identify projects containing YAML (.yml or .yaml) files while addressing common challenges, such as rate limitations and search query restrictions to minimize the risk of missing repositories during data retrieval. The chapter is organized into three main sections. Section 3.2 explains how data is collected from both SWH and GitHub, including the design of custom queries to identify relevant repositories and the integration of origin URLs from SWH for enhanced reproducibility. Section 3.3 then describes the refinement of the query strategy—targeting repositories with higher star counts and recent creation dates—to overcome indexation issues encountered in Phase 1, along with the implementation of parallel data retrieval and storage mechanisms. Finally, Section 3.4 summarizes the outcomes of the dataset collection process, presenting detailed statistics that underscore the efficacy of the methodologies employed.

Dataset preparation scripts are uploaded in a public GitHub repository and can be accessed[1].

[1] `https://github.com/manasbede/DevOps-Dialect-Editor/tree/main/Dataset_Collection_Process`

3.1 Overview

To conduct empirical studies in the software engineering field, obtaining relevant datasets is a challenging task. Lack of appropriate tools, change in the code base and lack of documentation of extraction processes make the dataset collection process difficult and fail to reproduce a similar dataset [19]. In order to create a generalized, multi-platform editor, we needed a diverse dataset. To validate the generalization of the metamodel, we collected CI/CD pipelines (YAML format) from various open-source projects, available on GitHub and SWH. In this research, we collected CI/CD pipeline dataset in two different phases. Figure 3.1 and Figure 3.3 are a representation of the dataset collection process for the first and second phase, respectively. In the first phase, we used SWH and GitHub as open-source repository platforms, while in the second phase we collected data only from the GitHub. Through the Software Heritage initiative, all publicly available software is collected in source code form together with its development history, replicated in large quantities to ensure its preservation, and made available to everyone so that the embodied knowledge is not lost to humanity.

3.2 Phase 1

In SWH [38], source-code history is stored as snapshots. Each snapshot has a unique persistent identifier called SWHID for each component of a project, resulting in well-structured source code replication as shown in the right section of Figure 3.1. We therefore endeavored to use Software Heritage exclusively to benefit from the greater reproducibility of the dataset. By traversing a graph data structure using SWHID, one can access any component of a project. Software Heritage provides multiple ways to search for source projects, such as through the graph service, fuse framework, and web search. Fuse frameworks and graph services require the storage of Software Heritage locally, which was not feasible in our case given the size of Software Heritage graphs. Graph Query can be used on AWS instances to access the archive, but its rate limit is a drawback. Software Heritage offers a web search API

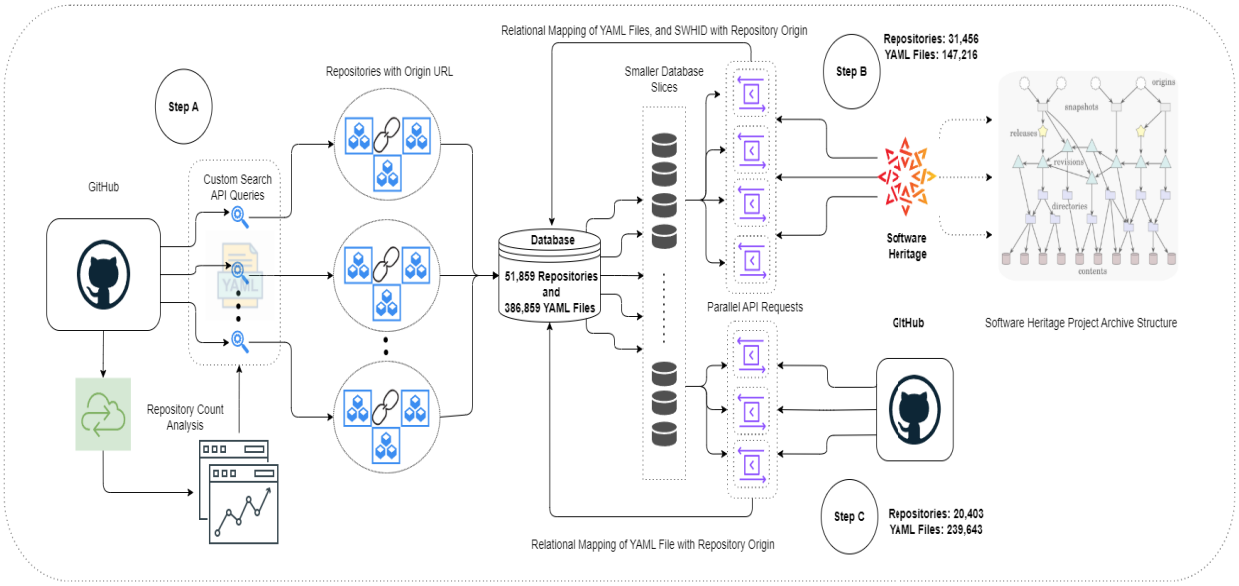

Figure 3.1: Phase 1 Data Collection Process

option that can be used to find the origin of source codes, but there are some drawbacks, such as the limit of 10,000 results per query, and the lack of customization of the query. Query search does not have a checkpoint facility to keep track of retrieved search results. Hence, it was not possible to retrieve more than 10,000 results even with several queries.

3.2.1 Step A: Query Design and GitHub Repository Identification

With the limitations of the Software Heritage APIs in mind, we shifted our focus to GitHub-origin open-source projects. GitHub offers a comprehensive set of REST APIs that enable access to a wide range of public repositories. However, these APIs come with their own restrictions—for instance, the GitHub search query API limits results to 100 resources per page and caps the total at 1000 resources per query. This necessitated a more targeted approach to capture relevant data.

DevOps pipelines for platforms like GitHub Actions, GitLab, Azure DevOps, Circle CI, Bamboo, and Bitbucket are typically defined using the YAML file format. These files are not centralized in one location within a project; instead, they can be distributed throughout the repository. To address this, we broadened our search to include all projects containing

YAML files, rather than limiting ourselves to repositories with explicitly labeled pipeline files.

To effectively identify repositories that might contain DevOps pipelines, we designed a custom search query focused on locating files with the ".yml" or ".yaml" extensions. This custom query allowed us to filter out repositories that do not contain any YAML files, thereby increasing the likelihood that the remaining repositories included valid CI/CD pipeline definitions. This step was crucial in laying the groundwork for more refined analyses later in the process.

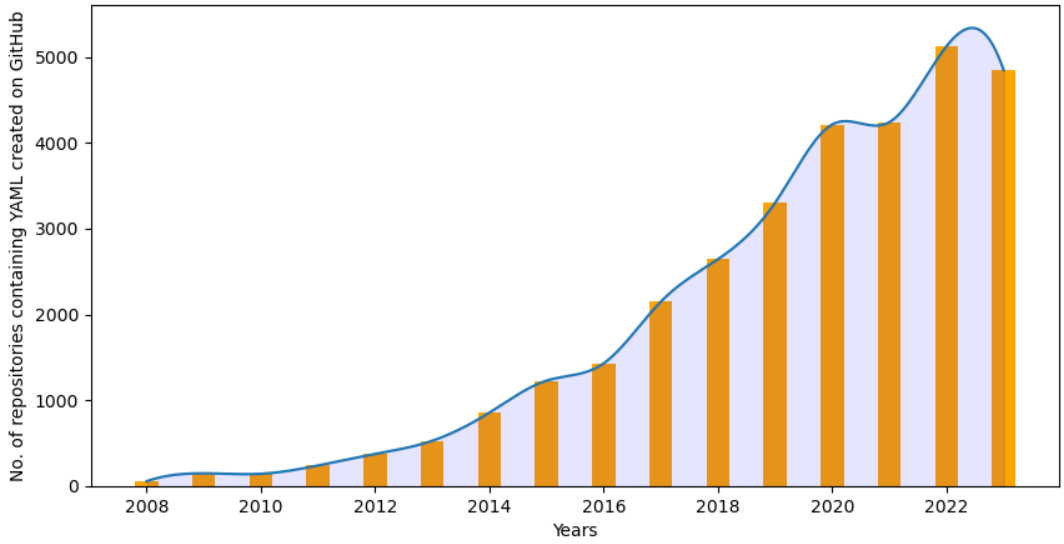

Figure 3.2: Count of GitHub Repositories containing YAML/YML Files

To overcome the inherent limitations of the GitHub search API, we employed a two-step approach. First, we executed a query to determine the total count of repositories matching our criteria. This preliminary step provided insights into the overall distribution of repositories over time—from GitHub's launch in 2008 to October 2023—as illustrated in Figure 3.2. Based on this temporal distribution, we crafted additional queries segmented by different time spans. This strategy minimized overlap between queries and ensured comprehensive coverage of the repository landscape.

The final output of this process was a detailed list of repositories, along with their origin URLs, amounting to a total of 51,859 open-source repositories containing YAML files. The left section of Figure 3.1 visualizes the search query process and highlights how our methodical approach effectively navigated the API limitations. This robust dataset forms a critical foundation for further stages of our analysis, ultimately aiding in the identification and study of DevOps pipelines across multiple platforms.

Query Parameters

GitHub REST APIs[2] use multiple tags for query design. These tags can be combined together for custom query design for specific endpoints. Each endpoint may have different supported tags. For query design, this thesis used the repository endpoint.

GitHub Trends Query: The GitHub Trends Query was formulated to search for repositories containing files with the YAML extension by utilizing the filename parameter tag. This query was specifically designed to analyze annual trends in repository creation. Accordingly, the 'created' tag was employed as a dynamic parameter, with defined start and end dates, to capture the distribution of repository creation in a year. After executing each query, the response was stored for further analysis. In the collected response, we looked for the repository count from each query and plotted a graph for better visualization.

GitHub URL Scraping Query: Based on the annual repository creation distribution obtained from the GitHub Trends Query, we designed the GitHub URL Scraping Query to retrieve repository URLs efficiently. The observed distribution was employed to dynamically define date intervals, thereby mitigating the limitation of receiving only 1,000 results per individual query. In addition, the query incorporated a pagination attribute (page number) to systematically iterate through the results. This combined strategy of dynamically determined date slots and paginated responses ensured that we maximized the retrieval of repository URLs while minimizing the overall number of queries executed.

[2] `https://docs.github.com/en/rest?apiVersion=2022-11-28`

Through these adjustments, we established a more robust and reliable approach for data collection, ensuring that the final dataset would accurately reflect contemporary trends in DevOps pipeline development while minimizing the potential for query-related inconsistencies.

3.2.2 Step B: Data Extraction from Software Heritage

The origin URLs played a crucial role in facilitating the retrieval of projects from Software Heritage. By leveraging these URLs, we could directly query the Software Heritage archive to locate and access corresponding projects, thereby streamlining the dataset collection process. This direct mapping ensured that each project could be accurately identified and linked to its original source, enhancing the reproducibility of our data.

A total of 31,456 projects were identified as available on Software Heritage. This substantial figure underscores the breadth and diversity of the archive, making it an invaluable resource for constructing a comprehensive dataset. The availability of such a large number of projects provides a solid foundation for subsequent analyses and comparative studies across various software ecosystems.

Software Heritage offers a web-crawling API that enables users to access project content based on the provided origin URLs. The API facilitates the retrieval of detailed project information, including source code and development history, but it does so under a rate limit of 1,200 requests per hour. This limitation necessitated the implementation of efficient query management strategies to ensure that our data collection remained systematic and within the defined usage constraints.

The top right section of Figure 3.1 illustrates this process in detail, showing how the origin URLs are used to search for and retrieve projects from the Software Heritage archive. This visual representation highlights the integration of our approach, which combines direct URL-based queries with effective API management to maximize data retrieval. Overall, this method not only enhances the reproducibility of the dataset but also demonstrates a robust approach to handling large-scale data collection in the face of practical constraints.

3.2.3 Step C: Data Extraction from GitHub

As the recently created projects were not yet available on Software Heritage, we employed the GitHub REST API to collect YAML files from those projects. This alternative approach allowed us to capture the most up-to-date repository information that had not been archived in Software Heritage. The bottom right section of Figure 3.1 illustrates this process in detail, outlining the steps taken to retrieve the YAML files directly from GitHub. This visual representation reinforces the rationale behind the decision to supplement our dataset with data from GitHub.

Including projects that are not available on Software Heritage introduces a significant methodological consideration. The absence of certain recently created projects in Software Heritage necessitated this additional step to achieve comprehensive coverage. Our strategy was to prioritize the acquisition of a larger dataset, even if it meant incorporating projects outside the Software Heritage archive. By doing so, we ensured that our analysis could benefit from a broader set of data points, which is critical for understanding the full spectrum of DevOps pipeline usage across various platforms. Nevertheless, we maintained rigorous reporting standards by documenting the individual results obtained from both Software Heritage and GitHub. Aggregated results are also presented, enabling clear differentiation between data sources while preserving transparency in our methodology.

The Software Heritage-based results retain the inherent benefit of greater reproducibility, given that they are derived from a well-curated archival repository with consistent historical data. This aspect of our dataset is particularly valuable for studies requiring long-term data consistency and traceability. Conversely, the additional results obtained via the GitHub REST API contribute to a significantly larger dataset. This expanded dataset provides a more comprehensive view of current trends and practices in DevOps pipeline configuration.

3.3 Phase 2

After collecting data in the first phase, we observed an indexation issue with the GitHub REST API search query when using the filename tag. Specifically, the query did not consistently return the same list of repositories, thereby undermining the reliability of our initial data collection method. This inconsistency raised concerns about the completeness and stability of the dataset obtained through the custom search query. To address this problem, we undertook a revision of our query strategy. Rather than relying on a search query that solely targeted repositories containing files with the '.yml' or '.yaml' extensions, we implemented an alternative approach that incorporated additional repository attributes, i.e., popularity and repository creation date. This modification aimed to mitigate the observed indexation issues by providing a more reliable selection criterion.

Prior to commencing the actual data collection process with the modified query, we conducted a thorough validation procedure. The revised query was executed 50 times over a period of 10 days, and the results from each execution were meticulously compared. This repeated testing confirmed the consistency of the returned repository list, thereby demonstrating the effectiveness of the modifications in resolving the indexation issue.

With modified queries, we collected data in two steps as shown in Figure 3.3.

3.3.1 Step A: Refined Query Strategy

In Step A, we concentrated on refining the query strategy to overcome the indexation challenges encountered in Section 3.2. Specifically, we replaced the previous filename based search mechanism, which relied solely on detecting files with the `.yml` or `.yaml` extensions, with a more comprehensive set of selection criteria. A critical modification involved establishing a repository popularity threshold, determined by the number of 'stars' a repository has received. We adopted a minimum threshold of 100 stars to confine the search to projects exhibiting greater activity and a higher probability of containing well-maintained pipeline files. By focusing on repositories with a higher star count, we not only improved the con-

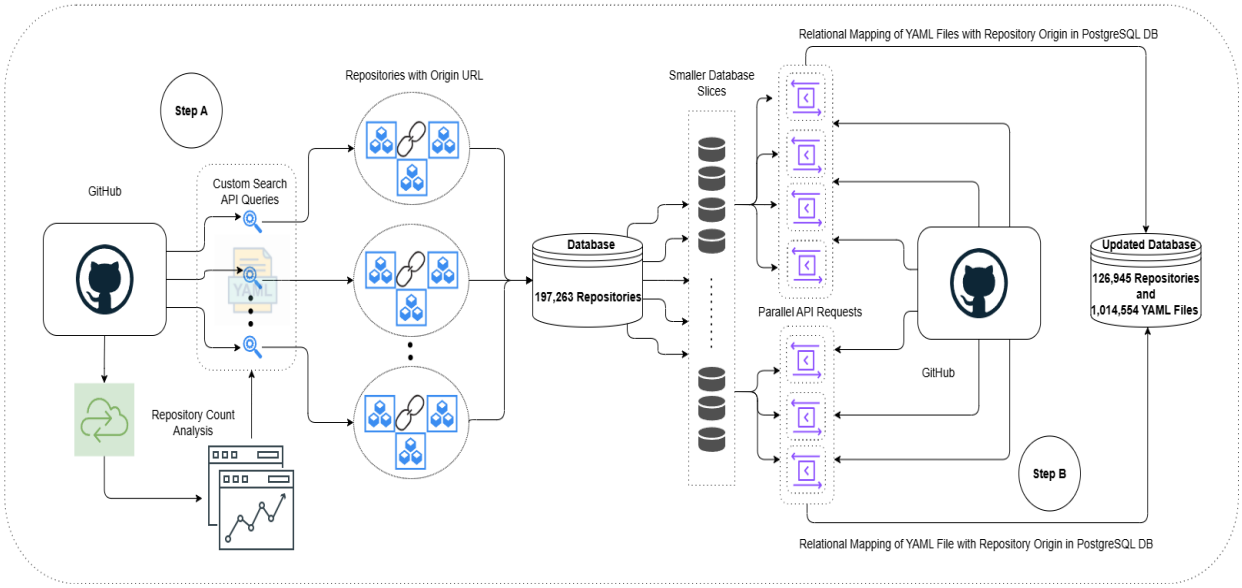

Figure 3.3: Phase 2 Data Collection Process

sistency of the search results but also increased the likelihood of capturing repositories of significant relevance and activity.

Furthermore, as mentioned in the left section of Figure 3.3, the refined query restricted the search to repositories created within the past five years—from October 2018 to October 2023. The decision to limit the search to repositories created in the last five years was motivated by trends observed during the first phase using the GitHub Trends Query and the custom date parameters derived for the GitHub URL Scraping Query, as described in Section 3.2. Our earlier findings revealed that many pipelines developed before the launch of GitHub Actions were predominantly based on Travis CI, reflecting its popularity at that time. Consequently, focusing on more recent repositories ensured that our dataset would more accurately represent current practices in CI/CD pipeline development, particularly those aligned with modern platforms like GitHub Actions.

The data retrieved from these refined queries were subsequently stored in a PostgreSQL database along with the corresponding origin URLs, thereby facilitating efficient data management and future retrieval.

3.3.2 Step B: Parallel Data Retrieval and Storage

In Step B, our focus shifted to the systematic retrieval and storage of YAML files from the repositories identified in Step A. We employed a parallel querying strategy to expedite the collection of YAML files from the previously stored URLs on GitHub. This method involved partitioning the overall dataset into smaller slices, allowing simultaneous data retrieval operations. Within each targeted repository, the search for YAML files was conducted up to the second level of the project directory structure, ensuring that the majority of relevant files were captured without traversing unnecessary depths.

As mentioned in the right section of Figure 3.3, each YAML file retrieved during this phase was recorded in our database along with pertinent metadata, including the file name, directory path, and its corresponding URL. The use of parallel processes not only enhanced the efficiency of data collection but also minimized the impact of GitHub's rate limiting constraints. By systematically storing each file and its metadata in the database, we established a robust and scalable framework that facilitated subsequent pre-processing and analysis steps. This organized storage strategy is critical for ensuring data integrity and for supporting further evaluation of the DevOps pipeline specifications.

3.4 Results

Ultimately for the first phase, as mentioned in the central section of Figure 3.1, we collected 386,859 YAML source files (147,216 from Software Heritage and 239,643 from GitHub) and created a database containing the origin URL for each file and SWHID in case of files collected from Software Heritage for future reference. This whole process took more than 75 hours to complete, involving 12 parallel processes (ten for Software Heritage on an Intel i7 processor with 16GB RAM and two for GitHub on an AMD Ryzen7 processor with 16GB RAM).

In the second phase, as shown in the central part of Figure 3.3, we collected 197,263 repository URLs and stored them in a PostgreSQL database for further operations. Using

Table 3.1: Summary of Dataset Collection Statistics for Software Heritage and GitHub Data Retrieval

Parameters	**Phase 1**		**Phase 2**
	Software Heritage	**GitHub**	**GitHub**
Rate Limit	1,200	5,000	5,000
Number of Repositories	31,456	20,403	197,263
Average Requests per Repository	20	20	22
Average Repositories Checked per Hour	60	250	247
Total Estimated Time in Hours	524.26	81.60	798.7
Parallel Processors	10	2	5
Expected Time in Hours	52.42	40.8	159.7
Actual Time Taken	75.2	41.2	172.4
Total Files Collected	147,216	239,643	1,014,554

parallel processing ultimately collected 1,014,554 YAML files from 126,945 repositories and stored file paths mapped to origin URL. This whole process took more than 170 hours to complete, involving 5 parallel processes (5 processes on an Intel i7 processor with 16GB RAM).

Table 3.1 illustrates the detailed statistics about the dataset scraping process.

3.5 Summary

This chapter presents a detailed account of the dataset collection process, which is critical for the empirical evaluation of our multi-platform editor and metamodel. After a brief overview (Section 3.1), this chapter continues with the Phase 1 data collection process (Section 3.2). Phase 1 employed a dual-source approach, utilizing Software Heritage for its reproducibility and GitHub for its extensive repository data, to collect 386,859 YAML source files. A custom query strategy was implemented to identify repositories containing YAML files, and repository metadata, including origin URLs, was stored for further analysis. Section 3.3 discusses Phase 2 of data collection. Phase 2 addressed indexation challenges encountered with the GitHub REST API by refining the query to target repositories with a minimum of 100 stars and limiting the search to repositories created within the last five years. This

revised strategy, validated over repeated executions, allowed for the consistent retrieval of additional data exclusively from GitHub.

Chapter 4

PolyPipeline DSL Specification

In this chapter, we present the design and evaluation of our generalized metamodel, which serves as a unified representation of CI/CD pipelines across multiple DevOps platforms and insights on parsing accuracy of open source pipelines collected in two distinct phases. More concretely, this chapter aims to investigate the following research question:

RQ 1

To what extent does the proposed domain-specific language represent a generalized abstraction of CI/CD concepts across multiple platforms?

The chapter is organized into three main sections. Section 4.1 details the core metamodel with its key concepts and relationships. Section 4.2 details the enhancements introduced to extend the metamodel from Section 4.1 to support the transformation between different textual syntaxes of DevOps platforms. The core metamodel accommodates the diverse features of platforms such as GitHub Actions, GitLab, Bamboo, BitBucket, Circle CI, and Azure DevOps, while the extended metamodel supports the transformation between GitHub Actions and Azure DevOps.

Section 4.3 outlines the Pre-processing Architecture, the systematic approach employed to transform raw pipeline specifications into a standardized format. This section describes the methods and tools used to filter, validate, and convert YAML files, ensuring compatibility with our multi-platform editor.

Finally, Section 4.4 validates the generalized metamodel and provides a comprehensive evaluation of the metamodel and the pre-processing architecture. Collectively, these sections demonstrate the robustness of the approach and its potential to parse multiple dialect pipelines.

Xtext grammar and editor setup scripts are uploaded in a public GitHub repository and can be accessed[1].

4.1 CI/CD Concept Metamodel Design

We investigate the most common definitions from the GitHub Actions, GitLab, Bamboo, BitBucket, Circle CI, and Azure DevOps platforms, which allows us to understand how pipelines work across different platforms. The proposed metamodel identifies the abstract syntax of these six DevOps automation platform dialects. All these dialects utilize the YAML file format, which makes it easy to identify similarities. We base our work on an existing metamodel previously specified by Corinne Pulgar [5], which is a generic representation of pipelines but is heavily influenced by GitHub Actions and GitLab. To generalize this into a multi-platform text editor, we incorporated additional classes and tested the metamodel's applicability to multiple dialects to ensure it is more adaptable and covers a wider range of features offered by different CI/CD automation platforms. At the end of this section, we discuss the differences to Corinne Pulgar's work in greater detail. Figure 4.1 shows the improved metamodel, with color-highlights for the various supported dialects. The following paragraphs explain the main concepts in the metamodel, while highlighting them in italic.

A standard continuous integration and continuous delivery automation *Pipeline* consists of three main components: basic Configuration, Job or Stage, and Event. Hence, a pipeline specification starts with a declaration of basic pipeline *Configuration* like name, run-name, and default parameters in the case of GitHub Actions, while it starts with a declaration of the pipeline *Configuration* containing global pool details along with *Variable* and *Re-*

[1] `https://github.com/manasbede/DevOps-Dialect-Editor/tree/main/Editor`

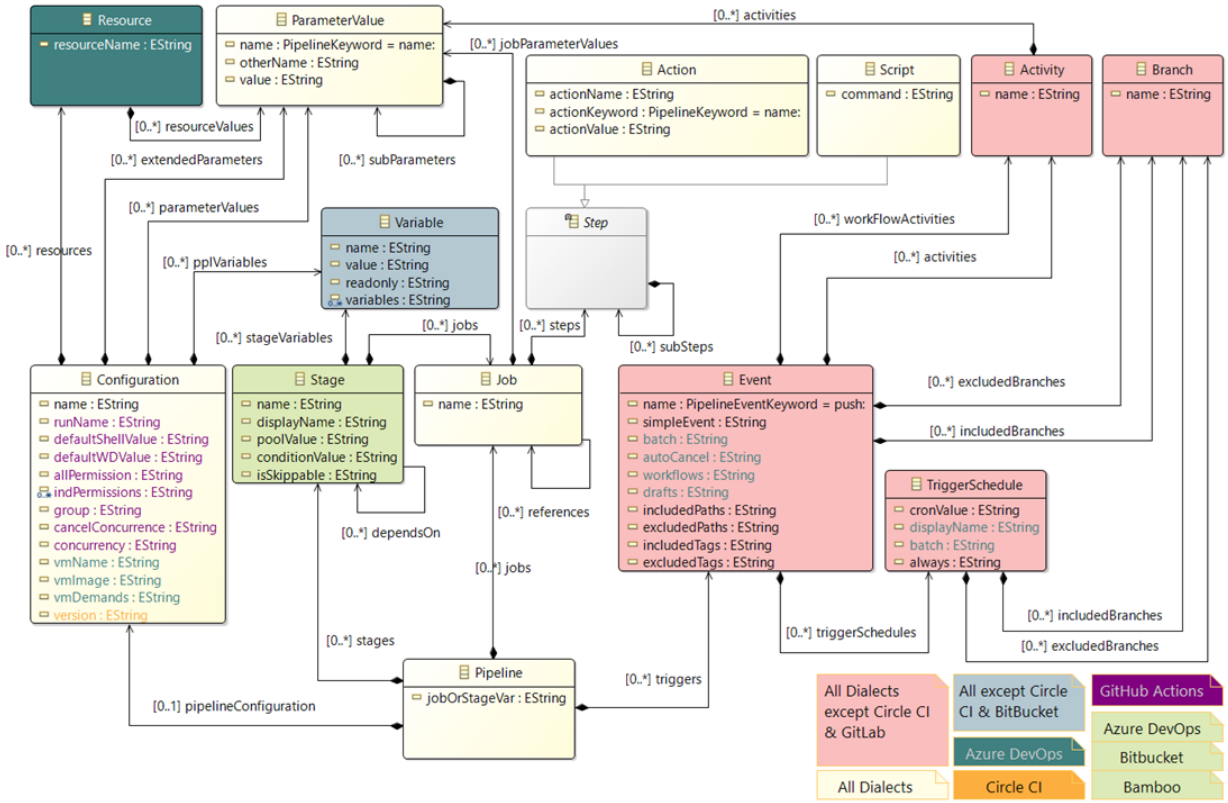

Figure 4.1: Metamodel for Multi-Platform DevOps Language

source declaration in the case of Azure DevOps. The editor implements custom validation rules that examine these initial parameter declarations to automatically detect the dialect. Furthermore, it is designed to throw an exception if a configuration from a different dialect is declared, thereby preventing any mixing of dialects during the specification of a pipeline. Listing 4.1 depicts a simple example of GitHub Actions syntax where line 2 illustrates the declaration of the pipeline name. Listing 4.2 depicts an example of Azure DevOps syntax where lines 2 to 6 illustrate the declaration of the pipeline Configuration using several ParameterValues as explained later in this section. GitLab and Circle CI share many similarities with GitHub Actions, while BitBucket and Bamboo are similar to Azure DevOps, hence we do not show separate examples for them for illustration.

Each pipeline is triggered by a specific *Event*. These Events are defined by the starting keyword "on:" (in the case of GitHub Actions), "triggers" (in the case of Bamboo), and "trigger" (in the case of Azure DevOps), which act similarly to event listeners. Every declared

Event can be periodically scheduled using *TriggerSchedule*. Every TriggerSchedule has a cron attribute to trigger an Event. An Event could be of any type like "push" or "pull" actions performed on branches. Hence, each of these Events contains configurations related to *Branches* and *Activities*. Like Events, TriggerSchedule also contains configurations related to previously declared Branches. The "branches" keyword declared in BitBucket directly acts like a trigger. Whereas, in the case of Circle CI and GitLab, pipelines are defined in a specific branch of the repository. These pipelines are automatically triggered when a push event occurs on that branch, hence they do not explicitly specify triggers or branches.

```
1  # GitHub Actions Example
2  name: Java CI
3  on:
4    push:
5      branches: [‘*’]
6  defaults:
7    run:
8      shell: ‘echo JDK Setup’
9
10 jobs:
11   build-on:
12     permissions: read-all
13     runs-on: ubuntu-latest
14     steps:
15       - uses: actions/checkout@v2
16       - name: Set up JDK 8
17         uses: actions/setup-java@v3
18         with:
19           java-version: 8
20           distribution: zulu
21         cache: maven
22       - name: Build with Maven
23         run: ‘export JAVA8_HOME=${{ steps.setup-java8.outputs.path }}
24         && mvn -B verify
25         -Dgpg.skip=true --global-toolchains
26         .github/workflows/toolchains.xml -Pgithub-workflow’
27   build-jdk17:
28     needs: [build-on]
29     steps:
30       - name: Set up JDK 17
31         run: ‘export JAVA17_HOME=${{ steps.setup-java17.outputs.path }}
32         ./.github/workflow/InstallJDK.sh --status \"SUCCESS\"’
```

Listing 4.1: GitHub Actions Pipeline Syntax Example

In the GitHub Actions example, line 3 starts with an Event listener keyword "on:". Here "push" is the Event name and if it occurs on "*" (any) branch, the Pipeline would automati-

cally trigger. Similarly, in the Azure DevOps, the keywords for an Event are “trigger”, “pr”, and “schedules”. In Listing 4.2 line 8 and 10 shows how Event is declared in Azure DevOps. In a pipeline, an event name can be a value from the PipelineKeyword enumeration or it can be another value. Both, Azure DevOps and GitHub Actions offer hierarchical keyword declaration with “:” (e.g., “push:”, line 4 in Listing 4.1) and also inline keyword declaration (e.g., “push”). To address this declaration pattern we used a name/keyword attribute pair. Only one of name and keyword can be declared at a time. This pattern is followed in the Event and ParameterValue classes.

```
1  # Azure DevOps Pipeline Example
2  parameters:
3  - name: JDKSetup
4    type: stage
5    displayName: Environment Setup for the Project
6    defaults: "export .profile"
7
8  schedules: ‘cron: *****’
9
10 trigger: main
11
12 stages:
13 - stage: JDK8_Setup
14   jobs:
15   - job: pre-setup
16     steps:
17     - uses: ‘./profileLoad’
18       name: Initialize JDK 8 Setup
19       parameters:
20         envFile: ‘.aml/environments/*/environment.yml’
21         validateOnly: true
22         failOnMissing: false
23         workspaceName: ‘$(WORKSPACENAME)’
24 - stage: JDK8_Install
25   jobs:
26   - job: build-on
27     displayName: JDK 8 Installation
28     steps:
29     - script: ‘$HOME/jdksetup8.sh -path $WL_PATH -jarFile $JAR_LOC \
30       jarWls8.jar -Dgpg.skip=true’
```

Listing 4.2: Azure DevOps Pipeline Syntax Example

The next component of the Pipeline is the Stage or Job. An xor constraint between the composition of Pipeline with Stage and Pipeline with Job ensures that both Stage and Job

cannot be present at the top level. Bamboo, BitBucket, and Azure DevOps platform define each main task as a *Stage*, and sub-tasks inside each Stage are defined with the help of Jobs as declared in Listing 4.2 from line 12 to line 30, whereas Circle CI, GitHub Actions, and GitLab group all instructions into Jobs directly, as declared in Listing 4.1 from line 10 to line 32. *Job* is identified by a unique job ID (line 11 in Listing 4.1 and line 15 in Listing 4.2) and information about a virtual machine or operating system on which it is executed (line 13 in Listing 4.1). A Job may be dependent on the execution status of other Jobs (line 28 in Listing 4.1). Apart from the basic job configurations and step declarations, a Job can have additional JobParameterValues consisting of ParameterValues as explained in the second next paragraph.

Each Job contains a series of Steps that need to be executed sequentially. A *Step* could be an Action or Script and may have additional sub-steps of type Action or Script. The *Script* includes command-line instructions that need to be executed and the *Action* includes other configurations like setting environment, default values, strategy matrix, container declaration, etc. In the GitHub Actions example, line 14 starts with the Step declaration block. The first Step is declared on line 15 and includes an Action, the second from line 16 to 21 and includes an Action on line 17, and the third from line 22 to 26 and includes a Script that features a command line instruction. This Script is declared using the "run:" keyword, as mentioned on line 23. Similarly, in the Azure DevOps example, lines 16 to 23 illustrate the Step declaration syntax.

This metamodel introduces *ParameterValue*, which helps declare parameters in the "ParameterName: StringValue" format. ParameterValue is referred to by Configuration, Resource, Job, and Activity. In Configuration, for configuring parameter declarations, it may require parameterValues of type ParameterValue. In a Job, apart from name, steps, and dependent job declarations, other job parameters such as permission, display-name, and runs-on are declared. These parameters are specified as jobParameterValues using ParameterValue. For example, in Listing 4.1, line 13 holds information about the virtual machine or operating system on which it is executed, while line 12 contains details about the permissions

granted to this job. Similarly, in the Azure DevOps example (Listing 4.2), line 27 specifies the displayName as a job parameter, which is of type ParameterValue.

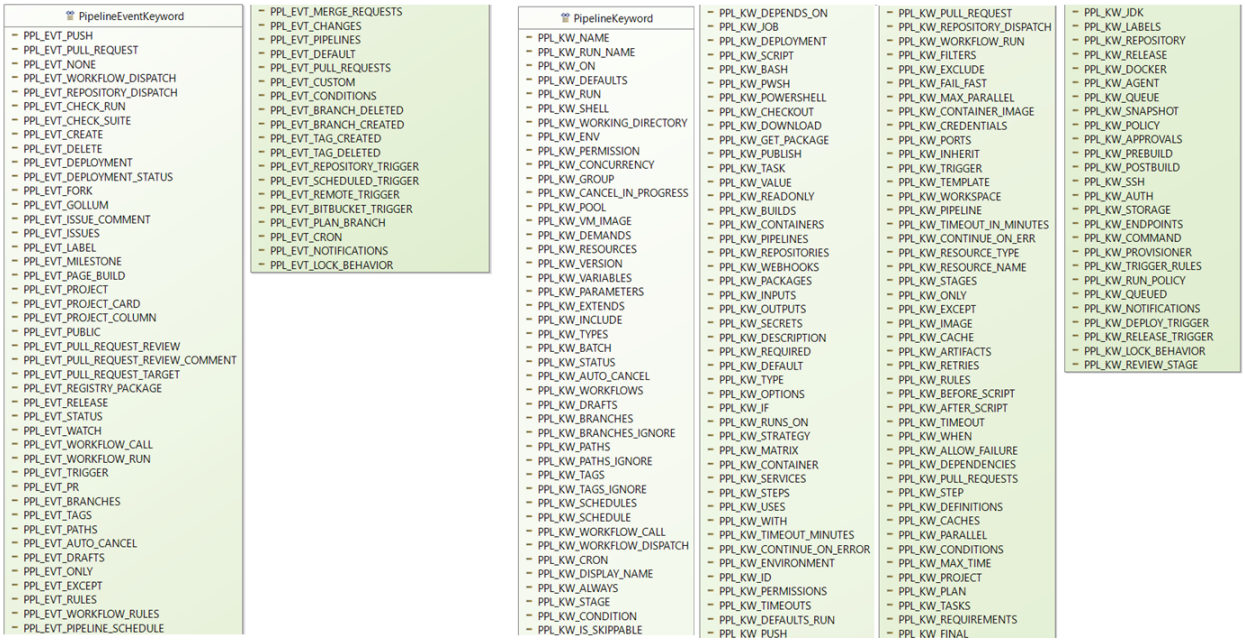

Figure 4.2: Enumerations in Metamodel

The enumeration *PipelineKeyword* includes all possible keywords that can be used for a pipeline specification with GitHub Actions, GitLab, Bamboo, BitBucket, Circle CI, and Azure DevOps platforms. Along with PipelineKeyword to improve Event declaration, the *PipelineEventKeyword* enumeration includes all possible keywords that can be used for Event declaration. Figure 4.2 shows the complete list of literals for these enumerations. This enumeration is designed to restrict the use of keywords while configuring events and defining steps including actions. This enumeration plays a key role in detecting pipelines with malformed specifications.

To avoid use of unsupported keywords while defining pipeline specifications, we have defined a custom validation rule. Our custom validator rule detects dialects and restricts the use of unsupported keywords, in accordance with the defined specifications.

Note that an abstract NamedElement superclass could have been added for all classes in the metamodel; however, it was deliberately omitted for readability reasons. Compared to the previously existing metamodel [5], we introduced several enhancements and additions

to improve its functionality and expressiveness. Specifically, we added the pipeline Configuration class, which is responsible for holding information related to pipeline configurations. Along with Configuration, we introduced the Resource and Variable classes, each serving distinct purposes: the Resource class is designed to store external resource information, while the Variable class is intended for defining and managing global parameters, particularly in the case of Azure DevOps, GitHub Actions, GitLab, and Bamboo. Similarly, we incorporated a Stage class to facilitate the declaration of stages, which is relevant in the context of BitBucket, Bamboo, and Azure DevOps pipelines. Given the introduction of the Stage class, the Xtext-editor enforces syntax restrictions to ensure correctness. Specifically, it guarantees that a pipeline can declare a Stage only if it adheres to the Azure DevOps syntax, while prohibiting its use in cases where GitHub Actions syntax is followed. Additionally, we introduced the ParameterValue class to enhance the reusability of parameter declarations across different classes, including Configuration, Job, and Activity. This ensures a more structured and efficient way of defining parameters within the metamodel. Lastly, we introduced the TriggerSchedule class, which plays a crucial role in capturing automatic event triggers, further extending the capabilities of the metamodel.

Beyond the addition of new classes, we also implemented stricter rules to ensure syntax correctness and consistency across different pipeline definitions. One such improvement is the enforcement of a strict keyword-checking policy through the implementation of enumerations, which helps validate the correct usage of predefined keywords. Furthermore, we refined the structure of the DSL by incorporating additional attributes within classes. For instance, attributes such as defaultShellValue, defaultWDValue, vmName, vmImage, vmDemands, and runName were added to capture more specific details and enhance clarity. Additionally, we improved the organization of the metamodel by introducing new associations such as subSteps and subParameters, which contribute to a more structured and semantically rich DSL representation.

With all these enhancements to the existing metamodel [5], we achieved a more generalized and robust representation of pipelines. Along with modifications to the existing

metamodel, we have created an Xtext-based multi-platform editor, which allows developers to develop pipelines based on GitHub Actions, GitLab, Azure DevOps, BitBucket, Bamboo, and Circle CI. The editor provides syntax validation and keyword highlighting features.

4.2 Enhanced Metamodel Design

The CI/CD concept metamodel provides a unified representation of the most commonly employed CI/CD concepts in pipeline specifications across various DevOps platforms. However, the initial metamodel did not capture all declarative keywords as attributes, which posed challenges when designing M2M transformation rules in ATL defined in Chapter 5. To enable more accurate transformations, we have developed an updated metamodel, as illustrated in Figure 4.3, which highlights the newly introduced attributes in bold. This enhanced metamodel retains all the core concepts and design principles of the original model while introducing small but significant extensions.

The previous version of the metamodel in Section 4.1 is an abstract representation of CI/CD concepts and does not capture concrete keywords as attributes. For example, a Job can have multiple dependent Jobs. In GitHub Actions, the syntax for declaring dependent Jobs is "needs: [job1, job2, ..., jobn]", whereas in Azure DevOps, the syntax is "dependsOn: [job1, job2, ..., jobn]". The previous version of the metamodel did not store the "needs:" or "dependsOn:" keywords as an attribute. This is not an issue for the specification of pipelines, because the Xtext editor enforces the correct concrete syntax. However, this is an issue during the transformation between pipeline dialects. For this transformation, the textual representation is first converted into an XMI (i.e., object-based) representation of the source dialect through built-in Xtext functionality. This representation is then transformed by ATL into the object-based representation of the target dialect. Since the final output of the transformation must be the textual representation of the target dialect, an XMI-to-Text converter needs to be implemented. Although the custom validator method of the Xtext editor can detect the dialect based on the starting keywords and pipeline structure, the XMI-

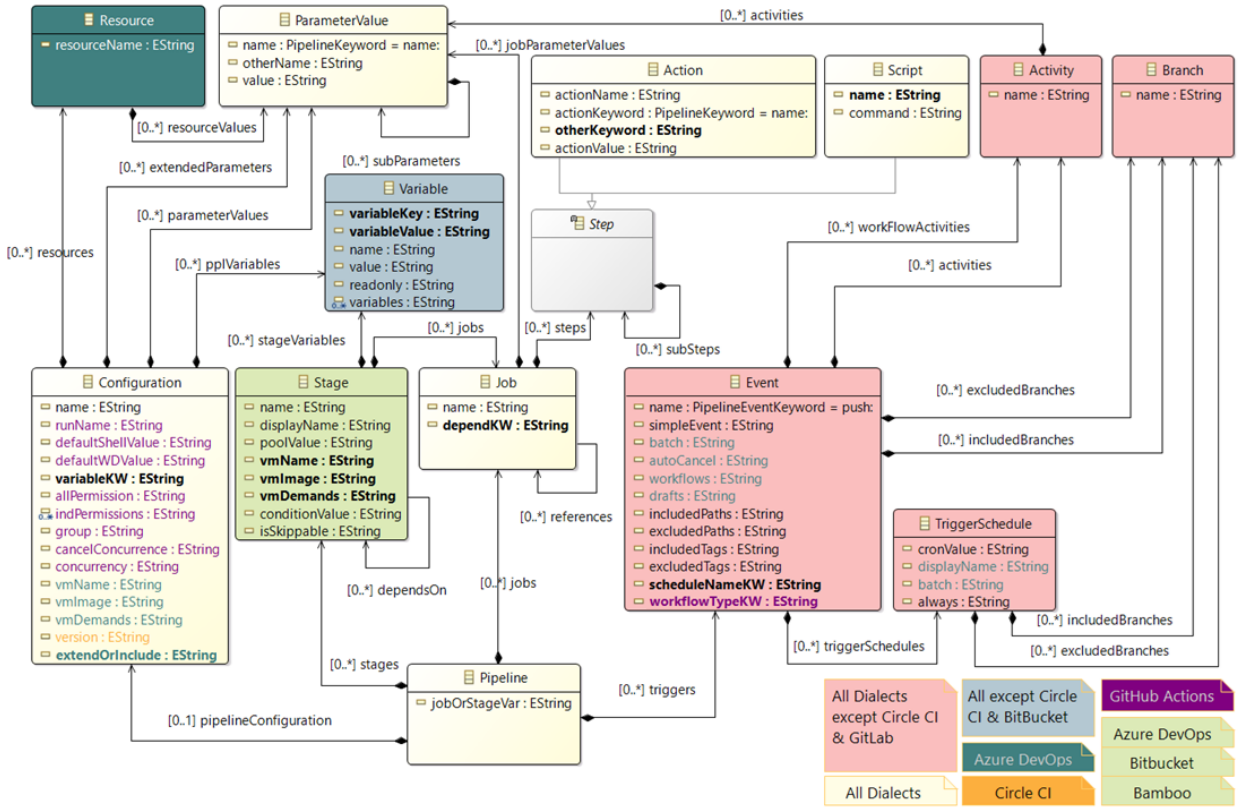

Figure 4.3: Enhanced Metamodel for M2M Transformation

to-Text converter mentioned in Chapter 5 faces an issue, because the XMI representation does not retain concrete syntax information. Without the *dependKW* attribute in Job, the XMI lacks information on whether the dependent Jobs use "needs:" (i.e., the target dialect is GitHub Actions) or "dependsOn:" (i.e., the target dialect is Azure DevOps) in the textual representation. This leads to ambiguity in a generic XMI-to-Text formatter. Most of the metamodel enhancements are due to this issue.

To address the above issue, the enhanced metamodel incorporates additional attributes in the classes *Configuration*, *Script*, *Action*, and *Event*, in addition to the *Job* class. These attributes are essential to capture the keywords declared in the dialect. These keywords help to detect the pipeline dialect, and ultimately help in the implementation of the transformation.

In the enhanced metamodel, the *Configuration* class has an *extendOrInclude* attribute. This attribute stores whether the pipeline configuration has an "include:" or "extends:" keyword declaration, and these keywords refer to the *extendedParameters* association in the

Azure DevOps dialect. While converting XMI to a textual Azure DevOps pipeline, the *extendOrInclude* attribute plays a crucial role in selecting "include:" or "exclude:". Also, in the Configuration class, *variableKW* attribute holds either "env:" keyword from GitHub Actions or "variables:" keyword from Azure DevOps.

A job has multiple steps, and each step could be of type *Action* or *Script*. The *Script* type has a *name* attribute that contains information about the script's start keyword, such as "run:", "script:", or "bash:". In the case of GitHub Actions, the "run:" keyword is used to declare non-login required execution commands, whereas Azure DevOps uses "script:" to declare non-login shell commands to be executed. The previous version of the metamodel did not explicitly capture these keywords but instead allowed the declaration of relevant command keywords directly while declaring the pipeline in the editor. This led to challenges in selecting the correct command keyword during the transformation from XMI to text. An *Action* has an attribute *otherKeyword* which is essential to hold a non *PipelineKeyword* enum value in the case of all dialects. As the previous version of the metamodel did not capture this additional Action keyword, it did not reflect correctly in the XMI representation.

Finally, the *Event* class has the *scheduleNameKW* and *workflowTypeKW* attributes. The GitHub Actions dialect uses "schedule:" to declare the *TriggerSchedule*, and, on the other hand, Azure DevOps uses the "schedules:" keyword for the same. The *scheduleNameKW* attribute holds either "schedule:" or "schedules:". In GitHub Actions, the "workflow_call" and "workflow_dispatch" keywords hold the runtime input values consumed by the pipeline. The *workflowTypeKW* attribute keeps track of these keywords.

Apart from the above issue related to the concrete syntax of keywords, the *Stage* class in the previous version of the metamodel had the *poolValue* attribute to hold complete information of VMs declared in Azure DevOps. However, it did not separate the inside parameters in detail like "name:", "vmImage:", and "demands:". Although this did not impact pipeline specification declaration in the editor, the transformation between Azure DevOps and GitHub Actions must map the "vmImage:" keyword to the "runs-on:" keyword

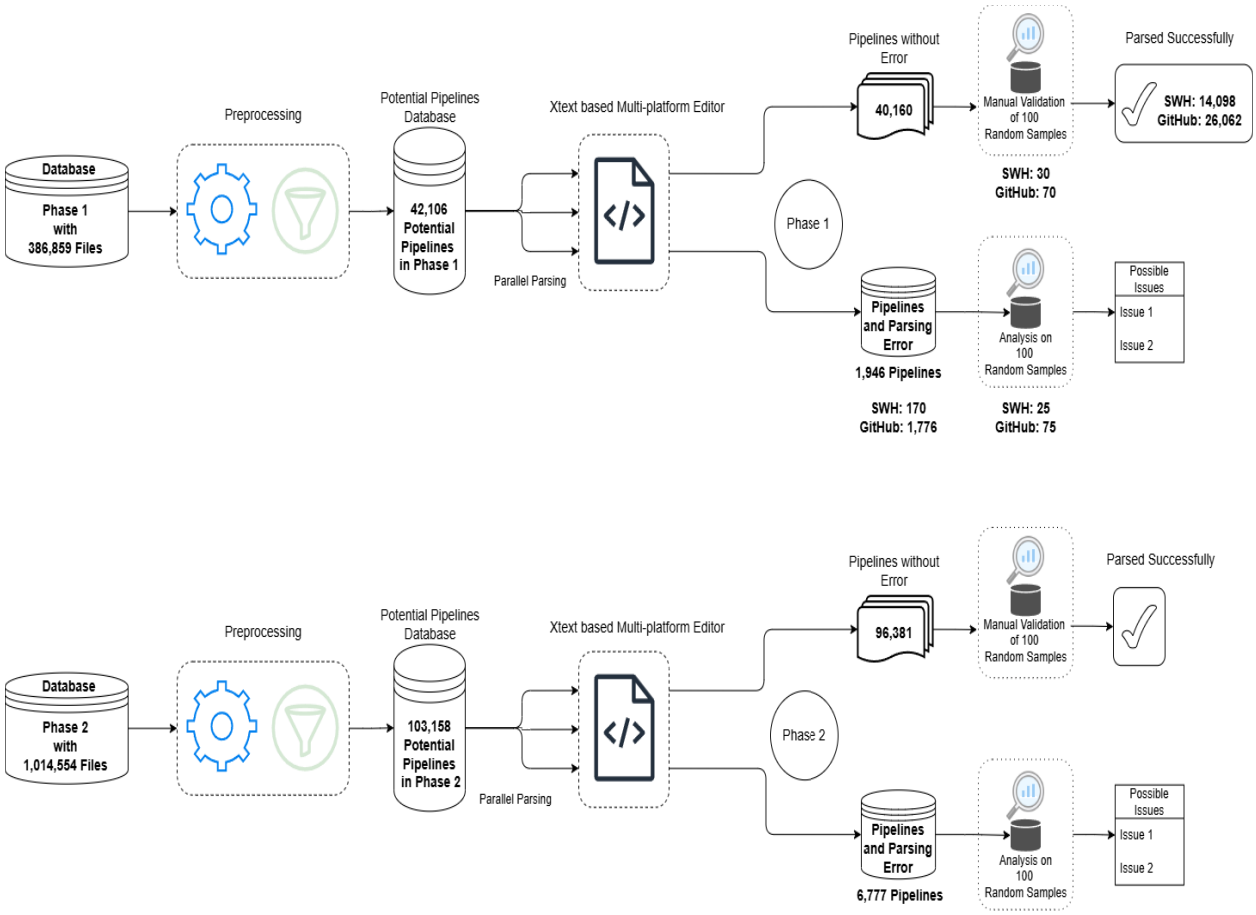

Figure 4.4: Preprocessing and Analysis for Phase 1 (Top) and Phase 2 (Bottom)

declared in Job. To address this issue, the enhanced metamodel incorporates the additional attributes *vmName*, *vmImage*, and *vmDemands* in the *Stage* class.

In the *Variable* class, there are two new attributes, *variableKey* and *variableValue*. These attributes help to map the *defaultShellValue* and *defaultWDValue* attributes from the Configuration class in GitHub Actions to "variables:" of Azure DevOps.

4.3 Dataset Pre-Processing and Analysis

As shown in Figure 4.4, various pre-processing activities were systematically performed to identify potential pipelines from the collected YAML files. To achieve this, all possible keywords were meticulously gathered from the official documentation of multiple CI/CD

platforms, including Azure DevOps[2], GitHub Actions[3], GitLab[4], BitBucket[5], Bamboo[6], and Circle CI[7]. These keywords were collected systematically to ensure a comprehensive understanding of all possible syntax variations and configurations used in different pipeline dialects.

Following the collection of these keywords, each YAML file was parsed using Python, and keywords were extracted directly from the pipeline definitions. This extraction ensured that each keyword was correctly identified and mapped to the predefined keyword lists obtained from the official documentation. The extracted keywords played a crucial role in determining whether a given YAML file corresponded to a pipeline from any of the considered dialects. To classify YAML files based on their pipeline dialect, the extracted keywords were systematically overlapped with the predefined dialect keywords. A strict selection criterion was applied, wherein only those YAML files that contained at least three keywords from one of the predefined dialects were considered. For each YAML file, the count of matching keywords was determined for each dialect, and the file was assigned to the dialect with the highest count of keyword matches. This systematic approach ensured an accurate classification of YAML files based on their structural and syntactic characteristics.

Through this analysis, a significant number of potential pipelines were identified across two phases. As shown in Figure 4.4, 42,106 potential pipelines were identified out of 386,859 pipelines in the first phase and 103,158 potential pipelines were identified out of 1,014,554 pipelines in the second phase. Observations from the first phase revealed that the majority of pipelines were developed on GitHub Actions (29,318), followed by GitLab (7,544), Azure DevOps (3,632), BitBucket (720), Bamboo (176), and Circle CI (98). To support this, we observed similar trends in the second phase too. Since GitHub and GitLab provide version control features, our analysis indicates that GitHub Actions and GitLab are the most widely used platforms for pipeline specification followed by Azure DevOps. In the final stage of

[2] https://learn.microsoft.com/en-us/azure/devops/pipelines/yaml-schema
[3] https://docs.github.com/en/actions/using-workflows/workflow-syntax-for-github-actions
[4] https://docs.gitlab.com/ee/ci/yaml/
[5] https://support.atlassian.com/bitbucket-cloud/docs/bitbucket-pipelines-configuration-reference/
[6] https://docs.atlassian.com/bamboo-specs-docs/7.2.4
[7] https://circleci.com/docs/introduction-to-yaml-configurations/

dataset preparation, additional processing steps were carried out using Python. Specifically, regular expressions within the scripts were checked and reformatted to ensure compatibility with Xtext syntax. Since Xtext does not directly support regular expressions, they were converted into the Xtext String format by enclosing them in either single ('*expression*') or double ("*expression*") quotes. If a regular expression already contained double quotes, it was enclosed in single quotes, whereas if it contained single quotes, it was enclosed in double quotes. This conversion ensured that all regular expressions adhered to Xtext's formatting requirements.

Additionally, a strict indentation policy was enforced across all pipelines due to an existing limitation in the Xtext-based multi-platform editor. This policy was systematically implemented using the PyYAML module, which standardized indentation levels and ensured uniformity in pipeline formatting. The adoption of a strict indentation policy resulted in well-structured and more easily interpretable pipeline specifications, reducing potential parsing errors in the Xtext-based environment.

As the final step in dataset preparation, file extensions were standardized for all potential pipeline files. Specifically, ".Pipeline" was defined as a custom file extension for the editor. The file extensions of all identified pipeline files were converted from ".yaml" or ".yml" to the ".Pipeline" format, ensuring consistency across the dataset. This standardization facilitated seamless integration with the multi-platform editor and enabled a uniform approach to pipeline specification and analysis.

4.4 RQ 1: Evaluation of DSL Generalizability

RQ 1 aims to check the generalizability of the proposed DSL representing CI/CD concepts across multiple platforms. This thesis is interested in presenting generalized representation of different CI/CD concepts used by multiple dialects. This can be evaluated if pipelines defined in different dialects parse with this representation with minimum pre-processing. To validate the same, this section presents the final outcomes of the dataset collection and

parsing process, highlighting the proportion of pipelines successfully parsed and the primary causes of parsing failure.

4.4.1 Validity of Phase 1 Dataset

At the end of the dataset collection phase 1, as mentioned in Figure 4.4, a total of 42,106 (14,268 SWH, 27,838 GitHub) potential pipelines of different dialects were identified from the total of 381,859 YAML files collected from the GitHub and Software Heritage collection archives. To parse all pipelines, a custom Java method was defined based on EMF/Xtext's resource loader and validator. This custom method was divided into four parts.

1. Create a custom resource loader from the metamodel
2. List all files from the source directory and create URI
3. Load resources by resolving the URI and with the custom resource loader
4. Generate a report after validating loaded resources with the resource validator

In the validation part, each parsing error was stored for further analysis. Due to the DSL's handling of indentation rules, the editor sometimes did not correctly identify the end of the file (EOF), as shown in Listing 4.5 line 43. As this error does not prevent (a) the file from being loaded properly and (b) any further analysis of the file, we decided to consider files with an EOF error to be correctly parsed. Further enhanced metamodel with additional captured keywords helped to rectify this false positive error. From the final findings shown in Figure 4.4, 40,160 (14,098 SWH, 26,062 GitHub) pipelines were parsed correctly, i.e., 95.37% of the pipelines. To ensure that the parsed pipelines are correctly structured pipelines, we manually cross-checked 100 random samples (30 SWH, 70 GitHub) from the pipelines that were parsed successfully. During manual validation, we did not find any ambiguity, error, or syntax error in the specifications.

Similarly, checking all of the remaining 1,946 (170 SWH, 1,776 GitHub) files manually was infeasible in the available time, hence we again opted to examine a randomly selected

sample of 100 (25 SWH, 75 GitHub) files that did not parse correctly. A detailed statistical analysis of issues identified in the random samples shows that the following issues were observed most prominently:

1. In 56% of the cases, pipeline specifications were malformed, for example:

   (a) Declaration of Stages and individual Jobs at the same level in the same pipeline. In Listing 4.3, line 10 has a single Job declaration and line 17 has a Stages declaration. When Stages are declared, Jobs have to be nested inside Stages.

   (b) As shown in the example in Listing 4.4, lines 3 and 22 both declare events with the "on" keyword. The "on" keyword can be used only once to declare events after declaring the pipeline name and basic configurations. Only the first declaration is valid.

   (c) As shown in Listing 4.5, "on" keyword to indicate an event in job step declaration. In the GitHub Actions declaration, the "on" keyword can be used to declare events after declaring the pipeline name and basic configurations but not for a step of a job.

   (d) "defaults" declaration in event declaration. The "defaults" keyword provides an option to run a default command-line instruction, and it can be only declared with job parameters or pipeline configurations but not with events.

   (e) Declaration of "run-name" at the end instead of the beginning of the pipeline.

2. In 44% of the cases, the files had a different intended use. After manually checking the files and their corresponding projects, we found that most of these files comprised source configurations for various executable scripts. Although the files had specifications that closely resembled those of the pipeline, their purpose was distinct. Listing 4.6 is an example of such kind of file. This file is a source file for another programming script. Lines 6, 7, 59, and 60 are custom keywords defined for the other programming script and this programming script refers to the pipeline syntax defined from lines 8 to 29 and from 61 onward as functions.

```yaml
parameters:
- name: CIPipeline
  type: Data
trigger: 'none'

pool: vmImage: ubuntu-latest

variables: template: variables.group.yaml

jobs:
- job: Preintegrate
  displayName: Environments -IntegrationPreStep
  steps:
  - template: templates/aml-cli-install/step.yaml


stages:
- stage: aml_ensure_env
  displayName: 'Environments - Integration'
  jobs:
    - job: integrate
```

Listing 4.3: Error While Declaring Job and Stage at Same Level

```yaml
name: k8s-test

on: [push, pull_request]

jobs:
  apply-all-yaml:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v2
      - name: install kubeconform
        run: |
          'curl -L https://github.com/yannh/kubeconform/releases/latest/download/kubeconform-linux-amd64.tar.gz \
          | tar xz && sudo mv kubeconform /usr/local/bin &&
          kubeconform -v'
      - name: check all yaml
        run: |
          './bin/test.sh | tee -a apply.txt'
      - name: check apply log for error
        run: |
          'if [ -s apply.txt ]; then exit 1; fi'

on:  [push]
```

Listing 4.4: Multiple Times Same Keyword Declaration

```yaml
1 name: build-airflow-exporter
2
3 jobs:
4   docker:
5     on:
6       push:
7         branches: [main]
8     runs-on: ubuntu-latest
9     steps:
10      - name: Set up data
11        uses: docker/setup-qemu-action@v1
12
13      - name: Login to DockerHub
14        uses: docker/login-action@v1
15        with:
16          username: ${{ secrets.DOCKERHUB_USER }}
17          password: ${{ secrets.DOCKERHUB_TOKEN }}
18
19      - uses: actions/checkout@v2
20
21      - name: Build and push latest
22        run: make airflow-exporter
23        env:
24          DOCKERHUB_USER: ${{ secrets.DOCKERHUB_USER }}
25          DOCKERHUB_PASS: ${{ secrets.DOCKERHUB_PASS }}
26          DOCKERHUB_TOKEN: ${{ secrets.DOCKERHUB_TOKEN }}
27
28      - name: Report Status
29        if: always()
30        uses: ravsamhq/notify-slack-action@v1
31        with:
32          status: ${{ job.status }}
33          notify_when:  'failure'
34        env:
35          SLACK_WEBHOOK_URL: ${{ secrets.SLACK_WEBHOOK_URL }}
36
37      - name: Build and test latest
38        run: test airflow-exporter
39        env:
40          DOCKERHUB_USER: ${{ secrets.DOCKERHUB_USER }}
41          DOCKERHUB_PASS: ${{ secrets.DOCKERHUB_PASS }}
42          DOCKERHUB_TOKEN: ${{ secrets.DOCKERHUB_TOKEN }}
43          uses:  ravsamhq/notify-slack-action@v2
```

Listing 4.5: Unexpected Keyword Error (Line 5) and EOF Error (Line 43)

```
# Automations configurations for Multiple CI/CD Pipelines for Actions-Helper Content
# Bash Script Reference: SystemARM.ksh
# Execution Step : $HOME/ARM/WSD/$WORKSPACE/Pipeline/SystemARM.ksh -step 'ScriptTriggerMainDeployment'

step: 'ScriptTriggerMainDeployment'
execution:
  name: CI/CD Release Pipeline
  on:
    push:
      branches:
        - master
  jobs:
    build-and-deploy:
      runs-on: ubuntu-latest
      steps:
        - name: Git Checkout
          uses: actions/checkout@v2

        - name: Setup Hugo
          uses: peaceiris/actions-hugo@v2
          with:
            hugo-version: 0.71.0
            extended: true

        - name: Setup Azure Service Principal
          uses: azure/login@v1.1
          with:
            creds: ${{ secrets.AZURE_CREDENTIALS }}
    ......
step: 'CDTrigger'
execution:
  name: '[DEPLOY] Deploy Azure Functions'

  on:
    workflow_call:
      inputs:
        SERVICE_DIRECTORY:
          type: string
          required: true
          description: |
            A path from repository root to the Azure Functions directory.
            This directory should contain the `package.json` and `host.json` files.
        BUILD_WORKFLOW_NAME:
```

Listing 4.6: Source File for Other Program

Note that, in all of the above issues, the editor was supposed to flag these files as invalid. Based on the limited random sample selected for valid and invalid files, the editor appears to correctly identify valid but also invalid pipeline specifications.

4.4.2 Validity of Phase 2 Dataset

In Phase 2 (Section 3.3), as shown in Figure 4.4, we analyzed 1,014,554 YAML files, all from the GitHub open-source platform, identifying 103,158 as potential pipelines. Using the same Java-based parser and validator employed in Section 4.4.1, 96,381 files (93.45%) were successfully parsed. To verify parsing accuracy, we randomly sampled 100 files from those successfully parsed; each was confirmed to be a valid CI/CD pipeline, indicating that the parsing mechanism consistently handles conventional YAML structures and adheres to the multi-platform metamodel specifications.

A total of 6,777 files (6.55%) did not parse successfully. Since manually checking every unparsed file was infeasible, we conducted a random inspection of 100 such files to diagnose the most frequent issues. Through this inspection, we identified three main categories of failures:

1. Malformed Pipeline Specifications in 41% of Cases:

    (a) Conflicting Hierarchies: Multiple high-level constructs (such as "stages", "jobs", or "triggers") appeared at the same hierarchy level, rather than nesting them according to the metamodel's rules. This resulted in ambiguity about the sequence in which tasks were meant to run.

    (b) Repeated or Misused Keywords: Some files invoked the same keyword (for example, "on" or "trigger") multiple times in separate sections, or placed keywords (such as "run-name") under the wrong category, causing the parser to reject the file's structure.

    (c) Invalid Pipeline Parameters: Various environment variables or job parameters were declared in non-compliant sections, generating syntactic conflicts with the expected pipeline format.

    (d) Legacy or Deprecated Keys: Certain files used keywords from older CI/CD configurations (for example, Travis-based references) within a GitHub Actions pipeline, introducing incompatibilities with the updated metamodel.

(e) Syntax Errors from Inline Scripts: Certain files contained script blocks written in a way that conflicted with the parser's assumptions about indentation and keyword usage, creating parse-time errors.

2. Non-Pipeline Files in 53% of Cases:

   (a) Configuration or Script Files: Many YAML files were intended to configure applications or scripting environments and only incidentally used terminology that resembled pipeline syntax. These references confused the detection logic, causing the files to be flagged as pipeline candidates.

   (b) Partial Pipeline-Like Sections: A few files embedded snippets that looked like fragments of a CI/CD pipeline, but they were supplemented by large segments of unrelated code or documentation. As a result, the overall structure did not align with the expected pipeline schema.

   (c) Mixed Schema Definitions: Some YAML files blended CI/CD syntax with other schemas such as Docker Compose or application configuration, leading to ambiguous declarations that the parser could not interpret consistently.

3. Editor Limitation in 6% of Cases:

   (a) Unsupported YAML Constructs for Pipelines: Advanced YAML features such as anchors, aliases, or inter-pipeline references were used where the parser strictly expected simple pipeline statements, thereby complicating or invalidating the overall file structure.

Overall, 93.45% of the pipeline files identified in Phase 2 were parsed successfully, and the random checks confirmed that these files indeed represented valid CI/CD pipelines. The remaining 6.55% were primarily invalid configurations or non-pipeline YAML files. But in a few samples, the pipeline used cross-reference parameters from other pipelines, which is accepted by the dialect but currently not supported by this editor. These findings confirm that the parser remains effective for the majority of modern GitHub Actions, Azure DevOps,

GitLab, BitBucket, Bamboo, and Circle CI pipelines, while also highlighting that ongoing refinement is needed to accommodate advanced YAML constructs and inter-pipeline references from real-world pipeline configurations.

4.5 Summary

In conclusion, this chapter answers the first research question on evaluation of DSL generalizability and provides a comprehensive overview of the metamodel's evolution and the subsequent parsing outcomes of two distinct data collection phases. Section 4.1 details the core metamodel with its key concepts and relationships. Section 4.2 details the enhancements introduced to extend the metamodel from Section 4.1 to support the transformation between different textual syntaxes of DevOps platforms. In both Section 4.3 and Section 4.4, the observed parsing success rates underscore the ability of the metamodel to parse only valid pipelines from different dialects. The next chapter presents the ATL-based transformation for inter-dialect migration between the Azure DevOps and GitHub Actions platforms and discusses the transformation results observed for the pipelines collected in Phase 1 and 2.

Chapter 5

Inter-Dialect Transformation of Pipeline Specifications

This chapter aims to present an ATL-based model-to-model transformation to achieve inter-dialect migration within the GitHub Actions and Azure DevOps dialects. Furthermore, this chapter aims to evaluate dialect migration feasibility by preserving dialect-specific features. More concisely, this chapter aims to investigate the following research question:

RQ 2

To what extent does this DSL environment help with migration between pipeline dialect specifications while preserving functionality flow and incorporating dialect-specific features?

This chapter begins with an M2M ATL transformation approach for GitHub Actions to Azure DevOps in Section 5.1. Subsequently, the M2M transformation approach for Azure DevOps to GitHub Actions is discussed in Section 5.2. This transformation process takes pipeline specifications in XML Metadata Interchange (XMI)[1] format as input and the generated output is also in XMI format. Xtext provides a generic conversion from textual format to XMI, but does not convert XMI back to the original text format. Hence Section 5.3 describes a custom formatter method declared using Java to convert the transformed XMI

[1] `https://www.omg.org/spec/XMI/`

version of a pipeline into textual format. Finally, Section 5.4 of this chapter provides insights on the observed results from the transformation method applied to GitHub Actions and Azure DevOps pipelines collected from Phase 1 and Phase 2.

Transformation scripts are uploaded in a public GitHub repository and can be accessed[2].

5.1 GitHub Actions to Azure DevOps Transformation

As described in Chapter 4, we have successfully represented generalized and robust CI/CD concepts in a single unified metamodel. This metamodel is now capable of parsing multiple DevOps dialects, including Azure DevOps and GitHub Actions, thereby supporting our hypothesis that it can be used to address the inter-dialect migration issue. For outplace model-to-model transformations, ATL is widely recognized as one of the most effective platforms, particularly due to its structured approach to converting domain-specific elements between metamodels. Consequently, as a first step in tackling the inter-dialect migration challenge, we chose to employ the ATL platform and the metamodel outlined in Figure 4.3 to facilitate GitHub Actions–to–Azure DevOps dialect migration.

As noted in Chapter 3 and Chapter 4, GitHub Actions and Azure DevOps stand out as the most widely used pipeline specification platforms, which is why we began our efforts by focusing on GitHub Actions to Azure DevOps transformations. By initially concentrating on this prominent migration path, we aim to demonstrate the metamodel's adaptability and the ATL platform's capabilities, paving the way for more extensive cross-dialect support in future phases.

The DevOps pipeline has three primary components: *Pipeline* and its *Configuration*, *Job* and *Stage*, and *Event*. The ATL-based transformation script, which relies on rule-based concept mapping, uses both the input and output metamodel defined in the "Pipeline.ecore" file (introduced in Section 4.2). This transformation script begins by applying the *Pipeline* and *Configurations* rule, which maps the configurations defined in GitHub Actions to those

[2] `https://github.com/manasbede/DevOps-Dialect-Editor/tree/main/Transformation`

in Azure DevOps. This rule maps a *Pipeline* class in GitHub Actions to a *Pipeline* class in Azure DevOps. Furthermore, the *Configuration* class in GitHub Actions is mapped to the *Configuration* class in Azure DevOps.

In the *Configuration* class, the rule starts by mapping the *name* attribute. Since Azure DevOps also uses a *name* attribute to store the pipeline name, the transformation simply assigns "name:" to "name:" as shown on line 1 in Listing 5.1 and Listing 5.2. The next *Configuration* attribute, *runName*, is specific to GitHub Actions and does not have a direct alternative in Azure DevOps pipeline syntax. Likewise, *defaultShellValue* and *defaultWD-Value* are GitHub Actions attributes that store global values and are therefore mapped to the *Variable* class in Azure DevOps. The "env:" keyword in GitHub Actions holds all global parameters used by subsequent sections of the pipeline. In the metamodel, these parameters are declared under *Variable* class. Similarly, in Azure DevOps any global parameters are declared using "variables:" keyword and declared under the *Variable* class in metamodel. Thus, all "env:" parameters are mapped to "variables:" by linking the *variableKW* attribute in the *Configuration* class to a "variables:" value. Listing 5.1 has "env:" declaration on line 2-6, which is mapped to "variables:" on line 2-6 in Listing 5.2. Another keyword in GitHub Actions, "permissions:", has no equivalent in Azure DevOps, because permissions and access control are managed through the user interface rather than inline declarations. Similarly, the "concurrency:" and "cancel-in-progress" keywords declared in GitHub Actions do not have any direct equivalent in Azure DevOps. Table 5.1 shows in detail the *Pipeline* and *Configuration* mappings.

Next, the translation script has a rule to map *Events* defined in GitHub Actions to *Events* defined in Azure DevOps (called triggers in Azure DevOps). In GitHub Actions, all *Events* are declared by starting with the "on:" keyword. An *Event* could be "push:", "pull_request:", or "schedule:" in GitHub Actions. Each "push:" *Event* declared in the GitHub Actions pipeline creates an *Event* in Azure DevOps and assigns its *name* attribute to the "trigger:" keyword. For an *Event* of type "pull_request:" declared in GitHub Actions, an *Event* is created in Azure DevOps with *name* attribute assigned to "pr:". Listing 5.1 has "push:"

Event declared on line 9 and "pull_request:" declared on line 20. These *Events* are respectively mapped with "trigger:" and "pr:" in Azure DevOps as mentioned on lines 7 and 19 in Listing 5.2. An *Event name* can be declared inline in the editor and it is stored in the *simpleEvent* attribute. Hence, the *simpleEvent* from GitHub Actions is mapped to the Azure DevOps *simpleEvent*.

```
name: build-snowflake-exp
env:
  VMIMAGENAME: ubuntu-latest
  REPO_NAME: "$(Build.Repository.Name)"
  BRANCH_NAME: "$(Build.SourceBranchName)"
  System.Debug: 'true'

on:
  push:
    branches:
      - "develop/*"
      - "feature/*"
      - master
    branches-ignore:
      - "releases/old*"
    paths-ignore:
      - docs/
      - README.md

  pull_request: none
```

Listing 5.1: GitHub Actions Configurations and Events

Each *Event* can have multiple included or excluded branches, which are associated with the *Branch* class through the *includedBranches* and *excludedBranches* associations. All included and excluded branches are mapped exactly as they are when transforming to Azure DevOps. After branches, *includedPaths*, *excludedPaths*, *includedTags*, and *excludedTags* attributes are mapped as they are defined in the GitHub Actions event. At last, the "schedule:" keyword in GitHub Actions is replaced by "schedules:" and assigned to the *scheduleNameKW* attribute in an event. A *cronValue*, which indicates the event's auto trigger value, is stored inside the *TriggerSchedule* class, and this class is associated with the *Event* via the *triggerSchedules* association. Hence, this transformation script has a rule to map *TriggerSchedule* from GitHub Actions to *TriggerSchedule* in Azure DevOps. Lines 10-18 from Listing 5.1

Table 5.1: Mapping of Pipeline and Configuration Keywords and Metamodel Elements

Configuration Keywords		Metamodel Elements	
GitHub Actions	**Azure DevOps**	**GitHub Actions**	**Azure DevOps**
–	–	Pipeline	Pipeline
–	–	Configuration	Configuration
name:	name:	name	name
run-name:	displayName:	runName	runName
defaults: run: shell:	variables:	defaultShellValue	pplVariables
defaults: run: working-directory:	variables:	defaultWDValue	pplVariables
env:	variables:	variableKW, pplVariables	variableKW, pplVariables
secrets:	variables: secret:	parameterValues	pplVariables
permissions:	–	allPermission, indPermission	–
concurrency:	–	concurrency	–
cancel-in-progress:	–	cancelConcurreny	–
group:	–	group	–

and lines 8-18 from Listing 5.2 illustrate clearly how branches and paths defined in GitHub Actions are transformed to Azure DevOps.

```
1  name: build-snowflake-exp
2  variables:
3    VMIMAGENAME: ubuntu-latest
4    REPO_NAME: "$(Build.Repository.Name)"
5    BRANCH_NAME: "$(Build.SourceBranchName)"
6    System.Debug: 'true'
7  trigger:
8    branches:
9      include:
10       - "develop/*"
11       - "feature/*"
12       - master
13     exclude:
14       - "releases/old*"
15   paths:
16     exclude:
17       - docs/
18       - README.md
19 pr: none
```

Listing 5.2: Transformed Azure DevOps Configurations and Events

Now, looking at the rest of the attributes defined in the *Event* class, *batch*, *autoCancel*, *workflows*, and *drafts* are not defined in GitHub Actions. Also, "workflow_call:" and "work-

flow_dispatch:" GitHub Actions keywords assigned to the *workflowTypeKW* attribute do not have any specific equivalent in Azure DevOps. The "workflow_dispatch:" keyword declared in GitHub Actions allows manual triggering directly from GitHub's UI with custom input parameters. This behavior is equivalent to setting "trigger: none" in Azure DevOps and defining runtime parameters. Similarly, *workFlowActivities* and *activities* from *Event* do not have any direct equivalent in Azure DevOps.

In summary, the *Event* rule clearly maps the event configurations between GitHub Actions and Azure DevOps by preserving branch filters, paths, tags, and schedules, while acknowledging that some GitHub-specific parameters do not have a direct counterpart in Azure DevOps. Table 5.2 shows in detail the *Event* mapping.

Table 5.2: Mapping of Event Keywords and Metamodel Elements

Event Keywords		**Metamodel Elements**	
GitHub Actions	**Azure DevOps**	**GitHub Actions**	**Azure DevOps**
–	–	Event	Event
–	–	simpleEvent	simpleEvent
–	–	activities	–
–	–	workflowActivities	–
on: push:	trigger:	name	name
on: pull_request:	pr:	name	name
on: schedule:	schedules:	scheduleNameKW, TriggerSchedule	scheduleNameKW, TriggerSchedule
on: workflow_dispatch:	trigger: none	workflowTypeKW	name
branches:	branches: include:	includedBranches	includedBranches
branches-ignore:	branches: exclude:	excludedBranches	excludedBranches
tags:	tags: include:	includedTags	includedTags
tags-ignore:	tags: exclude:	excludedTags	excludedTags
paths:	paths: include:	includedPaths	includedPaths
paths-ignore:	paths: exclude:	excludedPaths	excludedPaths

Lastly, the main component of the pipeline is *Jobs* or *Stages*. GitHub Actions do not have *Stages*, and all execution instructions are defined in *Steps*, which are in turn defined in *Jobs* directly. However, Azure DevOps offers *Stages* that contain *Jobs*, and each *Stage* represents a main task. Each *Stage* is executed sequentially, as outlined in the specifications. With this logic, all interdependent *Jobs* are assigned to different *Stages*. By default, all non-dependent

Jobs declared in GitHub Actions are executed in parallel. Hence, all non-dependent *Jobs* are mapped to a single *Stage*, allowing them to run concurrently where supported. If a *Job* depends on another *Job*, that *Job* is linked to the next *Stage*. If a *Job* depends on more than one *Job*, the *Stage* mapping is decided based on the *Stage* mapping of the *Jobs* on which it depends.

Now, from the perspective of rule application, an initialization step uses two lists: one for *Jobs* depending on other *Jobs* for their execution (dependentJobs), and another for *Jobs* with no dependencies (independentJobs). During this initialization, the main rule creates a *Stage*. It then calls two additional rules, depending on whether a *Job* has dependencies. For *Jobs* without any dependencies, the JobtoDefaultStage rule is invoked, taking the list of independent *Jobs* and a predefined *Stage* as arguments.

Next, the main rule calls the JobtoNewStage rule, providing a parameter consisting of dependentJobs and the name of the subsequent *Stage*. This rule generates a new *Stage* with the specified name and checks whether each *Job*'s dependency is no longer in the current dependentJobs list. It also keeps a temporary list tracking *Jobs* that have already been assigned to *Stages*, removing them from the helper variable once they are mapped. If the list remains non-empty, the same function is called recursively with the updated dependentJobs list and a new *Stage* name, continuing until the helper variable contains no further dependent *Jobs*. Listing 5.3 has "config", "deploy", "test", and "deploy-jdk" jobs (lines 22-102). "deploy" job is dependent on "config", and "test" job is dependent on "deploy" job. After transformation to Azure DevOps as shown in Listing 5.4, "config" and "deploy-jdk" jobs are mapped to "stage0" (default stage), "deploy" job is mapped to "stage1", and finally "test" job is mapped to "stage2".

A *Job* has a *dependKW* attribute, which stores "needs:" for GitHub Actions or "dependsOn:" for Azure DevOps. Line 34 in Listing 5.3 refers to the "needs:" keyword and it is mapped to "dependsOn:" keyword as mentioned on line 49 in Listing 5.4. When transforming *Jobs* from GitHub Actions to Azure DevOps, all *references* are preserved exactly as in the original specification. In addition, each *Job* may include additional configuration pa-

rameters of type *ParameterValue*, referenced by the *jobParameterValues* association. These parameters are dialect-specific, so not all of them can be directly mapped to Azure DevOps. Consequently, we created a rule that only generates a *ParameterValue* for those parameters with a direct equivalent in Azure DevOps. As shown in Table 5.3, the corresponding GitHub Actions Job parameters are mapped to appropriate Azure DevOps keywords. The same approach applies to *subParameters*.

Table 5.3: Mapping of Job Keywords and Metamodel Elements

Job Keywords		**Metamodel Elements**	
GitHub Actions	**Azure DevOps**	**GitHub Actions**	**Azure DevOps**
–	–	Job	Stage, Job
name:	displayName:	name	name
runs-on:	pool:	jobParameterValues	jobParameterValues
if:	condition:	jobParameterValues	jobParameterValues
needs:	dependsOn:	references, dependKW	references, dependKW
container:	container:	jobParameterValues	jobParameterValues
services:	services:	jobParameterValues	jobParameterValues
strategy:	strategy:	jobParameterValues	jobParameterValues
strategy: matrix:	strategy: matrix:	jobParameterValues, subParameters	jobParameterValues, subParameters
strategy: max-parallel:	strategy: maxParallel:	jobParameterValues, subParameters	jobParameterValues, subParameters
continue-on-error:	continueOnError:	jobParameterValues	jobParameterValues
timeout-minutes:	timeoutInMinutes:	jobParameterValues	jobParameterValues
uses:	uses: repositories:	jobParameterValues	jobParameterValues. subParameters
defaults:	–	jobParameterValues	–
with:	variables:	jobParameterValues	jobParameterValues

Every *Job* uses one or more *Steps* to specify instructions. Each *Step* can be an *Action* or a *Script*, where an *Action* contains tasks essential for preparing the environment to run instructions, and a *Script* holds the actual commands required to complete the task. *Steps* may include *subSteps* of any type (*Action* or *Script*).

```
22 jobs:
23   config:
24     uses: aplaceformon/workflows/.github/workflows/default-config.
    yaml@main
25     with:
26       deployment-role: ${{ inputs.deployment-role || null }}
27       application-role: ${{ inputs.function-role || null }}
28       aws-region: ${{ inputs.aws-region || null }}
29       env: ${{ inputs.env }}
30   steps:
31     - run: "./activate profile"
32   deploy:
33     runs-on: ubuntu-latest
34     needs: [config]
35     steps:
36       - uses: actions/download-artifact@v3
37         with:
38           name: ${{ inputs.artifact }}
39           path: .
40       - id: debug
41         run: "Debug lambda artifact\nunzip -l lambda.zip\nfind .\n"
42       - uses: aws-actions/configure-aws-credentials@v2
43         with:
44           role-to-assume: ${{ needs.config.outputs.deployment-role }}
45           aws-region: ${{ needs.config.outputs.aws-region }}
46       - id: Lambda
47
48 .....
81 set -- \"$@\" --timeout '${{ inputs.timeout }}'\nset -- \"$@\"
    --memory-size \
82 '${{ inputs.memory }}'\nset -- \"$@\" --role '${{ inputs.function-role
    }}'\n\
83 set -- \"$@\" --description '${{ inputs.description }}'\naws lambda
    update-function-configuration \
84 \"$@\""
85   test:
86     runs-on: ubuntu-latest
87     needs: [deploy]
88     steps:
89       - run: "$HOME/ginger/sanity/non-prod-single.sh"
90
91   deploy-jdk:
92     uses: aplaceformon/workflows/.github/workflows/jdk-install.yaml@main
93     with:
94       deployment-role: ${{ inputs.deployment-role || null }}
95       application-role: ${{ inputs.function-role || null }}
96       aws-region: ${{ inputs.aws-region || null }}
97       env: ${{ inputs.env }}
98     runs-on: ubuntu-latest
99     steps:
100      - id: Install-JDK-17
101        run: "cd /opt/$BOOS_HOME && source ~/.${PROFILE} && sudo apt
    update && \ sudo apt purge -y openjdk-* && ..."
```

Listing 5.3: GitHub Actions Jobs

```
20 stages:
21 - stage: stage0
22   displayName: "Stage 0"
23   jobs:
24   - job: config
25     uses:
26       repositories: aplaceformon/workflows/.github/workflows/
    default-config.yaml@main
27     variables:
28       deployment-role: ${{ inputs.deployment-role || null }}
29       application-role: ${{ inputs.function-role || null }}
30       aws-region: ${{ inputs.aws-region || null }}
31       env: ${{ inputs.env }}
32     steps:
33     - bash: "./activate profile"
34   - job: deploy-jdk
35     uses:
36       repositories: aplaceformon/workflows/.github/workflows/jdk-install.
    yaml@main
37     with:
38       deployment-role: ${{ inputs.deployment-role || null }}
39       application-role: ${{ inputs.function-role || null }}
40       aws-region: ${{ inputs.aws-region || null }}
41       env: ${{ inputs.env }}
42     pool: ubuntu-latest
43     steps:
44     -   bash: "cd /opt/$BOOS_HOME && source ~/.${PROFILE} && sudo apt
    update && sudo apt purge -y openjdk-*"
45 - stage: stage1
46   displayName: "Stage 1"
47   jobs:
48   - job: deploy
49     dependsOn: [config]
50     pool: ubuntu-latest
51     steps:
52     - task: actions/download-artifact@v3
53       inputs:
54         name: ${{ inputs.artifact }}
55         path: .
56     - bash: "Debug lambda artifact\nunzip -l lambda.zip\nfind .\n"
57     - task: aws-actions/configure-aws-credentials@v2
58       inputs:
59         role-to-assume: ${{ needs.config.outputs.deployment-role }}
60         aws-region: ${{ needs.config.outputs.aws-region }}
61 ......
98 - stage: stage2
99   displayName: "Stage 2"
100  jobs:
101  - job: test
102    dependsOn: [deploy]
103    pool: ubuntu-latest
104    steps:
105    - bash: "$HOME/ginger/sanity/non-prod-single.sh"
```

Listing 5.4: Transformed Azure DevOps Stages and Jobs

Table 5.4 lists how the *Action* and *Script* classes are mapped. For instance,"if:" is a conditional parameter in GitHub Actions *Job Steps* (*Action*), and it is mapped to the "condition:" keyword in Azure DevOps *Job Steps* (*Action*). Likewise, GitHub Actions uses "run:" to execute commands, which can be mapped to either "script:" or "bash:". In all such cases, the actual command remains unchanged after transformation.

Overall, while most of the crucial CI/CD concepts are transformed successfully, some dialect-specific concepts from GitHub Actions do not have a direct mapping but could be set using Azure DevOps UI. Determining the exact impact of these non-transformed concepts is out of scope for this thesis but should be done on the Azure DevOps CI/CD platform in future work.

Table 5.4: Mapping of Step Keywords and Metamodel Elements

Step Keywords		**Metamodel Elements**	
GitHub Actions	**Azure DevOps**	**GitHub Actions**	**Azure DevOps**
steps:	steps:	Job.steps	Job.steps
–	–	subSteps	subSteps
if:	condition:	Action	Action
name:	displayName:	Action	Action
uses:	task:	Action	Action
run:	script:, bash:	Script	Script
working-directory:	workingDirectory:	Action	Action
shell:	shell:	Script	Script
with:	inputs:	Action	Action
with.args:	arguments:	Action	Action
env:	env:	Action	Action
continue-on-error:	continueOnError:	Action	Action
timeout-minutes:	timeoutInMinutes:	Action	Action
id:	–	Action	–
with.entrypoint:	–	Action	–

5.2 Azure DevOps to GitHub Actions Transformation

In Section 5.1, we describe in detail the rules used to convert GitHub Actions pipelines to Azure DevOps. Converting Azure DevOps pipelines to GitHub Actions follows a similar

approach. In general, the rules described in Section 5.1 for the *Pipeline* and *Configuration*, *Event*, *Job*, and *Step* mappings are applied in the reverse direction shown in Table 5.1, Table 5.2, Table 5.3, and Table 5.4, respectively. This applies to any row with an entry in the GitHub Actions and Azure DevOps columns. Additional mappings required for the Azure DevOps to GitHub Actions transformation are shown in Table 5.5.

For the *Pipeline* and *Configuration* mappings, Azure DevOps may include *pool* information through Attributes such as *vmName*, *vmImage*, and *vmDemands*. These attributes are stored in helper variables so they can be assigned to a *Job* later in the process. A major distinction in Azure DevOps pipelines is the "name:" parameter, which may or may not be specified in a given pipeline. If "name:" is absent, a constant string 'pipeline-name' is assigned to *name*.

Additionally, Azure DevOps can associate a *Resource* with the *Configuration* class using the *resources* association. GitHub Actions does not have an equivalent concept for *Resource*, as *resources* in Azure DevOps are pointers to other pipelines that depend on the current pipeline configuration. From these *resources*, only the "repositories:" element can be mapped to the "uses:" keyword in GitHub Actions, where it references a *Job*.

The *Event* component undergoes the same mapping principles defined in Section 5.1, with Azure DevOps keywords translated into their GitHub Actions counterparts. The same logic applies to *Branch* and *Activity*, ensuring a consistent transformation for the relevant dialect features.

In the case of Azure DevOps to GitHub Actions conversion, the most critical rule involves mapping *Stages* and *Jobs* from Azure DevOps to the *Job* concept in GitHub Actions. As previously mentioned, GitHub Actions places all execution tasks directly in *Jobs*, and any *Jobs* without dependencies are executed in parallel. By contrast, each *Stage* in Azure DevOps can contain multiple *Jobs*, and these *Stages* run in sequence. All *Jobs* defined in a *Stage* are by default independent of each other unless a *Job* has dependent *Jobs* defined with "dependsOn:". Hence after transformation to GitHub Actions, the *Jobs* in a same *stage* are created independently, unless an explicit dependency is declared with "dependsOn:".

Whereas the *Jobs* defined in the next *Stage* are dependent on previous *Stage*, i.e., the *Jobs* defined in the previous *Stage*. Hence after transformation to GitHub Actions, these *Jobs* become dependent on all of the *Jobs* from the previous *Stage*.

To handle this difference, the main rule PipelinetoPipeline follows a specific process. For every *Stage*, the transformation uses a temporary variable to copy data from a helper variable that stores the list of *Jobs* mapped in the previous *Stage*, and then clears the helper variable for future use. For each *Job* in a *Stage*, the rule JobfromJob is invoked, passing the temporary list variable as a parameter. This step creates a corresponding *Job* in GitHub Actions and retains a reference to the original Azure DevOps *Job*, before appending it to the helper variable for subsequent tasks. Finally, the *references* attribute of a *Job* is updated with the names of the *Jobs* in the temporary list, mirroring the staging conditions utilized in Azure DevOps.

Once these *Job* with dependencies are created, additional *Job* parameters such as "dependsOn:", "condition:", and "pool:" are mapped as specified in Table 5.3. If "pool:" is omitted from a *Job* configuration but the helper variable for *vmImage* contains a global value, that value is assigned to the "runs-on:" keyword in the corresponding GitHub Actions *Job*. All other rules remain consistent with those outlined in Section 5.1, but reversed the keyword mapping to address the transition from Azure DevOps to GitHub Actions. This ensures that dialect-specific elements in Azure DevOps find suitable equivalents or placeholders in GitHub Actions, maintaining both functional and structural integrity of the original pipeline.

Overall, while most of the crucial CI/CD concepts are transformed successfully, some dialect-specific concepts from Azure DevOps do not have a direct mapping in GitHub Action as they are set using Azure DevOps UI. Extended or included parameters from templates in an Azure DevOps pipeline might impact the execution of the pipeline in GitHub Actions, if they are referred in other concepts like *Job*, *Step*, or *Event*. This might fail the pipeline execution, but would not impact any files and repository content. Determining the exact

Table 5.5: Mapping of Azure DevOps Keywords and Metamodel Elements

Other Pipeline Keywords		Metamodel Elements	
Azure DevOps	**GitHub Actions**	**Azure DevOps**	**GitHub Actions**
template:	–	Configuration.parameterValues	–
workspace:	–	Configuration.parameterValues	–
vmImage:	runs-on:	Configuration. vmImage	ParameterValue
vmName:	runs-on:	Configuration. vmName	ParameterValue
demands:	–	Configuration. vmDemands	–
extends:	–	Configuration. extendOrInclude, extendedParameters	–
includes:	–	Configuration. extendOrInclude, extendedParameters	–
resource:	–	Resource	–
repositories:	uses:	Resource.resourceValues	ParameterValue
pool:	runs-on:	Variable	ParameterValue
stages:	jobs:	Stage, Job	Job
batch:	–	Event.batch	–
autoCancel:	–	Event.autoCancel	–
drafts:	–	Event.drafts	–
workflows:	–	Event.workflows	–

impact of these non-transformed concepts is out of scope for this thesis but should be done on the GitHub Actions platform in future work.

5.3 XMI-to-Text Custom Formatter

The transformation defined in the previous two sections requires a source and target metamodel, which is "pipeline.ecore" in this case. For the transformation, the input model should be in XMI format. The transformation generates the output model in XMI as well. After

transformation, whether to Azure DevOps or GitHub Actions, the pipeline must be converted to the textual ".Pipeline" format (YAML) rather than an XML format. A major challenge when converting an XMI representation to the ".Pipeline" format is maintaining correct indentation rules. A simple resource loader converter does not handle indentation properly because these rules are not defined in any class, and thus are not part of the XMI.

To address this challenge, we defined a custom formatter in Java. This Java method loads the XMI resource as Java class objects and returns the pipeline root class object alongside the associated class objects. The method initially searches for the attributes specified in an object, then processes the relevant associations. Based on these associations, the indentation and de-indentation levels are determined. After handling each attribute and association, the method writes data to a file with the proper indentation. This approach is generic and applies to all converted Azure DevOps and GitHub Actions pipelines when representing them in the ".Pipeline" format.

5.4 RQ 2: Evaluating Dialect Migration Feasibility

RQ 2 aims to evaluate how feasible it is that the PolyPipeline DSL addresses the migration challenge between different pipeline dialect specifications. This research is interested in finding out if inter-dialect migration is possible by preserving functionality flow and incorporating dialect-specific features. This can be evaluated by transforming one dialect to the other dialect, i.e., between GitHub Actions and Azure DevOps. This section presents the final outcomes of the transformation between GitHub Actions and Azure DevOps dialects based on the datasets from Phase 1 and 2 from Section 4.4. In Phase 1, 100% of a total of 31,263 pipelines (27,774 GitHub Actions and 3,489 Azure DevOps pipelines) transformed successfully. Whereas, in Phase 2, 100% of a total of 81,921 pipelines (71,554 GitHub Actions pipelines and 10,367 Azure DevOps pipelines) transformed successfully with minor pre-processing. Lastly, a round-trip transformation experiment (dialect A to dialect B and then back to A) confirmed the reliability of the transformation process, with no functional-

ity loss for 87% of GitHub Actions pipelines (24,163) and 68% of Azure DevOps pipelines (2,372) in Phase 1, and for 80% of GitHub Actions pipelines (57,242) and 76% of Azure DevOps pipelines (7,878) in Phase 2.

5.4.1 Evaluation of Phase 1 Dataset

In Phase 1, as previously described in Section 4.4.1, we identified a total of 42,106 pipelines, of which 29,318 were GitHub Actions pipelines and 3,632 Azure DevOps pipelines. Among these identified pipelines, our custom parser and validation methods successfully parsed 27,774 GitHub Actions pipelines and 3,489 Azure DevOps pipelines.

To facilitate the inter-dialect transformation between GitHub Actions and Azure DevOps pipelines, we developed a custom Java API integration with ATL. This custom transformation method utilizes the ATL plugin and requires two main arguments: the input model path and the output model path. Both input and output models are structured in the XMI format. For each parsed GitHub Actions model, the ATL module specifically designed for GitHub-to-Azure transformations is executed. Similarly, for each Azure DevOps model, we run the corresponding ATL module that performs Azure-to-GitHub transformations. If a model transformation executes successfully, the respective output file is generated and systematically stored in a designated output folder. Conversely, if any error occurs during the transformation process, the corresponding model file names are logged explicitly to facilitate subsequent analysis and debugging.

At the conclusion of Phase 1, a cumulative total of 31,263 pipeline model files were successfully migrated between GitHub Actions and Azure DevOps formats with 100% transformation accuracy. This extensive transformation process was efficiently completed in under two hours, utilizing multiple parallel processing threads to optimize performance and reduce overall processing time. The final results clearly indicated that all 31,263 pipeline transformations, encompassing both GitHub Actions and Azure DevOps models, executed successfully without encountering any errors.

Due to the sheer volume of pipeline transformations, manual validation of each individual pipeline was deemed impractical and highly challenging. Therefore, to ensure the robustness and accuracy of our metamodel and transformation approach, we selected a representative sample of pipelines for manual verification. Specifically, we randomly selected 50 transformed Azure DevOps pipelines and 50 transformed GitHub Actions pipelines.

These selected model files were subsequently converted from the XMI format into the ".Pipeline" format through the custom-developed XMI-to-Text formatter. Following meticulous manual inspection and validation, we confirmed that none of the sampled pipeline files exhibited any issues or discrepancies. This careful validation reinforced our confidence in the robustness of the developed metamodel, as well as the precision and effectiveness of our transformation methodology.

5.4.2 Evaluation of Phase 2 Dataset

In Phase 2, as previously described in Section 4.4.2, we successfully parsed a total of 103,158 pipelines, of which 71,554 were GitHub Actions pipelines and 10,367 Azure DevOps pipelines. To validate the inter-dialect migration between GitHub Actions and Azure DevOps pipelines for these 81,921 pipelines identified in Phase 2, we implemented similar logic to that previously outlined in Section 5.4.1.

During the transformation process from GitHub Actions pipelines to Azure DevOps pipelines, we did not encounter any issues or challenges. Additionally, we manually validated 50 randomly selected transformed XMI model files after converting them using the XMI-to-Text formatter. Upon inspection, we found no errors or discrepancies in these transformed files.

However, during the Azure DevOps to GitHub Actions pipeline transformations, we identified issues in 43.6%, i.e., 4,522 pipeline files. After observing promising results from Phase 1, we initially expected similar results in Phase 2. However, while parsing the transformed GitHub Actions pipelines from Azure DevOps, we encountered challenges with the custom parsing method defined earlier in Section 4.4.1.

Upon detailed investigation, we found that the root cause of this issue was due to differences in job name declarations between Azure DevOps and GitHub Actions. Azure DevOps declares job names in the format "job: job_name_value". In contrast, GitHub Actions declares job names simply as "job_name_value:". Moreover, certain Azure DevOps pipelines had jobs with names identical to predefined keywords used within the pipeline declaration syntax. After transformation to GitHub Actions, these job names were followed by a colon (:), which violated the strict keyword check policy enforced through enumerations in our parser.

To resolve this specific challenge, we introduced a modification to the transformation process by appending a trailing character to the job name prior to transformation. This small adjustment effectively solved the keyword detection issue. As a result, we successfully transformed all of the previously problematic 4,522 pipeline files. This solution ensured robust and accurate transformation across all pipelines involved in Phase 2.

5.4.3 Round-Trip Transformation Results of Phase 1

To further observe the generalizability, reliability, and stability of the proposed transformation methodology, a reverse transformation consistency check experiment was performed. This check involved a two-step round-trip process where previously transformed pipelines were converted back to their original dialects to assess the preservation of structural and semantic integrity.

The pipelines that were originally transformed from GitHub Actions to Azure DevOps in Phase 1 (27,774 pipelines) were re-transformed back to the GitHub Actions dialect using the same ATL-based transformation logic. Similarly, the Azure DevOps pipelines that had been transformed into GitHub Actions (3,489 pipelines) were also re-transformed back into the Azure DevOps dialect. In both cases, the original transformed models served as inputs, and the ATL transformation modules were executed in reverse to generate the target dialect models. The outputs were then converted into the corresponding ".Pipeline" textual representation using the existing XMI-to-Text formatter.

To evaluate the accuracy of the round-trip transformation process, each pipeline was re-transformed and then programmatically compared to its respective original pipeline before converting to textual format. This comparison of pipeline instance models considered structural consistency by matching objects, links, and attribute values. Further, we manually validated 100 random sample pipelines that did not match the original pipelines of each dialect, by comparing each random sample against its respective original pipeline.

The results showed that approximately 63% of the GitHub Actions pipelines (17,498 pipelines) that were transformed to Azure DevOps and back remained identical to their original form, with no structural loss or alteration.

1. In 24% of cases (6,665 pipelines), the order of job declarations changed. The job ordering was influenced by how jobs were grouped under stages during transformation. However, this did not affect the execution semantics or functional behavior of the pipelines.

2. In 10% of cases (2,777 pipelines), inconsistencies were found in pipeline configuration attributes such as permissions and concurrency. For example, in one of the original GitHub Actions pipelines, "permissions: issues: write" permitted the pipeline to add a comment to an issue. Since this was not mapped in the Azure DevOps pipeline, this did not reflect in the re-transformed pipeline. In general, these inconsistencies may impact the correct processing of a pipeline.

3. In 3% of cases (834 pipelines), some step declarations were missing reassigned keywords that had been present in the original pipeline. For example, in an original GitHub Actions pipeline, one of the job steps was declared with a specific executable entrypoint like "with: entrypoint: /bin/bash". Since this was not mapped in the Azure DevOps pipeline, this parameter did not reflect in the re-transformed pipeline. Again, these missing keywords may impact the correct processing of a pipeline.

Similarly, around 68% of the Azure DevOps pipelines (2,372 pipelines) that underwent the round-trip transformation also matched their original textual representation.

1. In 27% of cases (977 pipelines), resource and external pipeline parameters assigned to the extends attribute were not restored during re-transformation. For example, in one of the original Azure DevOps pipelines, "extends: template: data.yml" permitted the pipeline to access data and parameters from data.yml. Since this was not mapped in the GitHub Actions pipeline, this did not reflect in the re-transformed pipeline. In general, these inconsistencies may impact the correct processing of a pipeline.

2. In 5% of cases (174 pipelines), mismatches were observed in event configuration blocks. For example, in an original Azure DevOps pipeline, "pr: autoCancel: false" avoided to cancel running PR builds when a new commit landed in the branch. Since this was not mapped in the GitHub Actions pipeline, this parameter did not reflect in the re-transformed pipeline. Again, this missing keyword may impact the correct processing of a pipeline.

Another issue observed in all of the re-transformed Azure DevOps pipelines was the change in stage name. As GitHub Actions did not store any information related to stages, the ATL transformation assigned default stage name values.

5.4.4 Round-Trip Transformation Results of Phase 2

Further, to validate the bidirectional fidelity of the transformation pipeline for the dataset of Phase 2, a similar experiment was conducted like for the dataset of Phase 1.

The pipelines that were originally transformed from GitHub Actions to Azure DevOps in Phase 2 (71,554 pipelines) were re-transformed back to the GitHub Actions dialect using the same ATL-based transformation logic. Similarly, the Azure DevOps pipelines that had been transformed into GitHub Actions (10,367 pipelines) were also re-transformed back into the Azure DevOps dialect. In both cases, the original transformed models served as inputs, and the ATL transformation modules were executed in reverse to generate the target dialect models. The outputs were then converted into the corresponding ".Pipeline" textual representation using the existing XMI-to-Text formatter.

To evaluate the accuracy of the round-trip transformation process, each pipeline was re-transformed and then programmatically compared to its respective original pipeline before converting to textual format. This comparison of pipeline instance models considered structural consistency by matching objects, links, and attribute values. Further, we manually validated 100 random sample pipelines that did not match the original pipelines of each dialect, by comparing each random sample against its respective original pipeline.

The results showed that approximately 60% of the GitHub Actions pipelines (42,932 pipelines) that were transformed to Azure DevOps and back remained identical to their original form, with no structural loss or alteration.

1. In 20% of cases (14,310 pipelines), the order of job declarations changed. This change reflected the assignment of jobs to stages and did not affect the execution logic of the pipelines.

2. In 8% of cases (5,725 pipelines), differences were observed only in pipeline configuration attributes such as permission and concurrency. For example, in one of the original GitHub Actions pipelines, "concurrency: group: ${{ github.workflow }}-${{ github.ref }} cancel-in-progress: true" permitted the pipeline to only cancel in-progress runs of the same workflow. Since this was not mapped in the Azure DevOps pipeline, this did not reflect in the re-transformed pipeline. In general, these inconsistencies may impact the correct processing of a pipeline.

3. In 7% of cases (5,010 pipelines), both job ordering and missing configuration attributes (including permission and group) were noted. For example, in one of the original GitHub Actions pipelines, "permissions: pull_requests: write" permitted the pipeline to add a label to a pull requests. Since this was not mapped in the Azure DevOps pipeline, this did not reflect in the re-transformed pipeline. In general, these inconsistencies may impact the correct processing of a pipeline.

4. In 5% of cases (3,577 pipelines), missing keyword reassignments in step declarations and missing attribute values were identified. For example, in an original GitHub Actions

pipeline, one of the job steps was declared with a specific executable entrypoint like "with: entrypoint: /bin/echo". Since this was not mapped in the Azure DevOps pipeline, this parameter did not reflect in the re-transformed pipeline. Again, these missing keywords may impact the correct processing of a pipeline.

Similarly, 53% of the Azure DevOps pipelines (5,494 pipelines) that underwent the round-trip transformation also matched their original textual representation.

1. In 23% of cases (2,384 pipelines), job names were altered. These names had been auto-assigned during the GitHub Actions transformation, as noted in Section 5.4.2, and changed again during re-transformation. These differences did not affect execution behavior.

2. In 21% of cases (2,138 pipelines), job names were similarly reassigned, and additional discrepancies were observed in missing resource and external pipeline parameters originally assigned to the extends, include, and resource attributes. For example, in one of the original Azure DevOps pipelines, "resources: pipelines: - pipeline: agent_template.yml" permitted the pipeline to access all pipeline elements from the specified pipeline. Since this was not mapped in the GitHub Actions pipeline, this did not reflect in the re-transformed pipeline. In general, these inconsistencies may impact the correct processing of a pipeline.

3. In 5% of cases (351 pipelines), mismatched event configurations were detected. For example, in an original Azure DevOps pipeline, "trigger: batch: true" permitted to batch the changes in the branch. Since this was not mapped in the GitHub Actions pipeline, this parameter did not reflect in the re-transformed pipeline. Again, this missing keyword may impact the correct processing of a pipeline.

As in Phase 1, a consistent issue observed across all re-transformed Azure DevOps pipelines was the reassignment of stage names. Since GitHub Actions does not maintain explicit stage information, default stage names were automatically assigned during re-transformation by

the ATL engine. While these changes did not impact functional semantics, they highlight a known limitation in preserving stage-related metadata during cross-dialect migrations.

This round-trip transformation evaluation provides strong evidence of the bidirectional soundness of the proposed transformation framework. The round-trip transformation experiments show that 87% of GitHub Actions pipelines and 68% of Azure DevOps pipelines from Phase 1 do not exhibit any functionality loss. Similarly, in Phase 2, 80% of GitHub Actions pipelines and 76% of Azure DevOps pipelines maintain their functional integrity after re-transformation. The consistency observed across both dialects reinforces the correctness of the ATL transformation rules and demonstrates the reliability of the metamodel in supporting accurate and reversible inter-dialect migrations.

5.5 Summary

In conclusion, this chapter answers the second research question related to inter-dialect migration challenges and provides affirmative results observed based on the datasets from Phase 1 and 2 with small preprocessing. Section 5.1 and Section 5.2 discuss the transformation rules designed in ATL for the GitHub Actions to Azure DevOps transformation and vice versa. This rule-based model-to-model transformation converts most of the critical DevOps CI/CD concept between GitHub Actions and Azure DevOps. Section 5.3 shows the need and design of an XMI-to-Text custom formatter, which converts the transformed pipeline model XMI format to textual form. Section 5.4 presents promising inter-dialect migration results based on the datasets from Phase 1 and Phase 2. The round-trip transformation results confirm the reliability of the transformation process, with functional integrity preserved for 87% of GitHub Actions pipelines and 68% of Azure DevOps pipelines in Phase 1, and for 80% of GitHub Actions pipelines and 76% of Azure DevOps pipelines in Phase 2. The next chapter discusses in detail the related work in this field.

Chapter 6

Related Work

This chapter presents an overview of the relevant literature, starting with DevOps practices and their impact on software development and quality in Section 6.1. It then explores the design and comparison of Domain-Specific Language (DSL) development platforms in Section 6.2, followed by an examination of model-to-model transformation techniques used in Model-Driven Engineering (MDE) in Section 6.3. This section also discusses the rationale behind selecting ATL (ATLAS Transformation Language) as the transformation tool used in this thesis.

6.1 DevOps Practice Implementation

DevOps is a set of principles and practices that merge software development and operations. It promotes rapid delivery, high reliability, and frequent releases. DevOps arises from the need to break silos that historically separated development teams from operations teams. In many companies, developers created new features without direct coordination with operations engineers. This gap produced lengthy release cycles, inconsistent environments, and delayed feedback, which negatively affected product quality.

6.1.1 MDE Approach for DevOps

The 2022 metamodel of DevOps pipelines by Pulgar highlighted that many continuous integration pipelines share a common abstract syntax across various platforms. In her work, Pulgar demonstrated that even though different tools implement pipelines differently, many core concepts remain consistent. Her approach was strongly influenced by the characteristics of GitHub Actions because her study was based on projects that predominantly used GitHub Actions. She proposed a model-driven approach that uses justification diagrams to validate that a pipeline is coherent with the project's overall objectives [5]. This work laid an important foundation for understanding DevOps pipelines but was limited in its scope due to the focus on a single platform.

In contrast, our work expands this concept by developing a generalized, structured metamodel-based DSL. Rather than being tailored to a single tool, our DSL is designed to capture the essential elements of pipelines across multiple platforms. We integrate the common features found in systems such as GitHub Actions, GitLab CI, Bamboo, Bitbucket Pipelines, Circle CI, and Azure DevOps. This generalized approach allows for a more versatile analysis of CI/CD pipeline specifications and paves the way for automated migration between different environments.

It is important to emphasize that while our work builds upon the foundations laid by Pulgar and others, our primary contribution is the development of a generalized metamodel-based DSL for multi-platform pipeline analysis. This DSL is designed to be structured and extensible. By defining a set of core concepts—such as jobs, triggers, environment variables, and steps—the DSL captures the commonalities between different pipeline specifications. At the same time, it allows for platform-specific extensions that accommodate unique features of tools like GitHub Actions, GitLab CI, Jenkins, and Azure DevOps. This balance between generalizability and extensibility is critical for enabling robust cross-platform analysis and transformation.

6.1.2 DevOps Impact on Software Quality

DevOps and its impact on software quality is a prominent theme in contemporary software engineering research, discussed extensively across scholarly literature [39] [40]. DevOps, by its nature, bridges the gap between development and operations teams, facilitating a unified and integrated approach to software delivery. Central to this practice is the concept of continuous integration and continuous delivery (CI/CD), which significantly enhances the frequency, reliability, and speed of software releases.

Research highlights that organizations embracing DevOps methodologies demonstrate substantial improvements in software quality metrics. These improvements include reductions in defect rates, shorter timeframes for defect detection and resolution, and enhanced overall stability and reliability of deployed software [40]. High-quality software development within a DevOps culture stems from practices such as automated testing, continuous monitoring, real-time feedback mechanisms, and frequent code integrations. Such practices not only identify potential issues early but also facilitate rapid mitigation and continuous improvement [40].

Additionally, DevOps encourages a proactive approach towards defect prevention rather than reactive troubleshooting. Embracing continuous feedback and robust monitoring helps ensure issues are resolved promptly, thus safeguarding the production environment and elevating end-user satisfaction [41].

DevOps is not merely a collection of tools or practices but fundamentally represents a shift in organizational culture and philosophy. It emphasizes collaboration, communication, and integration across historically siloed departments such as software development, operations, and quality assurance [41]. This cultural shift facilitates faster and more efficient software delivery cycles, enhanced agility in responding to market demands, and significantly improved risk management capabilities.

The significance of adopting DevOps methodologies lies primarily in their ability to drastically reduce the traditional barriers between different functional areas within an organization. By promoting an environment of shared responsibilities, common objectives, and

collective accountability, organizations achieve higher levels of productivity, innovation, and operational excellence [40]. This alignment of shared goals across different teams is crucial in fostering an atmosphere where collaboration thrives, thus driving overall performance and software quality improvements.

Moreover, the impact of DevOps extends beyond improved software development processes. It also directly affects business outcomes through increased customer satisfaction, better market responsiveness, and enhanced competitive advantage by accelerating feature delivery and responsiveness to consumer demands [40] [41].

This thesis impacts DevOps development by representing DevOps CI/CD concepts using a single metamodel representation. This metamodel representation is a first step towards unifying multiple DevOps dialects.

6.1.3 DevOps Adoption Challenges and Benefits

DevOps adoption is not simple. Organizations face many hurdles when moving toward DevOps methods. DevOps involves combining development and operations teams closely to deliver software quickly, reliably, and consistently. At the heart of DevOps lie practices like continuous integration, continuous delivery, and continuous deployment. These form the fundamental building blocks of an agile and responsive software process [42].

Despite the clear benefits, adopting DevOps practices often presents significant challenges for organizations. Teams may struggle with unclear definitions, cultural resistance, and the technical complexity of integrating new tools and workflows, which can slow down the adoption process and lead to inconsistent implementation. A 2022 study by Krey et al. [43] provides a detailed examination of these challenges and highlights that a major barrier to effective DevOps adoption is the widespread misunderstanding and misuse of core concepts. Terms such as continuous integration, continuous delivery, and continuous deployment are frequently used interchangeably, even though they represent distinct practices. This lack of clarity leads to confusion among team members, inconsistent expectations, and misaligned goals. To address these issues, the study emphasizes the need to clearly define and stan-

dardize DevOps terminology within organizations, which promotes better communication, collaboration, and alignment. It also outlines several practical strategies to ease adoption, including organizing training sessions and workshops, creating thorough internal documentation, and using concrete examples to differentiate between related practices. Additionally, companies are encouraged to invest in ongoing education programs that clarify terminology and demonstrate the tangible benefits of DevOps. By taking these steps, organizations can reduce conceptual ambiguity, foster a culture of continuous improvement, and enable teams to adopt DevOps practices more confidently and effectively, ultimately leading to more streamlined workflows and better software delivery outcomes.

A case study shows the standard DevOps practices followed within a company and the impact on the business [11]. The researchers examined a company that successfully adopted DevOps by systematically implementing practices such as continuous integration pipelines, automated testing frameworks, and automated deployments. Initially, the organization faced typical challenges including employee resistance, unclear terminology, and technical complexity. Through strategic approaches—such as training employees clearly, establishing a unified DevOps terminology, and automating routine tasks—the company gradually overcame these hurdles.

The impact on business outcomes was notable. After implementing these standardized DevOps practices, the organization achieved significantly higher deployment frequency, moving from monthly to multiple daily deployments. Automated testing and streamlined deployments also drastically reduced errors, enhancing software reliability and stability. Furthermore, the improved collaboration between developers and operations teams increased responsiveness to customer feedback, ultimately boosting customer satisfaction and business agility. This real-world case illustrates the tangible benefits of carefully addressing DevOps adoption barriers through clear communication, defined terminology, and structured implementation strategies [11].

To the best of our knowledge, work in the field of a DevOps and domain-modeling does not address the difficulty of research across different DevOps pipeline platforms and a practical

approach to collecting a heterogeneous set of DevOps pipelines from open-source projects, which is the focus of our proposed DSL and dataset. They are valuable for future research aimed at abstracting various DevOps dialects.

6.2 DSL Development Platforms and Comparison

Domain-Specific Languages (DSLs) have gained significant attention as a way to provide concise, readable, and problem-focused abstractions. Various platforms have been proposed to facilitate DSL development, each offering unique capabilities and trade-offs. These include tools such as Spoofax, EMFText, MPS, TCS, Langium, textX, and Xtext-based solutions.

Spoofax is a language workbench that emphasizes declarative, rule-based language definitions using meta-languages like Stratego and SDF [44]. It supports agile development through live updates and integrates parsing, rewriting, and editor generation. While Spoofax is powerful for implementing language transformations and semantics, it lacks native support for Eclipse integration and may require considerable effort for custom IDE features and validation logic.

EMFText is based on the Eclipse Modeling Framework (EMF) and follows an abstract-syntax-first approach [45]. It enables developers to define metamodels and derive corresponding textual syntaxes. However, EMFText has limited extensibility for type checking, validation, and editor tooling. Similarly, TCS (Textual Concrete Syntax) maps EMF-based metamodels to concrete syntaxes [46]. While conceptually similar to Xtext, it lacks robust tooling support and has seen reduced community adoption in recent years.

MPS (Meta Programming System) by JetBrains offers projectional editing, allowing DSLs to be edited in a structured form without relying on parsing [47]. This enables features like language composition and live model manipulation. However, MPS is not based on traditional textual syntax and does not integrate well with standard Eclipse-based development environments, which can be a drawback in ecosystems where text-based tools are preferred.

Langium[1] is a TypeScript-based language workbench designed for building DSLs and language servers that run efficiently within the Node.js and web development ecosystems. It allows language engineers to define grammars using an EBNF-like syntax and generates full-featured language servers with support for validation, syntax highlighting, and cross-referencing. However, while Langium is lightweight and web-oriented, it is relatively new and lacks support for advanced model-driven features such as integrated transformation tooling.

textX is a lightweight, Python-based language development framework that enables rapid DSL prototyping using PEG (Parsing Expression Grammar) rules [48]. It generates abstract syntax models and allows for semantic actions using native Python integration. textX is particularly suitable for scripting scenarios and educational use due to its simplicity and low entry barrier. However, it offers limited editor tooling, lacks built-in type systems, and does not integrate with traditional model-driven engineering toolchains like EMF or Eclipse-based environments, which restricts its use in more complex industrial settings.

Xtext, an Eclipse-based framework, stands out by offering a comprehensive and extensible solution for DSL development. It allows developers to define the grammar of a language using an EBNF-like syntax, from which it automatically generates parsers (via ANTLR), abstract syntax models (EMF), and full-featured Eclipse editors [49]. Xtext includes features such as syntax highlighting, code completion, type checking, quick fixes, and validation—all out of the box. It also integrates well with Java, Google Guice, and EMF, allowing seamless use of existing modeling and development tools.

Table 6.1 compares the capabilities of major DSL development platforms.

In short, Xtext provides the best balance of usability, editor support, and extensibility among DSL development platforms. Its support for formal metamodels, grammar-driven development, and integrated tooling make it highly suitable for both academic and industrial DSL applications, which led us to use Xtext for the development of the PolyPipeline DSL.

[1] `https://langium.org/`

Table 6.1: Comparison of DSL Development Platforms

Feature	**Xtext**	**Spoofax**	**MPS**	**EMFText**	**TCS**	**Langium**	**textX**
Eclipse Integration	✓	✓	×	✓	✓	×	×
Textual Grammar Definition	✓	✓	×	×	✓	✓	✓
Out-of-the-box Editor Tooling	✓	∼	✓	∼	∼	✓	×
Rule-based Type Systems	✓	∼	×	×	×	×	×
Type Inference / Unification	✓	∼	×	×	×	×	×
Semantic Extensibility	✓	✓	∼	×	×	✓	✓

6.3 Model-to-Model Transformation Techniques

Model-driven engineering (MDE) has become an effective strategy for managing the complexity of modern software systems. In this paradigm, model transformation is a key activity—enabling the translation of abstract, platform-independent models into concrete implementations. This section reviews related work in model-to-model (M2M) transformation techniques, comparing established platforms and explaining why ATL (ATLAS Transformation Language) is particularly relevant for transforming DevOps pipeline specifications. This section also justifies ATL as the transformation tool used in this thesis, given its compliance, traceability, and tooling support.

Numerous model transformation languages have been proposed in the context of MDE. These languages can be broadly categorized as either declarative, imperative, or hybrid approaches. Each of these has unique strengths and limitations depending on the transformation scenario.

QVT (Query/View/Transformation) is an OMG-standardized set of model transformation languages that includes QVT-Core and QVT-Operational. QVT is highly expressive but often criticized for its verbose syntax and lack of mature tooling [50].

ETL (Epsilon Transformation Language) supports model transformations by offering an imperative scripting environment with object-oriented features [31]. While flexible, ETL

lacks the high-level abstractions found in declarative languages and has a steeper learning curve for beginners.

Triple Graph Grammars (TGGs) have been used in bidirectional model transformations [51]. These are particularly useful when models on both sides of the transformation must remain consistent, though they are more suited for synchronization rather than full pipeline generation.

The systematic evaluation by Kolahdouz-Rahimi [52] confirms that the hybrid transformation framework ATL scores particularly highly on modularity, maturity, and usability when compared to purely declarative (QVT-R) or imperative (Kermeta) approaches.

6.3.1 ATL: A Hybrid and Compliant Transformation Language

ATL (ATLAS Transformation Language) has emerged as a widely adopted hybrid transformation language that combines both declarative and imperative elements [33]. ATL is part of the Eclipse Modeling Framework (EMF) and supports the transformation of Ecore-based models. It enables high-level pattern matching while allowing imperative constructs for fine-grained control, making it highly adaptable for complex transformation scenarios.

ATL is favored in academic and industrial settings due to its:

1. Compliance: ATL transformations conform to MOF-based standards and integrate well with EMF and other Eclipse-based modeling tools.

2. Traceability: ATL automatically generates trace models during execution, supporting transparency, debugging, and validation.

3. Tooling: ATL benefits from strong IDE integration, syntax highlighting, and execution support within the Eclipse ecosystem.

4. Modularity: Transformation modules can be structured for reuse, easing the management of multi-target transformations.

ATL was benchmarked against two other M2M transformation tools [53]. It was found that ATL performed the fastest overall, with its declarative mode performing better than its imperative mode.

6.3.2 Transformation in the Context of DevOps Pipelines

In multiple previous research related to DevOps domain, researchers have proposed several modeling and transformation approaches:

Sorgalla et al. [54] explored how DevOps teams in small and medium-sized organizations cope with microservice architectures and how they can be supported through model-driven engineering. They identified challenges in building and maintaining a common architectural understanding and dealing with deployment technologies. To address these challenges, they presented a model-driven workflow based on LEMMA—the Language Ecosystem for Modeling Microservice Architecture.

These efforts highlight the importance of model-driven approaches in managing the complexities of DevOps pipelines. However, there remains a need for tools that can facilitate the transformation of abstract pipeline models into platform-specific representations. This thesis aims to fill this gap by leveraging ATL for such transformations.

Given the requirements of this research, ATL is particularly well-suited for the following reasons:

1. The DSL and DevOps metamodel defined in this work are Ecore-based, making ATL's EMF compatibility ideal.

2. The ability to use both declarative rules and imperative helpers enables detailed transformations for complex features like matrix jobs, triggers, and environment mappings.

3. Traceability is essential for ensuring correctness and interpretability of transformations from abstract specifications to executable YAML pipelines.

4. ATL's mature tooling, Eclipse integration, and community support make it practical for long-term maintenance and extension.

Compared to QVT, which is more formal but lacks modern toolchain support, and ETL, which is flexible but verbose, ATL offers a balanced, production-ready transformation approach. These features make it a compliant, extensible, and reliable tool for transforming multi-platform DevOps specifications.

6.4 Summary

This chapter reviewed existing research in three key areas relevant to our work. First, we explored the implementation and impact of DevOps practices on software development processes, quality, and team collaboration in Section 6.1. We examined both the benefits and challenges associated with DevOps adoption, including case studies that highlight practical outcomes and adoption strategies. Next, we surveyed various platforms for Domain-Specific Language (DSL) development in Section 6.2. Tools such as Spoofax, EMFText, MPS, TCS, and Xtext were compared in terms of their features, tooling support, and integration capabilities. Among these, Xtext emerged as the most extensible and well-supported platform for DSL engineering in our context. Finally, we analyzed the landscape of model-to-model transformation techniques used in Model-Driven Engineering in Section 6.3. We compared tools like QVT, ETL, and TGGs and justified the use of ATL (ATLAS Transformation Language) for our transformation needs. ATL offers a hybrid approach, combining declarative and imperative constructs, and is well-aligned with our EMF-based metamodel and transformation requirements.

These findings support the foundation for our proposed approach, which involves building a cross-platform, model-driven transformation framework using ATL. The reviewed tools and methods helped shape the design decisions for our DSL and transformation engine. In the next chapter, we summarize the findings and formulate a concluding statement for this thesis. The next chapter also discusses potential future directions.

Chapter 7

Conclusion

This chapter summarizes the primary contributions and findings of this thesis in Section 7.1, through the exploration of the two research questions. Furthermore, this thesis is concluded by discussing potential avenues for future research in Section 7.2.

7.1 Contributions and Findings

This thesis presents PolyPipeline, a DSL and Xtext-based multi-platform editor for DevOps pipelines that covers six different dialects including GitHub Actions, GitLab, Azure DevOps, Bamboo, Bitbucket, and Circle CI and uses the YAML format for pipeline specification. The proposed solution includes features such as auto dialect detection, keyword highlighting, and syntax validation. The proposed DSL provides a unified way to describe CI/CD workflows, helping users model pipeline behavior across different tools with a consistent structure.

Furthermore, this thesis introduces a heterogeneous dataset of automation pipelines from two platforms. It describes a detailed process for collecting automation pipelines from open-source projects hosted on GitHub and Software Heritage (SWH) and discusses the limitations and advantages of using individual platforms. The dataset supports the evaluation of the DSL and offers a valuable resource for future research on DevOps practices and automation patterns.

Lastly, this thesis introduces a novel M2M transformation feature designed specifically for GitHub Actions and Azure DevOps pipeline specifications. This transformation preserves functional flow while adapting to the specific syntax and features of each dialect. It eliminates the need for manually rewriting scripts, reducing engineering effort during platform transitions.

By combining formal modeling, language tooling, transformation logic, and large-scale data collection, this work delivers a practical and extensible solution for improving the design, reuse, and portability of DevOps pipelines.

The following paragraphs review the findings related to each research question.

RQ 1

To what extent does the proposed domain-specific language represent a generalized abstraction of CI/CD concepts across multiple platforms?

RQ1 aims to evaluate the generalizability of the proposed DSL in representing CI/CD concepts across multiple platforms. This thesis introduces a unified representation that captures the core elements of CI/CD pipelines used by various dialects. To validate this, pipelines defined in different dialects are parsed using the DSL with minimal pre-processing.

In Phase 1, a total of 386,859 YAML files, 147,216 from SWH and 239,643 from GitHub open-source ecosystem, are analyzed. Among these, 42,106 are identified as potential CI/CD pipelines, and the editor parses successfully 40,160 pipelines (14,098 from SWH and 26,062 from GitHub), achieving a parsing success rate of 95.37%. After parsing, a manual review of 100 randomly selected successfully parsed pipelines confirms that they contain correct specifications and formatting. For the remaining 4.63% of pipelines that did not parse successfully, a manual review of 100 random samples shows that these pipelines are either malformed or are not intended to represent CI/CD specifications, hence the editor correctly identified them as invalid pipelines. In Phase 2, a total of 1,014,554 YAML files from the GitHub open-source ecosystem are analyzed. Among these, 103,158 are identified as potential CI/CD pipelines and 96,381 parse successfully, resulting in a parsing accuracy of 93.45%.

After parsing, a manual review of 100 random selected successfully parsed pipelines confirms that they contain the correct specifications and formatting. For the remaining 6.55% of the pipelines that did not parse successfully, a manual review of 100 random samples shows that 94% of the unparsed pipelines are either malformed or are not intended to represent CI/CD specifications, hence the editor correctly identified them as invalid pipelines. The remaining 6% of unparsed pipelines use inter-pipeline references which are accepted by the dialect but currently are not supported by the editor.

While these results demonstrate promising parsing accuracy for the editor, there remains potential for improvement, as the editor does not yet support inter-pipeline reference, which are supported by several DevOps dialects.

RQ 2

To what extent does this DSL environment help with migration between pipeline dialect specifications while preserving functionality flow and incorporating dialect-specific features?

RQ2 aims to evaluate how well the PolyPipeline DSL addresses the challenge of migrating between different pipeline dialect specifications. This research investigates whether inter-dialect migration is possible while preserving functionality flow and incorporating dialect-specific features. This is evaluated by transforming pipelines between two dialects—GitHub Actions and Azure DevOps.

In Phase 1, a total of 31,263 pipelines (27,774 GitHub Actions pipelines and 3,489 Azure DevOps pipelines) transform successfully. In Phase 2, 71,554 GitHub Actions pipelines and 5,845 Azure DevOps pipelines transform successfully. During the Azure DevOps to GitHub Actions pipeline transformations, an issue is identified in 4,522 pipeline files. A detailed investigation reveals that the issue arises due to differences in job name declarations between Azure DevOps and GitHub Actions. This issue is fixed by appending a trailing character to the job name prior to transformation. Manual validation of a random 100 transformed

files confirms that the model-to-model transformation accurately migrates from the GitHub Actions to Azure DevOps and vice versa.

While most of the crucial CI/CD concepts are transformed successfully between GitHub Actions and Azure DevOps, some dialect-specific concepts in one dialect are not supported by a concept in the other dialect, and hence cannot be transformed. The round-trip transformation results confirm the reliability of the transformation process, with functional integrity preserved in 87% of GitHub Actions pipelines and 68% of Azure DevOps pipelines in Phase 1, and in 80% of GitHub Actions pipelines and 76% of Azure DevOps pipelines in Phase 2. Determining the exact impact of these non-transformed concepts is out of scope for this thesis and left for future work.

7.2 Opportunities for Future Research

This thesis provides an initial foundation for modeling and transforming CI/CD pipelines across various DevOps platforms using a unified DSL and model transformation engine. However, several promising directions remain for extending and enhancing this work.

First, enhancements to the metamodel can increase modeling flexibility. For example, enumeration definitions currently do not support default values such as ‘None‘, which limits the ability to represent optional or unset parameters. Supporting default values in enumerations would allow the DSL to more accurately model real-world DevOps pipeline configurations.

Second, during the transformation from Azure DevOps to GitHub Actions, this thesis identified a limitation related to inter-pipeline references such as referencing jobs, variables, or templates across pipeline stages. These constructs are common in real-world pipelines and are currently not supported in the editor. Enhancing the DSL and editor to recognize and validate cross-references will allow the tool to handle more complex workflows and improve transformation accuracy.

Third, further transformations can be developed between additional supported dialects, including BitBucket, Bamboo, CircleCI, and GitLab. Also, determining the exact impact of the non-transformed concepts should be done on the GitHub Actions and Azure DevOps platforms in future work.

Fourth, the development of a visual or web-based interface could improve accessibility and usability of the DSL and transformation features. Such a tool would allow practitioners to model, validate, and migrate pipelines within familiar development environments, without needing to interact directly with the underlying DSL grammar or transformation code.

Furthermore, the scraping and parsing processes developed in this thesis can be expanded into a scalable pipeline for collecting pipeline specifications from open-source platforms. These datasets can be reused in future empirical studies focused on CI/CD practices, transformation correctness, pipeline anti-pattern detection, and platform-specific evolution.

Finally, future work may build on frequently observed pipeline patterns identified in the heterogeneous dataset. These recurring patterns can be introduced as higher-level abstractions or reusable constructs in the DSL, reducing redundancy and improving expressiveness. Once integrated, it is valuable to explore whether existing pipeline specifications can be automatically migrated to adopt these abstractions. This extension has the potential to streamline pipeline creation and improve maintainability.

Appendix A

Xtext Grammar

```
/*********************************************************************
 * Xtext Grammar for Multi-Platform CI/CD Pipeline Language
 *
 * Author: Manas Bedekar
 * Date: December 2024
 *
 * Description:
 * This Xtext grammar defines a language that abstracts automation
 * pipelines for multiple CI/CD platforms such as GitHub Actions,
 * Azure DevOps, GitLab, Bitbucket, Bamboo, CircleCI, etc. The
 * language enables unified analysis of pipelines across different
 * platforms, supporting multi-platform dialects in a streamlined
 * environment. The grammar provides support for defining events,
 * configurations, stages, jobs, scripts, resources, and more.
 *
 * Purpose:
 * The goal is to enable cross-platform pipeline analysis, allowing users
 * to define pipelines in a consistent way across different platforms,
 * abstracting away the specifics of each individual CI/CD environment.
 * This is beneficial for tool developers, researchers, and organizations
 * who need a unified  representation to simplify the design and analysis
   of CI/CD pipelines.
 *
 * License:
 * This grammar is intended for educational and research purposes and
 * should be modified and used accordingly. Please reach out to the author
 * for inquiries about reuse or adaptation for commercial use.
 *

 * Contact Information:
 * Manas Bedekar
 * Email: manas.bedekar@mail.mcgill.ca, manasbedekar002@gmail.com
 * Organization: McGill University, Montreal
 *********************************************************************/

```

```
35 grammar ca.mcgill.devops.pipeline.Pipeline with org.eclipse.xtext.common.
    Terminals
36 hidden(WS, ML_COMMENT, SL_COMMENT)
37 import "http://www.eclipse.org/emf/2002/Ecore" as ecore
38 generate pipeline "http://www.mcgill.ca/devops/pipeline/Pipeline"
39
40
41 /* Pipeline Rule Declarations */
42 /*------------------------Basic Configurations--------------------------*/
43 Pipeline:
44   // GitHub Actions Config
45   (pipelineConfiguration=Configuration)
46
47   // Event Config
48   (('on:' (BEGIN triggers+=Event+ END)? ((triggers+=Event) | ('['triggers
    +=Event (',' triggers+=Event)*']'))?) | (triggers+=Event*))
49
50   // Job and Stage Config
51   (jobOrStageVar=('stages:'|'jobs:')
52       ((stages+=Stage*)|(BEGIN (jobs+=Job*) END))
53     )
54 ;
55
56 Configuration:
57   ('name:' name=UnquotedString)? &
58   ('run-name:'runName=UnquotedString)? &
59   ('defaults:' BEGIN 'run:' BEGIN ('shell:'defaultShellValue=(STRING|ID))
    ?&('working-directory:'defaultWDValue=(STRING|ID))? END END)? &
60   ('permissions:'((allPermission=ID)|(BEGIN (indPermissions+=
    IndPermissionValue*) END))) ? &
61   ('concurrency:'((BEGIN ('group:' group=(ID|COMPLEX_EXPRESSION)) ('cancel
    -in-progress:' cancelConcurrence=(ID|COMPLEX_EXPRESSION)) END) | (
    concurrency=(AnyData|ArrayList))))? &
62
63   // Azure DevOps Config
64   ('pool:'(BEGIN ('name:'vmName=ID)('vmImage:'vmImage=(ID|STRING))?&('
    demands:'vmDemands=(AnyData|HyphenValues))? END))? &
65   ('resources:' (BEGIN (resources+=Resource*) END))? &
66
67   // CircleCI Config
68   ('version:' version=VERSION)? &
69
70   // Common for GitHub Actions, Azure Devops, GitLab, CircleCI
71   (variableKW=('variables:'|'env:') (BEGIN (pplVariables+=Variable*) END))
     ? &
72   ('parameters:'(('-'parameterValues+=ParameterValue)*)) ? &
73   (extendOrInclude=('extends:'|'include:')(BEGIN (extendedParameters+=
    ParameterValue+) END))?
74 ;
75
76 Variable:
77   ((variableKey=ID':')(variableValue=(ID|STRING|COMPLEX_EXPRESSION))|
78   (('-'('name:'|'group:')name=AnyData)(BEGIN ('value:'value=AnyData)('
    readonly:'readonly=ID)? END)?))|
```

```
  (BEGIN
    name=ID':'
    (BEGIN
      variables+=KeyValue+
    END)
  END)
;

Resource:
  (resourceName=('builds:'|'containers:'|'pipelines:'|'repositories:'|'webhooks:'|'packages:'))
  (('-'resourceValues+=ParameterValue)*|(BEGIN('-'resourceValues+=ParameterValue)*END))
;

/*-------------------------Event Configurations------------------------*/
Event:
  (
    ((name=PipelineEventKeyword)|(simpleEvent=(ID)(':')?))('none')?(includedBranches+=Branch*)
    (BEGIN
      ( (('types:')(activities+=Activity))? &
        ('batch:' batch=ID)? &
        ('autoCancel:' autoCancel=ID)? &
        ('workflows:' workflows=(ArrayList|HyphenValues|ID))? &
        ('drafts:' drafts=ID)? &
        (('branches:')(((includedBranches+=Branch*)|('['includedBranches+=Branch (',' includedBranches+=Branch)*']'))|(BEGIN (includedBranches+=Branch*) (('include:')(includedBranches+=Branch*))? (('exclude:')(excludedBranches+=Branch*))? END)))? &
        (('branches-ignore:')(((excludedBranches+=Branch*)|('['excludedBranches+=Branch (',' excludedBranches+=Branch)*']'))| (BEGIN (excludedBranches+=Branch*) END)))? &
        (('paths:')(includedPaths=HyphenValues)? ((BEGIN (('include:')(includedPaths=HyphenValues))? (('exclude:')(excludedPaths=HyphenValues))? END))?)? &
        (('paths-ignore:')(excludedPaths=HyphenValues))? &
        (('tags:')(includedTags=HyphenValues)? ((BEGIN (('include:')(includedTags=HyphenValues))? (('exclude:')(excludedTags=HyphenValues))? END))?)? &
        (('tags-ignore:')(excludedTags=HyphenValues))?
      )
    END)?
  )|
  (
    ((scheduleNameKW=('schedules:'|'schedule:'))(triggerSchedules+=TriggerSchedule*))|
    ((workflowTypeKW=('workflow_call:'|'workflow_dispatch:'))(BEGIN workFlowActivities+=Activity* END))
  )
;

TriggerSchedule:
```

```
  (('-''cron:' cronValue=STRING) | (BEGIN ('-''cron:' cronValue=STRING) END))
  (BEGIN
    ('displayName:' displayName=UnquotedString)?
    ('branches:'(BEGIN (('include:')(includedBranches+=Branch*))? (('exclude:')(excludedBranches+=Branch*))? END))
    (('batch:' batch=ID) ? &
    ('always:' always=ID)?)
  END)?
;

Branch:
  (('-')? name=(ID|STRING))
;

Activity:
  (name=(AnyData|HyphenValues|ArrayList))|
  (
    name=('inputs:'|'outputs:'|'secrets:')
    //(BEGIN (activities+=WorkFlowKeyValue+) END)
    (BEGIN activities+=ParameterValue+ END)
  )
;

/*--------------------Stage & Job Configurations------------------------*/
Stage:
  (('-''stage:' name=AnyData))
  (BEGIN
    ('displayName:' displayName=STRING)? &
    ('pool:'((poolValue=AnyData)? (BEGIN ('name:'vmName=ID)?('vmImage:'vmImage=(ID|STRING))?('demands:'vmDemands=(AnyData|HyphenValues))? END)?))? &
    ('condition:'(conditionValue=AnyData))? &
    ('isSkippable:' isSkippable=ID)? &
    ('variables:' (BEGIN (stageVariables+=Variable)* END))? &
    ('dependsOn:' (dependsOn+=[Stage])*)?
    ('jobs:'(jobs+=Job)*)
  END)?
;

Job:
  (('-' ('job:'|'deployment:'))? name=(ID|STRING) (':')?)
  (BEGIN
    //(nameKW=('name:'|'displayName:') name=UnquotedString)? &
    (jobParameterValues+=ParameterValue*)
    (dependKW=('needs:' | 'dependsOn:') ((references+=([Job])|'none') | ('[' references+=[Job] (',' references+=[Job])* ']')))?
    ('steps:'(steps+=Step*))?
  END)
;

/*-------------------------Step Configurations-------------------------*/
Step:
  ('-')(Action | Script)
```

```
;

Script:
  (name=('run:'|'script:'|'bash:'|'pwsh:'|'powershell:'|'checkout:'|'download:'|'getPackage:'|'publish:'|'task:') ((command=AnyData) |
      (('|' | '>-')
      BEGIN
        command=AnyData
      END))
  )
  (BEGIN (subSteps+=(Action|Script))* END)?
;

Action:
  (('name:' actionName=UnquotedString)|
  (
    ((actionKeyword=PipelineKeyword)|(otherKeyword=ID':'))
    (actionValue=(AnyData|HyphenValues))?
  ))
  (BEGIN (subSteps+=(Action|Script))* END)?
;

/*-------------------------Parameter Values-------------------------*/
ParameterValue:
  ((name=PipelineKeyword)|(otherName=(ID|COMPLEX_EXPRESSION)':'))
  (
    (value=(AnyData|ArrayList|HyphenValues)(BEGIN (('-')?subParameters+=ParameterValue)* END)?)|
    ((BEGIN (('-')?subParameters+=ParameterValue)* END)|(('-'subParameters+=ParameterValue)*))
  )
;



/* Defining Enum */

/*-------------------------Event Enum--------------------------------*/
enum PipelineEventKeyword:
    PPL_EVT_PUSH = 'push:' |
    PPL_EVT_PULL_REQUEST = 'pull_request:' |
    PPL_EVT_NONE = 'none' |
    PPL_EVT_REPOSITORY_DISPATCH = 'repository_dispatch:' |
    PPL_EVT_CHECK_RUN = 'check_run:' |
    PPL_EVT_CHECK_SUITE = 'check_suite:' |
    PPL_EVT_CREATE = 'create:' |
    PPL_EVT_DELETE = 'delete:' |
    PPL_EVT_DEPLOYMENT = 'deployment:' |
    PPL_EVT_DEPLOYMENT_STATUS = 'deployment_status:' |
    PPL_EVT_FORK = 'fork:' |
    PPL_EVT_GOLLUM = 'gollum:' |
    PPL_EVT_ISSUE_COMMENT = 'issue_comment:' |
    PPL_EVT_ISSUES = 'issues:' |
    PPL_EVT_LABEL = 'label:' |
```

```
    PPL_EVT_MILESTONE = 'milestone:' |
    PPL_EVT_PAGE_BUILD = 'page_build:' |
    PPL_EVT_PROJECT = 'project:' |
    PPL_EVT_PROJECT_CARD = 'project_card:' |
    PPL_EVT_PROJECT_COLUMN = 'project_column:' |
    PPL_EVT_PUBLIC = 'public:' |
    PPL_EVT_PULL_REQUEST_REVIEW = 'pull_request_review:' |
    PPL_EVT_PULL_REQUEST_REVIEW_COMMENT = 'pull_request_review_comment:' |
    PPL_EVT_PULL_REQUEST_TARGET = 'pull_request_target:' |
    PPL_EVT_REGISTRY_PACKAGE = 'registry_package:' |
    PPL_EVT_RELEASE = 'release:' |
    PPL_EVT_STATUS = 'status:' |
    PPL_EVT_WATCH = 'watch:' |
    PPL_EVT_WORKFLOW_CALL = 'workflow_call:' |
    PPL_EVT_WORKFLOW_RUN = 'workflow_run:' |
    PPL_EVT_TRIGGER = 'trigger:' |
    PPL_EVT_PR = 'pr:' |
    PPL_EVT_BRANCHES = 'branches:' |
    PPL_EVT_TAGS = 'tags:' |
    PPL_EVT_PATHS = 'paths:' |
    PPL_EVT_AUTO_CANCEL = 'autoCancel:' |
    PPL_EVT_DRAFTS = 'drafts:' |
    PPL_EVT_ONLY = 'only:' |
    PPL_EVT_EXCEPT = 'except:' |
    PPL_EVT_RULES = 'rules:' |
    PPL_EVT_WORKFLOW_RULES = 'workflow.rules:' |
    PPL_EVT_PIPELINE_SCHEDULE = 'pipeline_schedule:' |
    PPL_EVT_MERGE_REQUESTS = 'merge_requests:' |
    PPL_EVT_CHANGES = 'changes:' |
    PPL_EVT_PIPELINES = 'pipelines:' |
    PPL_EVT_DEFAULT = 'default:' |
    PPL_EVT_PULL_REQUESTS = 'pull-requests:' |
    PPL_EVT_CUSTOM = 'custom:' |
    PPL_EVT_CONDITIONS = 'conditions:' |
    PPL_EVT_BRANCH_DELETED = 'bitbucket-branch-deleted:' |
    PPL_EVT_BRANCH_CREATED = 'bitbucket-branch-created:' |
    PPL_EVT_TAG_CREATED = 'bitbucket-tag-created:' |
    PPL_EVT_TAG_DELETED = 'bitbucket-tag-deleted:' |
    PPL_EVT_REPOSITORY_TRIGGER = 'repository:' |
    PPL_EVT_SCHEDULED_TRIGGER = 'scheduled:' |
    PPL_EVT_REMOTE_TRIGGER = 'remote:' |
    PPL_EVT_BITBUCKET_TRIGGER = 'bitbucket:' |
    PPL_EVT_PLAN_BRANCH = 'planBranch:' |
    PPL_EVT_CRON = 'cron:' |
    PPL_EVT_NOTIFICATIONS = 'notifications:' |
    PPL_EVT_LOCK_BEHAVIOR = 'lockBehavior:'
;

/*-------------------------Pipeline Enum-------------------------------*/
enum PipelineKeyword:
    PPL_KW_NAME = 'name:' |
    PPL_KW_RUN_NAME = 'run-name:' |
    PPL_KW_ON = 'on:' |
    PPL_KW_DEFAULTS = 'defaults:' |
```

```
    PPL_KW_WORKING_DIRECTORY = 'working-directory:' |
    PPL_KW_ENV = 'env:' |
    PPL_KW_PERMISSION = 'permission:' |
    PPL_KW_CONCURRENCY = 'concurrency:' |
    PPL_KW_GROUP = 'group:' |
    PPL_KW_CANCEL_IN_PROGRESS = 'cancel-in-progress:' |
    PPL_KW_POOL = 'pool:' |
    PPL_KW_VM_IMAGE = 'vmImage:' |
    PPL_KW_DEMANDS = 'demands:' |
    PPL_KW_RESOURCES = 'resources:' |
    PPL_KW_VERSION = 'version:' |
    PPL_KW_VARIABLES = 'variables:' |
    PPL_KW_PARAMETERS = 'parameters:' |
    PPL_KW_EXTENDS = 'extends:' |
    PPL_KW_INCLUDE = 'include:' |
    PPL_KW_TYPES = 'types:' |
    PPL_KW_BATCH = 'batch:' |
    PPL_KW_STATUS = 'status:' |
    PPL_KW_AUTO_CANCEL = 'autoCancel:' |
    PPL_KW_WORKFLOWS = 'workflows:' |
    PPL_KW_DRAFTS = 'drafts:' |
    PPL_KW_BRANCHES = 'branches:' |
    PPL_KW_BRANCHES_IGNORE = 'branches-ignore:' |
    PPL_KW_PATHS = 'paths:' |
    PPL_KW_PATHS_IGNORE = 'paths-ignore:' |
    PPL_KW_TAGS = 'tags:' |
    PPL_KW_TAGS_IGNORE = 'tags-ignore:' |
    PPL_KW_SCHEDULES = 'schedules:' |
    PPL_KW_SCHEDULE = 'schedule:' |
    PPL_KW_WORKFLOW_CALL = 'workflow_call:' |
    PPL_KW_WORKFLOW_DISPATCH = 'workflow_dispatch:' |
    PPL_KW_CRON = 'cron:' |
    PPL_KW_DISPLAY_NAME = 'displayName:' |
    PPL_KW_ALWAYS = 'always:' |
    PPL_KW_STAGE = 'stage:' |
    PPL_KW_CONDITION = 'condition:' |
    PPL_KW_IS_SKIPPABLE = 'isSkippable:' |
    PPL_KW_JOB = 'job:' |
    PPL_KW_DEPLOYMENT = 'deployment:' |
    PPL_KW_VALUE = 'value:' |
    PPL_KW_READONLY = 'readonly:' |
    PPL_KW_BUILDS = 'builds:' |
    PPL_KW_CONTAINERS = 'containers:' |
    PPL_KW_PIPELINES = 'pipelines:' |
    PPL_KW_REPOSITORIES = 'repositories:' |
    PPL_KW_WEBHOOKS = 'webhooks:' |
    PPL_KW_PACKAGES = 'packages:' |
    PPL_KW_INPUTS = 'inputs:' |
    PPL_KW_OUTPUTS = 'outputs:' |
    PPL_KW_SECRETS = 'secrets:' |
    PPL_KW_DESCRIPTION = 'description:' |
    PPL_KW_REQUIRED = 'required:' |
    PPL_KW_DEFAULT = 'default:' |
    PPL_KW_TYPE = 'type:' |
```

```
    PPL_KW_OPTIONS = 'options:' |
    PPL_KW_IF = 'if:' |
    PPL_KW_RUNS_ON = 'runs-on:' |
    PPL_KW_STRATEGY = 'strategy:' |
    PPL_KW_MATRIX = 'matrix:' |
    PPL_KW_CONTAINER = 'container:' |
    PPL_KW_SERVICES = 'services:' |
    PPL_KW_USES = 'uses:' |
    PPL_KW_WITH = 'with:' |
    PPL_KW_TIMEOUT_MINUTES = 'timeout-minutes:' |
    PPL_KW_CONTINUE_ON_ERROR = 'continue-on-error:' |
    PPL_KW_ENVIRONMENT = 'environment:' |
    PPL_KW_ID = 'id:' |
    PPL_KW_PERMISSIONS = 'permissions:' |
    PPL_KW_TIMEOUTS = 'timeouts:' |
    PPL_KW_DEFAULTS_RUN = 'defaults.run:' |
    PPL_KW_PUSH = 'push:' |
    PPL_KW_PULL_REQUEST = 'pull_request:' |
    PPL_KW_REPOSITORY_DISPATCH = 'repository_dispatch:' |
    PPL_KW_WORKFLOW_RUN = 'workflow_run:' |
    PPL_KW_FILTERS = 'filters:' |
    PPL_KW_EXCLUDE = 'exclude:' |
    PPL_KW_FAIL_FAST = 'fail-fast:' |
    PPL_KW_MAX_PARALLEL = 'max-parallel:' |
    PPL_KW_CONTAINER_IMAGE = 'container.image:' |
    PPL_KW_CREDENTIALS = 'credentials:' |
    PPL_KW_PORTS = 'ports:' |
    PPL_KW_INHERIT = 'inherit:' |
    PPL_KW_TRIGGER = 'trigger:' |
    PPL_KW_TEMPLATE = 'template:' |
    PPL_KW_WORKSPACE = 'workspace:' |
    PPL_KW_PIPELINE = 'pipeline:' |
    PPL_KW_TIMEOUT_IN_MINUTES = 'timeoutInMinutes:' |
    PPL_KW_CONTINUE_ON_ERR = 'continueOnError:' |
    PPL_KW_RESOURCE_TYPE = 'resourceType:' |
    PPL_KW_RESOURCE_NAME = 'resourceName:' |
    PPL_KW_STAGES = 'stages:' |
    PPL_KW_ONLY = 'only:' |
    PPL_KW_EXCEPT = 'except:' |
    PPL_KW_IMAGE = 'image:' |
    PPL_KW_CACHE = 'cache:' |
    PPL_KW_ARTIFACTS = 'artifacts:' |
    PPL_KW_RETRIES = 'retries:' |
    PPL_KW_RULES = 'rules:' |
    PPL_KW_BEFORE_SCRIPT = 'before_script:' |
    PPL_KW_AFTER_SCRIPT = 'after_script:' |
    PPL_KW_TIMEOUT = 'timeout:' |
    PPL_KW_WHEN = 'when:' |
    PPL_KW_ALLOW_FAILURE = 'allow_failure:' |
    PPL_KW_DEPENDENCIES = 'dependencies:' |
    PPL_KW_PULL_REQUESTS = 'pull-requests:' |
    PPL_KW_STEP = 'step:' |
    PPL_KW_DEFINITIONS = 'definitions:' |
    PPL_KW_CACHES = 'caches:' |
```

```
    PPL_KW_PARALLEL = 'parallel:' |
    PPL_KW_CONDITIONS = 'conditions:' |
    PPL_KW_MAX_TIME = 'max-time:' |
    PPL_KW_PROJECT = 'project:' |
    PPL_KW_PLAN = 'plan:' |
    PPL_KW_TASKS = 'tasks:' |
    PPL_KW_REQUIREMENTS = 'requirements:' |
    PPL_KW_FINAL = 'final:' |
    PPL_KW_JDK = 'jdk:' |
    PPL_KW_LABELS = 'labels:' |
    PPL_KW_REPOSITORY = 'repository:' |
    PPL_KW_RELEASE = 'release:' |
    PPL_KW_DOCKER = 'docker:' |
    PPL_KW_AGENT = 'agent:' |
    PPL_KW_QUEUE = 'queue:' |
    PPL_KW_SNAPSHOT = 'snapshot:' |
    PPL_KW_POLICY = 'policy:' |
    PPL_KW_APPROVALS = 'approvals:' |
    PPL_KW_PREBUILD = 'preBuild:' |
    PPL_KW_POSTBUILD = 'postBuild:' |
    PPL_KW_SSH = 'ssh:' |
    PPL_KW_AUTH = 'auth:' |
    PPL_KW_STORAGE = 'storage:' |
    PPL_KW_ENDPOINTS = 'endpoints:' |
    PPL_KW_COMMAND = 'command:' |
    PPL_KW_PROVISIONER = 'provisioner:' |
    PPL_KW_TRIGGER_RULES = 'triggerRules:' |
    PPL_KW_RUN_POLICY = 'runPolicy:' |
    PPL_KW_QUEUED = 'queued:' |
    PPL_KW_NOTIFICATIONS = 'notifications:' |
    PPL_KW_DEPLOY_TRIGGER = 'deployTrigger:' |
    PPL_KW_RELEASE_TRIGGER = 'releaseTrigger:' |
    PPL_KW_LOCK_BEHAVIOR = 'lockBehavior:' |
    PPL_KW_REVIEW_STAGE = 'reviewStage:'
;


/* Defining Datatypes */
AnyData:
    STRING |Number | FloatingPoint | SimpleWord | COMPLEX_EXPRESSION |
   VERSION | HEX_INT;

WorkFlowKeyValue:
  (ID':')(BEGIN (('description:'|'required:'|'default:'|'type:'|'options
   :')(AnyData|HyphenValues))+ END)
;
KeyValue:
  ID':' (AnyData|HyphenValues);

IndPermissionValue:
  ('actions:'| 'attestations:'| 'checks:'| 'contents:'| 'deployments:'| '
   id-token:' | 'issues:'|'discussions:'|'packages:'|'pull-requests:'|'
   repository-projects:'|'security-events:'|'statuses:')
  ('read'|'write'|'none');
```

```
430
431 UnquotedString:
432   ID (ID|INT|COMPLEX_EXPRESSION|VERSION)*;
433
434 HyphenValues:
435   ('-' AnyData)+ |
436   (BEGIN ('-' AnyData)+ END);
437
438 Number:
439     INT;
440
441 FloatingPoint:
442     FLOAT_LITERAL;
443
444 SimpleWord:
445     ID;
446
447 ArrayList:
448   '['(AnyData (',' AnyData)*)?']';
449
450
451 /* Defining Terminal Datatypes */
452 terminal COMPLEX_EXPRESSION:
453   (((('a'..'z' | 'A'..'Z')('a'..'z' | 'A'..'Z' | '-')*)? '${{' -> '}}')('-$
    {{' -> '}}')*;
454
455 terminal HEX_INT returns ecore::EInt: '0x' ('0'..'9' | 'a'..'f' | 'A'..'F
    ')+;
456
457 terminal FLOAT_LITERAL returns ecore::EFloat:
458     ('-')?(INT '.' INT);
459
460 terminal VERSION:
461   INT ('.' (INT|ID))+;
462
463 terminal BEGIN: 'synthetic:BEGIN';
464
465 terminal END: 'synthetic:END';
466
467 @Override
468 terminal ID:
469     ('a'..'z' | 'A'..'Z' | '0'..'9' | '_' | '/' | '@' | '-' | '.' | '\\' |
     '&' | '(' | ')') ('a'..'z' | 'A'..'Z' | '0'..'9' | '_' | '/' | '@' |
    '&' |'-' | '.' | '\\' | '(' | ')' | '|' | '$' )*;
470
471 @Override
472 terminal ML_COMMENT:
473     '/*' -> '*/';
474
475 @Override
476 terminal SL_COMMENT:
477     '#' !('\n'|'\r')* ('\r'? '\n')?;
```

Listing A.1: Reference Xtext Grammar Code

Bibliography

[1] M. Senapathi, J. Buchan, and H. Osman, "DevOps capabilities, practices, and challenges: Insights from a case study," in *Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering 2018*, pp. 57–67, 2018. doi: 10.1145/3210459.3210465.

[2] W. P. Luz, G. Pinto, and R. Bonifácio, "Adopting DevOps in the real world: A theory, a model, and a case study," *Journal of Systems and Software*, vol. 157, p. 110384, 2019. doi: 10.1016/j.jss.2019.07.083.

[3] C. A. Cois, J. Yankel, and A. Connell, "Modern DevOps: Optimizing software development through effective system interactions," in *2014 IEEE International Professional Communication Conference (IPCC)*, pp. 1–7, IEEE, 2014. doi: 10.1109/IPCC.2014.7020388.

[4] M. S. Khan, A. W. Khan, F. Khan, M. A. Khan, and T. K. Whangbo, "Critical challenges to adopt DevOps culture in software organizations: A systematic review," *IEEE Access*, vol. 10, pp. 14339–14349, 2022. doi: 10.1109/ACCESS.2022.3145970.

[5] C. Pulgar, "Eat your own DevOps: a model driven approach to justify continuous integration pipelines," in *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems: Companion Proceedings*, pp. 225–228, 2022. doi: 10.1145/3550356.3552395.

[6] M. M. Bedekar and G. Mussbacher, "A multi-platform specification language and dataset for the analysis of DevOps pipelines," in *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems (MODELS Companion 2024), ACM*, pp. 264–274, 2024. doi: 10.1145/3652620.3686247.

[7] M. Z. Toh, S. Sahibuddin, and M. N. Mahrin, "Adoption issues in DevOps from the perspective of continuous delivery pipeline," in *Proceedings of the 2019 8th International Conference on Software and Computer Applications*, pp. 173–177, 2019. doi: 10.1145/3316615.3316619.

[8] F. M. Erich, C. Amrit, and M. Daneva, "A qualitative study of DevOps usage in practice," *Journal of Software: Evolution and Process*, vol. 29, no. 6, p. e1885, 2017. doi: 10.1002/smr.1885.

[9] Y. Wang, M. Pyhäjärvi, and M. V. Mäntylä, "Test automation process improvement in a DevOps team: experience report," in *2020 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, pp. 314–321, IEEE, 2020. doi: 10.1109/ICSTW50294.2020.00057.

[10] Research and Markets, "DevOps market: Global industry trends, share, size, growth, opportunity and forecast 2023-2028." https://www.researchandmarkets.com/reports/5732375/devops-market-global-industry-trends-share, 2023. Accessed on 2023-10-20.

[11] L. E. Lwakatare, T. Kilamo, T. Karvonen, T. Sauvola, V. Heikkilä, J. Itkonen, P. Kuvaja, T. Mikkonen, M. Oivo, and C. Lassenius, "DevOps in practice: A multiple case study of five companies," *Information and Software Technology*, vol. 114, pp. 217–230, 2019. doi: 10.1016/j.infsof.2019.06.010.

[12] L. Bass, I. Weber, and L. Zhu, *DevOps: A software architect's perspective*. Addison-Wesley Professional, 2015.

[13] G. Bou Ghantous and A. Gill, "DevOps: Concepts, practices, tools, benefits and challenges," *Pacific Asia Conference on Information Systems (PACIS), https://aisel.aisnet.org/pacis2017/96*, 2017.

[14] T. Offerman, R. Blinde, C. J. Stettina, and J. Visser, "A study of adoption and effects of DevOps practices," in *2022 IEEE 28th International Conference on Engineering, Technology and Innovation (ICE/ITMC) & 31st International Association For Management of Technology (IAMOT) Joint Conference*, pp. 1–9, IEEE, 2022. doi: 10.1109/ICE/ITMC-IAMOT55089.2022.10033313.

[15] J. Heiskanen, *Comparing DevOps processes in different cloud platforms from a software developers perspective*. PhD thesis, Karelia University Of Applied Sciences, 2020.

[16] M. A. Akbar, A. A. Khan, S. Mahmood, and S. Rafi, "A vision of DevOps requirements change management standardization," in *2022 IEEE 22nd International Conference on Software Quality, Reliability, and Security Companion (QRS-C)*, pp. 587–592, IEEE, 2022. doi: 10.1109/QRS-C57518.2022.00094.

[17] M. Shahin, M. A. Babar, and L. Zhu, "Continuous integration, delivery and deployment: a systematic review on approaches, tools, challenges and practices," *IEEE Access*, vol. 5, pp. 3909–3943, 2017. doi: 10.1109/ACCESS.2017.2685629.

[18] B. Fitzgerald and K.-J. Stol, "Continuous software engineering: A roadmap and agenda," *Journal of Systems and Software*, vol. 123, pp. 176–189, 2017. doi: 10.1016/j.jss.2015.06.063.

[19] R. Lefeuvre, J. Galasso, B. Combemale, H. Sahraoui, and S. Zacchiroli, "Fingerprinting and building large reproducible datasets," in *Proceedings of the 2023 ACM Conference on Reproducibility and Replicability*, pp. 27–36, 2023. doi: 10.1145/3589806.3600043.

[20] R. Di Cosmo and S. Zacchiroli, "Software heritage: Why and how to preserve software source code," in *iPRES 2017-14th International Conference on Digital Preservation*, pp. 1–10, 2017. https://hal.science/hal-01590958. Accessed on 2025-05-19.

[21] E. Kalliamvakou, G. Gousios, K. Blincoe, L. Singer, D. M. German, and D. Damian, "The promises and perils of mining GitHub," in *Proceedings of the 11th Working Conference on Mining Software Repositories*, MSR 2014, (New York, NY, USA), p. 92–101, Association for Computing Machinery, 2014. doi: 10.1145/2597073.2597074.

[22] G. Gousios, M. Pinzger, and A. v. Deursen, "An exploratory study of the pull-based software development model," in *Proceedings of the 36th International Conference on Software Engineering*, pp. 345–355, 2014. doi: 10.1145/2568225.2568260.

[23] T. F. Bissyandé, F. Thung, D. Lo, L. Jiang, and L. Réveillere, "Popularity, interoperability, and impact of programming languages in 100,000 open source projects," in *2013 IEEE 37th Annual Computer Software and Applications Conference*, pp. 303–312, IEEE, 2013. doi: 10.1109/COMPSAC.2013.55.

[24] C. Bird, P. C. Rigby, E. T. Barr, D. J. Hamilton, D. M. German, and P. Devanbu, "The promises and perils of mining git," in *2009 6th IEEE International Working Conference on Mining Software Repositories*, pp. 1–10, IEEE, 2009. doi: 10.1109/MSR.2009.5069475.

[25] G. Gousios, A. Zaidman, M.-A. Storey, and A. Van Deursen, "Work practices and challenges in pull-based development: The integrator's perspective," in *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, vol. 1, pp. 358–368, IEEE, 2015. doi: 10.1109/ICSE.2015.55.

[26] G. Gousios, "The ghtorent dataset and tool suite," in *2013 10th Working Conference on Mining Software Repositories (MSR)*, pp. 233–236, IEEE, 2013. doi: 10.1109/MSR.2013.6624034.

[27] R. Di Cosmo, "Archiving and referencing source code with Software Heritage," in *Mathematical Software–ICMS 2020: 7th International Conference, Braunschweig, Germany, July 13–16, 2020, Proceedings 7*, pp. 362–373, Springer, 2020. doi: 10.1007/978-3-030-52200-1_36.

[28] T. Kosar, S. Bohra, and M. Mernik, "Domain-specific languages: A systematic mapping study," *Information and Software Technology*, vol. 71, pp. 77–91, 2016. doi: 10.1016/j.infsof.2015.11.001.

[29] L. Bettini, *Implementing domain-specific languages with Xtext and Xtend*. Packt Publishing Ltd, 2016.

[30] M. Eysholdt and H. Behrens, "Xtext: implement your language faster than the quick and dirty way," in *Proceedings of the ACM International Conference Companion on Object Criented Programming Systems Languages and Applications Companion*, pp. 307–309, 2010. doi: 10.1145/1869542.1869625.

[31] D. S. Kolovos, R. F. Paige, and F. A. Polack, "The epsilon transformation language," in *Theory and Practice of Model Transformations: First International Conference, ICMT 2008, Zürich, Switzerland, July 1-2, 2008 Proceedings 1*, pp. 46–60, Springer, 2008. doi: 10.1007/978-3-540-69927-9_4.

[32] K. Czarnecki and S. Helsen, "Feature-based survey of model transformation approaches," *IBM systems journal*, vol. 45, no. 3, pp. 621–645, 2006. doi: 10.1147/sj.453.0621.

[33] F. Jouault, F. Allilaire, J. Bézivin, and I. Kurtev, "ATL: A model transformation tool," *Science of computer programming*, vol. 72, no. 1-2, pp. 31–39, 2008. doi: 10.1016/j.scico.2007.08.002.

[34] L. Lúcio, M. Amrani, J. Dingel, L. Lambers, R. Salay, G. M. Selim, E. Syriani, and M. Wimmer, "Model transformation intents and their properties," *Software & systems modeling*, vol. 15, pp. 647–684, 2016. doi: 10.1007/s10270-014-0429-x.

[35] T. Mens and P. Van Gorp, "A taxonomy of model transformation," *Electronic notes in theoretical computer science*, vol. 152, pp. 125–142, 2006. doi: 10.1016/j.entcs.2005.10.021.

[36] N. Kahani, M. Bagherzadeh, J. R. Cordy, J. Dingel, and D. Varró, "Survey and classification of model transformation tools," *Software & Systems Modeling*, vol. 18, pp. 2361–2397, 2019. doi: 10.1007/s10270-018-0665-6.

[37] M. Amrani, L. Lucio, G. Selim, B. Combemale, J. Dingel, H. Vangheluwe, Y. Le Traon, and J. R. Cordy, "A tridimensional approach for studying the formal verification of model transformations," in *2012 IEEE Fifth International Conference on Software Testing, Verification and Validation*, pp. 921–928, IEEE, 2012. doi: 10.1109/ICST.2012.197.

[38] Software Heritage. https://www.softwareheritage.org/. Accessed on 2024-03-13.

[39] J. Guerrero, K. Zúniga, C. Certuche, and C. Pardo, "A systematic mapping study about DevOps," *Journal de Ciencia e Ingeniería*, vol. 12, no. 1, pp. 48–62, 2020. doi: 10.46571/JCI.2020.1.5.

[40] G. Kim, J. Humble, P. Debois, J. Willis, and N. Forsgren, *The DevOps handbook: How to create world-class agility, reliability, & security in technology organizations.* 2nd ed., IT Revolution, 2021.

[41] M. Rajkumar, A. K. Pole, V. S. Adige, and P. Mahanta, "DevOps culture and its impact on cloud delivery and software development," in *2016 International Conference on Advances in Computing, Communication, & Automation (ICACCA)(Spring)*, pp. 1–6, IEEE, 2016. doi: 10.1109/ICACCA.2016.7578902.

[42] T. Dingsøyr and C. Lassenius, "Emerging themes in agile software development: Introduction to the special section on continuous value delivery," *Information and Software Technology*, vol. 77, pp. 56–60, 2016. doi: 10.1016/j.infsof.2016.04.018.

[43] M. Krey, "DevOps adoption: challenges & barriers," *55th Hawaii International Conference on System Sciences (HICSS), virtual, 3–7 January 2022,University of Hawai'i at Manoa*, vol. 114, p. 7297–7309, 2022. doi: 10.24251/HICSS.2022.877.

[44] L. C. Kats and E. Visser, "The spoofax language workbench: rules for declarative specification of languages and ides," in *Proceedings of the ACM International Conference on Object Oriented Programming Systems Languages and Applications*, pp. 444–463, ACM, 2010. doi: 10.1145/1869459.1869497.

[45] F. Heidenreich, J. Johannes, S. Karol, M. Seifert, and C. Wende, "Derivation and refinement of textual syntax for models," in *European Conference on Model Driven Architecture-Foundations and Applications*, pp. 114–129, Springer, 2009. doi: 10.1007/978-3-642-02674-4_9.

[46] F. Jouault, J. Bézivin, and I. Kurtev, "TCS: a dsl for the specification of textual concrete syntaxes in model engineering," in *Proceedings of the 5th International Conference on Generative Programming and Component Engineering*, pp. 249–254, ACM, 2006. doi: 10.1145/1173706.1173744.

[47] M. Fowler, "A language workbench in action - MPS." `https://martinfowler.com/articles/mpsAgree.html`, 2008.

[48] I. Dejanović, R. Vaderna, G. Milosavljević, and Ž. Vuković, "Textx: a python tool for domain-specific languages implementation," *Knowledge-based systems*, vol. 115, pp. 1–4, 2017. doi: 10.1016/j.knosys.2016.10.023.

[49] L. Bettini, "A DSL for writing type systems for Xtext languages," in *Proceedings of the 9th International Conference on the Principles and Practice of Programming in Java*, pp. 31–40, ACM, 2011. doi: 10.1145/2093157.2093163.

[50] D. Li, X. Li, and V. Stolz, "Qvt-based model transformation using xslt," *ACM SIGSOFT Software Engineering Notes*, vol. 36, no. 1, pp. 1–8, 2011. doi: 10.1145/1921532.1921563.

[51] E. Kindler and R. Wagner, "Triple graph grammars: Concepts, extensions, implementations, and application scenarios," tech. rep., Tech. Rep. TR-

ri-07-284, Department of Computer Science, University of Paderborn, 2007. https://www.researchgate.net/publication/267936590. Accessed on 2025-05-23.

[52] S. Kolahdouz-Rahimi, K. Lano, S. Pillay, J. Troya, and P. Van Gorp, "Evaluation of model transformation approaches for model refactoring," *Science of Computer Programming*, vol. 85, pp. 5–40, 2014. doi: 10.1016/j.scico.2013.07.013.

[53] M. Van Amstel, S. Bosems, I. Kurtev, and L. Ferreira Pires, "Performance in model transformations: experiments with ATL and QVT," in *International Conference on Theory and Practice of Model Transformations*, pp. 198–212, Springer, 2011. doi: 10.1007/978-3-642-21732-6_14.

[54] J. Sorgalla, P. Wizenty, F. Rademacher, S. Sachweh, and A. Zündorf, "Applying model-driven engineering to stimulate the adoption of DevOps processes in small and medium-sized development organizations: the case for microservice architecture," *SN Computer Science*, vol. 2, no. 459, 2021. doi: 10.1007/s42979-021-00825-z.